Techniques and Classifications in *Seqbuild*'s Pseudo-Random Number Generation

Richard Pham

September 2026

Contents

1 Introduction **3**

2 Uncovering the Determinism **4**

3 PRNG Classification and Computation Overview **8**

4 A Briefing Over Application of Theoretical Mathematics **15**

5 Four Randomness Tests **21**
5.1 The Qual Test 21
5.2 The Chain Test 24
5.3 The PRNG I/O Test 25
5.3.1 Variants on the PRNG I/O Test 27
5.4 The Cycle Test 27
5.5 A Characteristic Trait of Pseudo-Random Number Generators . 29

6 Weak Cases of Phenotypic Security **31**

7 Demonstrations with Randomness Tests **32**
7.1 Measuring Randomness on Classical Linear Congruential Generators 32
7.2 Basic Form-Fitters and Randomness 34
7.2.1 Description of Mathematical Structure 34
7.2.2 Qual and Chain Tests on Mdrgen 36
7.2.3 Cycle Test on Optri 43
7.2.4 Qual Test on Qval 47
7.3 Qual and Cycle Tests on Auto-Correcting Generators 49
7.3.1 Description of Mathematical Structure 49
7.3.2 Cycle Test on Gg Generator of Selected LCGs 52
7.3.3 Comparing Randomness Between Different Shadow Generator Modes 56

7.4 Auto-Generation of PRNGs 59
7.5 Discrepancies in Comparative Randomness 60
7.6 Tests on Various Auto-Generated *Seqbuild* PRNGs 65

8 Considering the Cryptanalyst **71**
8.1 Potential Entropy of a Pseudo-Random Number Generator . . . 71
8.2 Complicating the Image 72
8.3 When the Complexity Fails 73
8.4 PRNG Phenotypic Objective 76
8.5 One Approach to PRNG Emulation 78
8.6 Signature Patterns in PRNGs 79
8.7 Knowing When to Change 81

1 Introduction

As of this time in writing, the *Seqbuild* pseudo-random number generator (PRNG) system is equipped with 21 PRNG structures. The paper, *Some Primitive Structures in Seqbuild* [Pha25], provided a partial synopsis of the first batch of generators in the system, when it was still undergoing formative development. There are issues related to the theoretics of number generators in digital product implementation, that are to be discussed in this paper. The primary issue of PRNGs, especially used in security-related matters, revolves around determinism. On a functioning computer devoid of external sources of tampering, it is virtually impossible for it to escape from its programmatic input-output functions. It is, in concise phrasing, a consistent system. There are two classes of variables in a computer's input-output functions that obscure the underlying form U by which the qualities outputted cannot be used to invert back to U: externally-sourced, via another computer (including human beings), and those that are private/protected from observer knowledge and access, where access would allow the observer a non-null amount of control over the output qualities. In the grand scheme of situations, a digital PRNG system that passes randomness metrics is blackbox secured with whitebox vulnerabilities. The outputs cannot be mapped back to the underlying form U, responsible for generating them, from the perspective of an observer that does not have direct access to the host computer's memory. Accessing the host computer's memory would answer the whitebox vulnerability question. In biological terms, this trait of blackbox secured with whitebox vulnerabilities could be considered phenotypic security.

This paper goes into details on the output quality of the PRNGs. *Seqbuild* is equipped with four randomness tests. Chain Test and Qual Test can be used to gauge the "randomness" and act as signature acquisition on the PRNG in question. Signature acquisition also allows for basic difference operation between the test results (signatures) of any two PRNGs. The degrees of their similarity directly relate to their randomness in relation to each other. Signature acquisition is a common task in strategic automation of malware detection [I23], although the concept certainly pertains to testing randomness on pseudo-random number generators, practically a category of computer program that is conceptually at a lower level. The other two tests are PRNG I/O Test and Cycle Test. Most of the PRNGs in *Seqbuild* rely on parameter generators to output values, as per their algorithmic design. These parameter generators are PRNGs themselves, acting as deciders for algorithmic tasks of selection and providing pre-output values for their host generator to process. PRNG I/O Test uses Chain Test and Qual Test to measure how a host PRNG performs in comparison to its parameter PRNG. Cycle Test specifically focuses on the n-gram chunks, each unique one representing a distinct transition state, succeeding unique output values or sequences from a PRNG. The four randomness tests in *Seqbuild* are applied to select cases of the span of the 21 PRNG structures. Results are provided, along with observations on certain qualities that specific PRNG structures are inclined towards.

Theoretics revolving around work factor on pseudo-random number genera-

tion and memory attacks against PRNGs follow. Structural complexity, in the area of algorithm and data structure implementation, is a related consideration for work factor. Classical work by Shannon fixates on measures using bits. Commonly used structures such as trees and chains, rudimentary I/O functions such as linear combinations and polynomial expressions, and generator language are some of the contributing factors towards the *Seqbuild* system's dynamism. The technique of using under-determined systems is fundamental to allowing a PRNG, a deterministic finite automaton, to appear as a non-deterministic finite automaton. Formulae chosen to be solved by the underlying algorithms of the PRNG I/O functions play important roles in PRNG randomness. Some of these were already discussed in *Some Primitive Structures in Seqbuild*, although there was more to be written about their importance.

2 Uncovering the Determinism

There are two main categories of automata in the theory of computability and complexity, non-deterministic and deterministic finite automata.

Definition 2.1 (Deterministic Finite Automaton (DFA))**.** [Sip06, p. 35] A structure comprised of states and a language with elements that allow for transitioning between each state. A deterministic finite automaton has these attributes.

1. A finite set of states, Q.
2. A finite set of elements, Σ comprising a language.
3. A transition function, $\delta : Q \times \Sigma \longrightarrow Q$.
4. The start state q, $q \in Q$.
5. A finite subset A of accept (terminating) states in Q.

These structures are called deterministic because given a state $q \in Q$ and an element $s \in \Sigma$, either a single state is transitioned to if there is a transition state from q with s or no state transition.

A non-deterministic finite automaton is an extension of a DFA.

Definition 2.2 (Non-Deterministic Finite Automaton (NFA))**.** [Sip06, p. 53] A structure comprised of states and a language with elements that allow for transitioning between each state. A non-deterministic finite automaton has these attributes.

1. A finite set of states, Q.
2. A finite set of elements, Σ comprising a language.
3. A transition function, $\delta : Q \times \Sigma \longrightarrow P(Q), P(Q) \subseteq Q$.

4. The start state q, $q \in Q$.

5. A finite subset A of accept (terminating) states in Q.

These structures are called non-deterministic because given a state $q \in Q$ and an element $s \in \Sigma$, there can be multiple states possible to transition to.

Theorem 2.1. *Every non-deterministic finite automaton has an equivalent deterministic finite automaton. [Sip06, p. 55]*

The most basic PRNG in *Seqbuild* is the classical linear congruential generator (LCG).

Definition 2.3 (Linear Congruential Generator (LCG))**.** [Sch23] A deterministic number generator with these qualities.

- A start integer v_0, an additive a, a multiple m, and a modulo d.

The next output v_{t+1} from a linear congruential generator is

$$G(v_{t+1}) = (mv_t + a) \mod d.$$

Most of *Seqbuild*'s other PRNG structures use at least one parameter generator for their activity. A parameter generator is used by its host generator's algorithmic process in producing numbers. There are three main categories of parameter generator use.

1. pre-image values in formats of vectors, matrices, and singletons.

2. output value is converted to a modulated integer i to select the i'th value from a sequence.

3. output values are used as modulated integers to permute the ordering of a sequence.

Below is a basic non-deterministic automaton from a naive observer perspective.

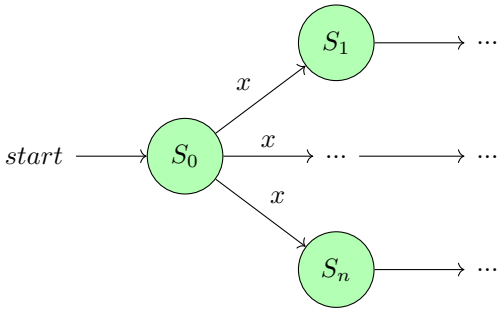


Figure 1: A partial view of an NFA, from the perspective of an observer O, starting from state S_0. O knows parameters x, one of the inputs for the transition to the next state. However, it does not know Q, the hidden input, therefore not knowing which of the S_i is the next state.

This is the abstraction of problems a cryptanalyst is faced with in predicting a PRNG's next output value/s. Some principles for the practice of engineered randomness were strongly considered for the development of *Seqbuild*.

Definition 2.4 (General-Purpose Pseudorandom Generator)**.** [Gol10, p. 12] A deterministic polynomial-time algorithm G serves as a pseudorandom generator if its primary function for outputting values is a stretch function,

$$l : \mathbb{N} \to \mathbb{N},$$

$$l(k) > k; k \in \mathbb{N}.$$

The seed size of G is a positive integer k. Comparing $l(k)$ outputs from generator G with those from a uniform distribution U over the same output space $S(G)$ of G yields negligible difference from a distinguisher function D,

$$|Pr(D(G(l(k)))) - Pr(D(U_{l(k)}[S(G)]))| < 1/p(n);$$

`p any polynomial function with output in` $\mathcal{N}$.

There should be a relatively high difficulty of discovering the primary output function f to a PRNG G. That is, given any contiguous sequence $\overrightarrow{S}$ of output values from G, there is an infinitesimal probability of finding

$$f = f^{-1}(S),$$

such that knowledge of f entails not only the formula/e of variable operands and operators, but also the parameters (hidden state) relevant to f.

Assuming the algorithm of the generator G is publicly known, these parameters that make up G's hidden state can be considered **hardcore predicates**. If the algorithm were secret, then the difficulty of finding f must be at least as hard as in the case of knowledge of it. The below definition provides an ideal statistical rule to follow for implementing cryptographically secure pseudo-random number generators.

Definition 2.5 (Hardcore Predicate)**.** [Gol10, p. 22]A polynomial-time computable predicate $b : X^* \to X$ is called a hardcore of a function f if for every probabilistic polynomial-time sampling algorithm A over distribution of X, every polynomial p of strictly positive integer output, and all sufficiently large n,

$$Pr_{x \in X^*}[A(X^*) = b(X^*)] < \frac{1}{2} + \frac{1}{p(n)}.$$

Hardcore predicates, in PRNG technical application, are on the order of secrecy equivalent to private encryption keys. The term, second-order seed, is used in *Some Primitive Structures in Seqbuild* to define the set of a PRNG's parameters that are mutable input variables during the course of number generation.

Definition 2.6 (Second-Order Seed)**.** A number generating structure $G^{(?)}$ uses a function $G^{(?)}.f$. Structure $G^{(?)}$ has first-order seed q_0 such that

$$\texttt{init}_G(q_0) = G^{(?)}$$

and second-order seed q_1,

$$q_1 = \texttt{parameters}(G^{(?)}.f).$$

This second-order seed is mutable, unlike the finalistic single use of q_0 in initializing $G^{(?)}$ from generator type G.

The choice of words emphasize the mutable quality of the PRNG parameters, whereas the text by Goldreich used "hardcore predicates" predominantly in statistical definitions. A naive observer would have low likelihood of discovering the hardcore PRNG predicates. Instead, they would be relegated to the limited information of the PRNG output values $\overrightarrow{S}$, such as encrypted messages of $\overrightarrow{M}$,

$$\overrightarrow{S} + \overrightarrow{M}.$$

It is reasonable to assume the infinitesimal $\frac{1}{p(n)}$ probability in these formulae pertain to agents naive on the qualities of the PRNG in their question. The complexity class, $BPTIME$, is a basic start to understanding how a virtually non-existent probability of discovery could, over a time period of acquiring gainful information, explode to a much higher probability of acquiring one or more hardcore predicates.

Definition 2.7 (BPTIME (Complexity Class))**.** [He25] A class of decision problems P solvable by a probabilistic Turing Machine M, such that for any polynomial function $t : \mathcal{N} \to \mathcal{N}$, there exists a constant $c > 0$ of time, satisfying the inequalities of probability,

$$Pr_{p \in P[\mathbb{1}]}(M(p, \texttt{accept}) = 1) > 2/3 \ \texttt{or}$$

$$Pr_{p \in P[\mathbb{0}]}(M(p, \texttt{reject}) = 1) > 2/3,$$

and the runtime constraints

$$M(*) < c \times t(n).$$

Accurate PRNG emulation from an adversarial party is a security risk. Goldreich discusses the concept of "derandomization" in one of his writings.[Gol10]

Definition 2.8 (Derandomization)**.** [Gol10, p. 36] Suppose there exists a pseudorandom number generator G and an emulating generator G'. If G' has less random elements to it, equivalently a deterministic finite automaton in implementation,

$$R_{deg}(G') < R_{deg}(G),$$

and satisfies

$$\sum_{i=1}^{n} \mathbb{1}(G(1) \neq G'(1)) < \frac{1}{p(n)}$$

for any polynomial p of strictly positive integer output, G' is effective in derandomization of G.

Any effective third-party spoofer to a pseudo-random number generator will attempt its derandomization by constructing an alternative generator. In later sections of this paper, further definition and elaboration are provided on challenges to third-party success. Objectives such as maintaining uniform probabilistic distribution, memory security, and PRNG auto-correction from third-party feedback data are critical security techniques for PRNG secrecy.

3 PRNG Classification and Computation Overview

At present, there are nine classes of PRNG structures in the *Seqbuild* system. The list goes as follows.

- Class 0: linear congruential generator, LCG variants, and LCG form-fitters.

  - **lcg**,**lcgv2**,**lcgv3**, Modular Decomposed Representative Generator (**mdrgen**).

- Class 1: linear combination and polynomial-based.

  - R-Chainhead Generator (**rch**), Integer Decision Forest Generator (**idforest**), Underdetermined Linear System Generator (**udls**).

- Class I: identity.

  - **echo**.

- Class 2: multi-factored form-fitters with noise adders.

  - Search Space Iterating Network Operator Generator (**ssino**), Gauge-Guided Generator (**gg**), Shadow Generator (**shadow**).

- Class T: time or singleton exploder.

  - PRNG Decimal Delta (**pdd**).

- Class D: differentials of form-fitters between two-dimensional points.

  - **fit22**, Lagrange Polynomial Solver (**lps**).

- Class 3: stacked vector-based form-fitters with noise adders.

  - n-to-m Vector Generator (**n2m**), Affine Fit Search Generator (**afs**).

- Class O: operation-based.

  - Operation Triangle Generator (**optri**), Congruence Shift Generator (**cshift**), Partitioned Multi-Generator (**pmulti**).

- Class V: quotient and polynomial differential form-fitters of vector inputs.

  - Quotient Value Generator (**qval**), Polynomial Interpolation Differential Generator (**pid**).

The source code for all of these generators, primitive structures that can be used as standalone generators or merged together, is made freely available to the public under the Massachusetts Institute of Technology (MIT) license. There are some noteworthy patterns in these generators, the foremost one being that all of them except for **lcg**,**lcgv2**,**echo**, and **pdd** use at least one parameter generator in their algorithmic processes. Abstract descriptions of each primitive structure's computation complexity are provided in Table 1.

Description of PRNGs That Use 1+ Parameter PRNGs		
PRNG Name	Primary Form	Parameter Generator/s Activity
lcgv3	A generator that outputs ternary vectors, used to adjust the pre-image values from a base LCG.	• updating to another ternary vector: $R[?(V[1], V[1], ?V[1], ?(V[2] \oplus V[3]), S[x])]$. • outputting one value: $?V[1], ?V[1]$.
mdrgen	Modular Decomposed Representative, a sequence of index ranges paired with (multiple,additives) and a corresponding sequence of modulo values	populate output queue: #1. $R[V[1]]$. #2. $V[q]$,$(R^t(R^{t-1}) \rightarrow \mathcal{R}^n)$ $V[q]$ times.
rch	A unidirectional path (chain) of nodes, each node a mutable linear combination or polynomial expression.	outputting one value • $F(V[1])$,$?R[V[q\&x]]$.

idforest	x single-rooted n-ary trees.	• to form a new tree: $R[V[q]]$. • populate output queue: S,S,$V[q]$.
udls	An underdetermined linear system of equations.	• declare underdetermined linear system of equations: $R[x]$. • generate output values: $F(V[q])$,$?(R[V[q]] \oplus R[x])$.
ssino	A sequence N of nodes, each an LCG, mutable modular decomposed representative, or mutable operation triangle operator. Every node is a root node to a tree, comprised of a subset of N. Every tree T_i is connected to j_i other trees.	• populate queue with output values: S, $V[q] * x$. • updating mutable nodes: $(R[S[q], P[x], ?V[q]] * q) * x$.
gg	Base number sequences from parameter PRNG.	populate output queue: • $R[V[1], V[q], V[1], ...$ $\#1.\ x\ *$ $(V[2], S[x], V[q])]$. $\#2.\ V[q], x]$.

shadow	Comma-separated values (csv) file of a number sequence.	- set new base vector by drawing from file and adding previous vectors (shadows) in output cache to it: • $R[V[1], V[1], V[q]]$. - deriving new vector from base vector: #1. $V[2], V[1] \oplus V[q]$; modular decomposed representative shadow. #2. $V[x] * 2, V[q] * x$; ternary vector shadow. #3. $V[x] * x$; factor vector shadow. #4. $V[2], P[x]$; operation triangle shadow.
fit22	Sequence of two-dimensional points. Every contiguous pair is fitted with a curve that is a scaled logarithmic or exponential function.	• process next value and possible changes to form: $V[1], ?R[V[4] * x], V[1]$.
lps	Sequence of two-dimensional points used in Lagrangian Polynomial Interpolation.	• process next value and possible changes to form: $V[1], ?R[V[q] * 2 * x], V[1]$.
n2m	A sequence of n-to-m vector functions.	• populating output queue from vectors: $V[q] * x, P[x], ?R[V[2], S[1]]$.

afs	A pair of matrices (X, Y), both matrices of equal shape to each other. Affine functions are used to map X to Y.	instantiate (X, Y) matrices: • $V[q] * x, V[3], ?V[1]$. update hypotheses (reform): #1. $R[V[q] * x, V[2] * x]$; stochastic. #2. $R[V[q] * x, S[x], V[2]]$; greedy.
optri	Upper-right triangle, associated with forward and backward pairwise operators.	• populate output queue: $?V[x], ?S[x]$; 45-90 split. $S[1], ?S[1], ?S[1]$; flip-derivation. • output one value $?V[1]$; add noise to value drawn from triangle before outputting it.
cshift	A congruence formula that intermittently adjusts contiguous elements from a parameter generator.	• outputting one value, possible adjustments to pre-image values: $V[1], V[1], ?R[V[4]]$.
pmulti	A sequence of PRNGs, each assigned a non-constant non-overlapping span in a contiguous range $r \in \mathcal{R}^2$), the spans comprising a partition of r.	• assigning another partition: $R[V[1], V[q] * x, P[x]]$.

qval	Sequence of real numbers used as operands for quotients of arbitrary lengths.	• process next value: $V[1], ?(S[2] * x)$.
pid	Sequence of two-dimensional points used in differential Lagrangian Polynomial Interpolation.	• process next value and possible changes to form: $V[1], ?R[(V[q] * 2, V[1], V[1]) * x]$.

Table 1: A general and partial description of *Seqbuild* PRNGs that use at least 1 parameter generator to generate numbers. **Key:** $V[i]$ uses the next i output values from parameter PRNG G_p, $S[i]$ selects i values from G_p, P uses G_p to permute a sequence, and R reforms the exemplification of primary form. Symbol q denotes a positive integer from or derived from G_p and x is a positive integer that is a structural variable or constant, related to the PRNG's primary form. Not all x's are equal, likewise for q's, and the category $x \subset q$. When $'?'$ is in front of an expression, the expression is one that operates by a counter that intermittently activates according to the counters set by structural and parameter PRNGs.

The term "form-fitters" was informally used in *Some Primitive Structures in Seqbuild* to describe the generation techniques of the PRNGs in *Seqbuild*'s initial release. Table 1 provides a general description of the usage of parameter generators.

Definition 3.1 (Form-Fitter (Characteristic of Pseudo-Random Number Generator))**.** A pseudo-random number generator G is a form-fitter if the algorithm A it uses to output values is fixated on an exemplification E_F of an underlying form F, such as sequence, matrix, tree, and graph. This exemplification can be mutable. Form F is also associated with an objective function O_F that A fits (solves) for, during the course of its generated output.

Form-fitters serve useful purposes in generating numbers. The form is a constant that ideally allows for exemplifications in an infinite space. Some forms of underdetermined design allow for exemplifications to, in turn, accept input values from a parameter generator to reach an output value. Qualities such as variance are altered from one contiguous subsequence of output values to another from a form-fitter that allows for mutable exemplification of form. In Table 1, there is some clarification needed for the "R" (reform) key in the parameter activity descriptions. When the generator algorithm reforms its primary form exemplification, it uses its parameter generator, for example, to repopulate the variables pertaining to that primary form. The reform, depending on the means

of that process, such as through its parameter generator, may result in a roughly equal exemplification to the prior exemplification.

Table 1 does not disclose the memory requirements of each PRNG's primary form. Generators such as **mdrgen** require a significantly greater amount of computing to arrive at the exemplification of their form. In the case of **mdrgen**, the form objective is to find the shortest sequence of (multiple,additive) pairs and the corresponding modulo sequence that maps to a replication of a reference sequence. The runtime of this search was provided in *Some Primitive Structures in Seqbuild*. In cases of reference sequences containing integers of high absolute value, such as greater than 10^6, and/or sequences with high absolute difference between contiguous elements, **mdrgen** slows significantly on the order of hundreds of times. The PRNG implementation places constraints on the search for (multiple,additive) pairs. Without constraints placed on the tree nodes that bear polynomial and linear combination I/O functions, the **idforest** generator would frequently crash from the *exploding value* problem. The **afs** generator is a *Seqbuild* PRNG with an exceptional trait, a high memory usage for its primary form, the exemplification a second-order seed. Its form is two matrices (X, Y), each of dimension $n \times m$, n the number of samples. The objective is to find the k highest scoring centroids, each an affine function mapping X to Y', in order to compare (calculate differences) with the actual Y.

Generators such as **pmulti** and **gg** have primary form objectives that prioritize uniform distribution. The **pmulti** generator is instantiated with a one-dimensional super-range R_{all}, and each of its k generators is assigned a unique sub-range in a partition $\mathcal{P}(R_{all})$. The partition changes according to a counter threshold set by its parameter generator, and the k generators are assigned sub-ranges for that new partition. The algorithm for this generator is not as computationally intensive as **gg**, relying solely on its parameter generator to assign values for the partitioning. Specifically, its k generators are cyclically used as the parameter generator for partitioning, in the event that the counter threshold is reached. The property of self-reflection is not programmed into **pmulti**. That is, this *Seqbuild* PRNG does not calculate the output properties of each of its k generators, instead changing the exemplification of each form upon reaching every counter threshold set by one of those k generators. The heavy implication of this efficiency is the pre-image values are merely adjusted according to the ranged modulii, with no additional calculation, before serving as output values. Generator **pmulti** thus lacks structural complexity in its primary form. The **gg** generator is equipped with two primary classes of derivation to act for the objective of uniform distribution. Although **gg**'s algorithm has greater structural complexity, indicated by the generator's much heavier computing for adjusting pre-image output values, it does not guarantee output values pass specified uniform distribution requirements. These derivation methods will be discussed in a section after this one, when the metrics that **gg** observes are explained. The structural complexity of **gg** is a result of its form's objective, uniform distribution, rather than in the case of the Class 3 generators, **n2m** and **afs**, and the Class 1 generators, **idforest** and **rch**. These four other generators base their output values on data structures that typically

consume memory on the scale of multiples of that of **gg**.

Every one of the PRNGs in Table 1 disproportionately depends on its parameter generator/s, and these parameter generator/s effectively act as the "kernel" to their host PRNGs. The output quality of the **optri** generator depends almost entirely on its first-order seed (instantiating parameters). One critical variable in these seeds is the base sequence, by which the primary form of upper-right triangle is calculated from, respectively. Note that in Table 1, the primary forms of these generators are not mutable.

4 A Briefing Over Application of Theoretical Mathematics

The *Seqbuild* system is deterministic. The variety of pseudo-random number generators in *Seqbuild*, along with the capability to merge PRNGs together by pairwise operators and into tree structures that output numbers from cycling through their PRNGs, enhances their underdetermination in the context of their observers' knowledge.

Definition 4.1 (Pseudo-Random Number Generator Composition)**.** A composition operation on a sequence of generators $\overrightarrow{S}_G = G_1, G_2, \ldots, G_k$ produces another generator G, and can occur through two primary ways: piecewise and merge. Piecewise composition preserves the value v_i from generator G_i, thereby outputting v_i. Common formats for piecewise composition include $(n_1, n_2, \ldots, n_k)$ frequency, n_i the number of contiguous values from generator G_i to output before moving on to the next generator $G_{((i+1) \mod k)}$, and cyclical composition, commonly implemented as a tree of nodes to cycle through and essentially the $\langle 1 \rangle^k$ frequency. A merge operation requires $(k-1)$ pairwise operators,$\langle r_1, r_2, \ldots, r_{(k-1)} \rangle$, such that

$$r : G \times G \longrightarrow G.$$

The resultant from a merge composition is

$$G := G_1 r_1 G_2 r_2 \ldots G_k.$$

The mathematical field of abstract algebra provides definition to studying the relations between numbers, sets, and other numerical structures.

Definition 4.2 (Group)**.** [Sha17, p. 38] Let G be a non-empty set. Let $*$ be a binary operation on G. Then G and the operation $*$ form a group if the following hold:

- $a, b \in G \implies a * b \in G$ (closure of the product),
- $a, b, c \in G \implies a * (b * c) = (a * b) * c$ (associativity of the product),
- $\exists e \in G$ such that $a * e = e * a = a \forall a \in G$ (existence of identity),

- $\forall a \in G$, $\exists a^{-1} \in G$ such that $a * a^{-1} = e = a^{-1} * a$ (existence of inverses).

Definition 4.3 (Homomorphism)**.** [Sha17, p. 55] Let $(G, *)$ and $(H, \circ)$ be groups. A map Φ, $\Phi : G \longrightarrow H$, is a homomorphism if

$$\Phi(a * b) = \Phi(a) \circ \Phi(b).$$

The algebraic descriptor for "structure-preserving" map is "homomorphism", although the definition provided in this algebraic formalism is not specifically catered for comprehending the intricate complexity of relation shared between two entities, such as PRNG relations. As already mentioned in this paper, most of the PRNGs in *Seqbuild* require at least one parameter generator, in order to be operational. There are four categories of parameter generator usage: pre-image values ("V"), index selection ("S"), sequence permutation ("P"), and form reformation ("R"), a permutative usage of the first three categories. If the parameter generator is deterministic, which it most likely would be in the standalone context of the *Seqbuild* system, then its usage by its host generator would necessarily tend towards being a structure-preserving map unless there are irregular operations that deform the parameter generator's output values. Operations that build on permutations and towards objectives, coupled with the second-order seeds of generator primary forms, are extensively used in *Seqbuild*. A relevant concept that accomodates for the lack of bijectivity, stemming from this convolution, is"embedding".

Definition 4.4 (Embedding (Abstract Algebra))**.** [Sul23, p. 31] Suppose there is a map Φ from one group G to another group H, such that

$$\Phi : G \longrightarrow H$$

is an injective homomorphism. Then Φ is an embedding of G inside of H, since $\Phi(G)$ effectively serves as a "copy" of $G \in H$.

A foundational subfield in abstract algebra is in the study of ring structures.

Definition 4.5 (Ring (Abstract Algebra))**.** [Sha17, p. 316] A ring $(R, +, \circ)$ is a non-empty set R with two binary operations $+$ and $\circ$, such that:

- the set R forms an abelian group with $+$,
- the operation $\circ$ is associative on R,

$$a \circ (b \circ c) = (a \circ b) \circ c,$$

- R is distributive with respect to $(+, \circ)$,

$$a \circ (b + c) = (a \circ b) + (a \circ c),$$

$$(b + c) \circ a = (b \circ a) + (c \circ a).$$

Some rings are rather confined in value and convenience predictability. This area is where *Seqbuild*, a PRNG system that uses a significant amount of algebra in its operations across its primitive structures, shows weaknesses in some cases of second-order seeds given to its PRNGs.

The general description of PRNG algorithms in Table 1 indicate input-output functions that are predominantly comprised of more than one operation. To clarify on the operational use of a *Seqbuild* PRNG, when the PRNG G is to output a value, it is technically, or at least on the outside, an inputless function, as such:

$$G().$$

The abstraction provided by conventional algebraic definition lacks the definitional scaffolding to specify how in the *Seqbuild* system, parameter PRNGs operating for their host PRNG are essentially embedded objects. The term "object" is a typical noun used in computer programming lexicon to denote a program or subprogram or data structure (a program, all the same).

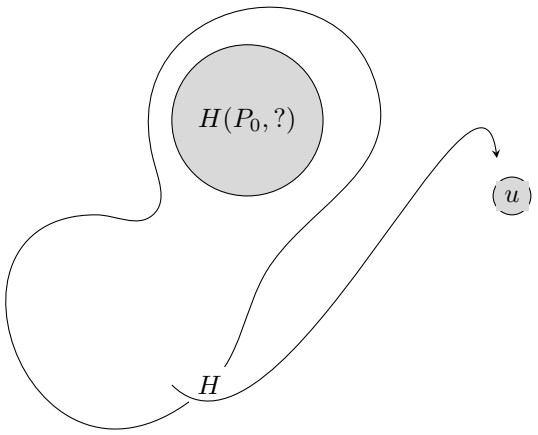


Figure 2: Parameter generator P_0 plays a critical role in the output u of host generator H, as H operates P_0 alongside its other values "?" of its second-order seed, denoted by $H(P_0, ?)$.

There are a few key challenges for an observer to prove that a generator P_0 is a parameter generator of another, G. In a predominant number of generator cases of output, except for cases of identities and small cycles (described in Section 7), along with the **echo** generator applied to a number sequence output by another generator,

$$G(n) \neq P_0(n);$$

`n a positive number specifying the number of iterations from the generator.`

Furthermore, a host generator's design frequently, but not always, results in

$$\big|im(G)\big| > c\big|im(P_0)\big|,$$

`c a real number at least 1.0.`

The term "phenotypic security" has already been used in this paper. In this section's discussion, it is appropriate to explain the dichotomous concepts of "genotypic" and "phenotypic", in the context of pseudo-random number generator computer programs, specifically the issues that a naive and/or adversarial observer to the PRNG would face.

- Genotypic Secrecy (Unknown of the State Kept Hidden):
  - Partial knowledge of the 2nd-order seed belonging to parameter generator P_0. A combination of naivety from the observer, with respect to the algorithms of P_0 and host G, and the complexity of computation from P_0 could negate the practical effectiveness of that partial knowledge.
- Phenotypic Secrecy (Unknown of the Before and After of The Observations):
  - The Many Generators to One Sequence Problem. Adversary finds a generator G' that outputs the same sequence $\overrightarrow{S}$ it observed from host generator G. It still cannot guarantee that it can reproduce the next k outputs from G. This is due to the many-to-one mapping problem. That is, there can be more than one PRNG that can produce the sequence $\overrightarrow{S}$.
  - The Many Seeds to One Sequence Problem. Algorithm of host generator G could produce the same sequence $\overrightarrow{S}$ via more than one parameter generator.

The two primary problems posed by phenotypic secrecy, in turn, fall under the observation problem described as temporal equality in *Some Primitive Structures in Seqbuild*. In other words, temporal equality is the concept where any two objects, number generators in this discussion, exhibit equal values in variable of expression only during certain spans of their activity, thus implying there is non-null probability the two objects differ in expression during other spans. Biology prefixes (geno,pheno) are fitting for describing the implementation of pseudo-random number generators, especially since the effect of a parameter PRNG on its host PRNG is the analogical equivalent of a DNA string on the physical manifestation of its central organism. Furthermore, the complex of systems design in modern software architecture is what enables digital electronic bodies of constants and motion.

Algebraic definitions have evolved significantly past the general homomorphism to accomodate for computation complexity.

Definition 4.6 (Weak Homomorphism (Algebra Relation))**.** [Sch11] Let there be two non-index universal algebras $\mathcal{A} = (A, F)$ and $\mathcal{B} = (B, G)$. Then a mapping $\Phi : A \to B$ is a weak homomorphism from $\mathcal{A}$ to $\mathcal{B}$ if for any n-ary fundamental operation $f \in F$, $n \in \mathcal{N}$, there exists an n-ary term operation $g \in B$ such that

$$\Phi(f(a_1, \ldots, a_n)) = g(\Phi(a_1), \ldots, \Phi(a_n)).$$

Definition 4.4 was developed to accomodate the demands of fuzzy mathematics, the branch of mathematics dealing with uncertainty in the form of infinitesimal non-zeros and their effects divergent from the cleanness of discrete mathematics.

Definition 4.7 (Functor (Category Theory))**.** [Cyp] A mapping F pertaining to categories C and D,

$$F : C \longrightarrow D,$$

that maps elements of C to those of D, and morphisms

$$f : a \to b$$

of C to morphisms of D,

$$F(f) : F(a) \to F(b),$$

such that

- $F(id_x) = id_{F(x)}$ [unit] and
- $f : a \to b, g : b \to c$ are morphisms of $C \implies F(g \circ C(f)) = (Fg) \circ D(F(f))$ [composition],

then F is a functor.

Definition 4.5 is an abstraction of the relations shared between PRNG computer programs when one acts as a parameter generator (embedded functional object) for a host generator. The class of PRNG is synonymously a category by Definition 4.5. In cases where the embedded generator depth (or degree) of a host generator does not exceed in statistical differentiability, effective machine-learning should be able to produce hash tables of these parameter-to-host generator relations with a relatively high-level of accuracy.

Host PRNG algorithms frequently "deform" their parameter PRNG output values. For example, at the i'th output index of the parameter generator, selecting one or more elements from a sequence $\overrightarrow{S}$ requires operation $x \mod |\overrightarrow{S}|$. This selection is equal from all PRNGs that output the value $x +/- |\overrightarrow{S}|$ at their i'th output index. Modulo operations are a common source for the many-to-one problems mentioned, and so, an algebraic definition for computer science would be useful.

Definition 4.8 (Modular Equivalent (Pseudo-Random Number Generator))**.** A pseudo-random number generator P_1 is a modular equivalent of another, P_0, if

- $P_1(n) \neq P_2(n); n \in \mathcal{N}$.
- A host generator G that uses either P_1 or P_0, as part of its second-order seed, outputs the exact same number sequence from index 0 towards infinity.

A weak modular equivalent of a PRNG, then, would satisfy temporal equality, but cannot be used by the host generator to produce the same numbers throughout any duration of output.

Seqbuild can merge any two PRNGs G_0 and G_1, the pair could be the same PRNG at an odd and an even instance, with a pairwise operator. Although the pairwise operator itself is homomorphic, the conventional definition as provided in Definition 4.2 does not account for merging their parameters and algorithms,

$$(G_0 + G_1)() \neq G_0() + G_1(),$$

due to every *Seqbuild* primitive structure having its own unique parametric set. The result of merging two PRNGs that appear random is one PRNG that the observer would have a negligible chance of decoupling one of the generators from the other.

To compensate for the PRNG predictability problem posed by homomorphisms, mappings that would lead to uncovering the determinism behind the PRNG, such as linear combinations and polynomials, some of *Seqbuild*'s primitive structures have significant degrees of structural complexity. This structural complexity shows itself in the memory required for **idforest** to store its trees of nodes associated with functions of linear combinations and polynomials that are mutable by parameter PRNGs, the number of vector pairs in **afs**, amongst other structures. Thus, if such a system as *Seqbuild* were to be operated from a secured device, observers not of the serving party to the program have to resort to hypotheses on a system that is under-determined, from their perspective. These problems of finding valid hypotheses from what is known or believed, one-to-many, were abstractly illustrated in Figure 1.

There are no limits, besides from those posed by device memory and computing constraints, to the number of degrees of parameter generators. In other words, the embedded generator depth for an arbitrary PRNG that uses one or more parameter generators is variable. This can be appropriately expressed in mathematical notation as

$$G(1) = A(G, G_1(n_1), E_F),$$

$$G_1(n_1) = A_1(G_1, G_2(n_2), E_F^{(1)}), \ldots,$$

$$G_{k-1}(n_{k-1}) = A_{k-1}(G_{k-1}, G_k(n_k), E_F^{(k-1)}),$$

$$G_k(n_k) = A_k(G_k, *, E_F^{(k)});$$

`G outputs 1 value, calling its parameter generator` G_1 `for calculations with its exemplified primary form` E_F`, thereby starting a chain process`

demanding an arbitrary number n_i of output values from every embedded generator G_i.

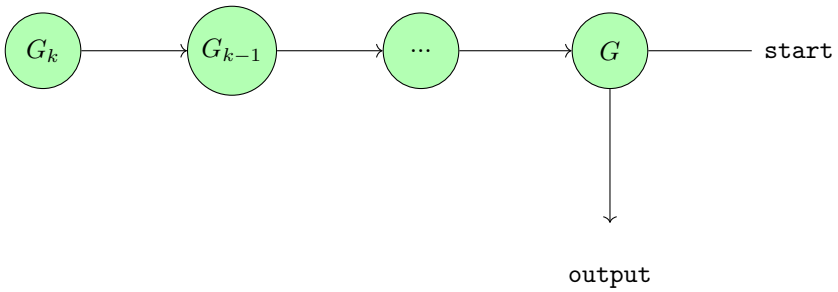


Figure 3: Generator G is to output a value. Its algorithm draws output values from parameter G_1 that then calls G_2 and so on, up through G_k. The embedded generator depth of G is therefore k.

5 Four Randomness Tests

The **Qual Test**,**Chain Test**, **PRNG I/O Test**, and **Cycle Test** were each designed to uniquely focus on specific aspects of number sequences from pseudo-random number generators. The first one is the most attentive to the minutiae in qualitative differences of the real number sequence in question. Chain Test focuses on the continuity of difference, along four metrics, between contiguous parts of a partitioned real number sequence. The third test is a comparative synopsis in output values between a parameter generator (input) and its host generator (output). It also uses a hardcoded criteria for determining which generator, out of two, is more "random". And the Cycle Test is a discretized alternative to the Chain Test. It is more of a pattern recognizer than a distinguisher through continuity. These tests use the technique of n-grams, common in natural language processing, to appropriately divide up sequences for piecewise measurement during processing.[JM]

5.1 The Qual Test

Test input is number of iterations (contiguous real numbers) from a PRNG. Output calculation consists of eight values.

Suppose the output sequence is S, $S \in \mathcal{R}^*$. Then Qual Test conducts these calculations on each n-gram S' of S. The default n is set to 10. The first seven values are averages over these n-grams.

- coverage (spatial),$\mathbb{C}$:

The minimal one-dimensional range that contains all elements of S' is $R_{S'}$. Given a parameter w, a positive real number acting as the radius, each element $s_i \in S$ is mapped to its respective span, r_i (an ordered one-dimensional range), via function D.

$$C_1(S') = \bigcup_{r_i \in D(S', w)} (r_i \bigcap R_S),$$

$$C(S') = \frac{\sum_{p_i \in C_1(S')} (p_i[1] - p_i[0])}{R_S[1] - R_{S'}[0]}.$$

Note: By default, radius w is set to 0.5. No duplicate counting of intersectional space between ranges of $D(S', w)$.

- unidirectional weighted point distance (uwpd), $\mathbb{U}_{pd}$:

  A forward summation of absolute differences between the i'th value in S' and the remaining elements, $S'[i+1:]$. Runtime is $\mathcal{O}(|S|^2)$.

$$\mathbb{U}_{pd}(S') = \frac{\sum_{s_i \in S'} (\sum_{s_j \in S'[i+1:]} (|s_j - s_i|))}{(\max(S') - \min(S')) \times (|S'| - 1)!}.$$

- categorical entropy, $\mathbb{E}$:

  Given an n-gram $S' \in S$, categorize each $s \in S'$ according to n labels, the range $[0, n-1]$.

$$L_{S'}(s) = \lfloor \frac{s - \min(S')}{(\max S' - \min S')/n} \rfloor,$$

$$S'' = \left\langle L_{S'}(s_i) \forall s_i \in S' \right\rangle,$$

$$\mathbb{E}(S') = \frac{\sum_{i=0}^{n-2} (|S''[i+1] - S''[i]|)}{(n-1) \times (|S''| - 1)}.$$

The next four values use an implementation of Kolmogorov complexity, $\mathcal{K}$.

```
function K(S)
    i := 1
    S := sorted(S)
    S_k := ⟨S[0]⟩
    while i < |S| do
        if S[i] ≠ S_k[−1] then
            S_k := S_k ⊕ (S[i])
        end if
        i+ = 1
    end while
    return |S_k|
end function
```

- $\mathcal{K}$-complexity of first-order ternary vector,k_1:

$$T(s_1, s_0) = \begin{cases} 1 \texttt{ if } s_1 > s_0\texttt{,} \\ 0 \texttt{ if } s_1 \equiv s_0\texttt{,} \\ -1 \texttt{ otherwise.} \end{cases}$$

$$T_{\overrightarrow{V}}(S_?) = \Big\langle T(S_?[i+1], S_?[i]) \forall i \in [0, |S_?| - 2] \Big\rangle,$$

$$S_1 = T_{\overrightarrow{V}}(S_1),$$

$$k_1 = \mathcal{K}(S_1).$$

- $\mathcal{K}$-complexity of second-order ternary vector,k_2:

$$k_2 = \mathcal{K}(S_1).$$

- most common contiguous subsequence, mcs-complexity, mcs: [1]

  Calculate map of frequencies for all contiguous subsequences $v \in S'$:

$$\mathcal{M} : v \to f_v; f_v \in \mathcal{N}_{\geqslant 0}.$$

  The MCS difference d_{mcs} for a subsequence v is

$$d_{mcs}(v) = |S'| - (\mathcal{M}[v] * |v|).$$

[1] *Seqbuild* is equipped with multiple variants of the mcs-complexity measure. The one described in the paper is the current default one used for Qual Test.

Sort all contiguous subsequences by alphabetical order into S'_v. If $|S'_v|$ is odd, the median is v_m, for a score of

$$mcs(S') = \frac{d_{mcs}(v_m)}{|S'|}.$$

Otherwise, the median $\{v_l, v_r\}$ results in

$$mcs(S') = \frac{d_{mcs}(v_l) + d_{mcs}(v_r)}{2|S'|}.$$

- modular characteristic ($\mathcal{K}$-complexity of S' modulated by an integer i), $k^{(i)}_{mod}$:

  Gather the union of all factors pertinent to $S'' = \Big\langle \lfloor s \rfloor \forall s \in S' \Big\rangle$:.

  $$F_{S''} = \bigcup_{s \in S''} (factors(s)) - \{1, 2\}.$$

  Calculate every modulated $\mathcal{K}$-complexity, for an aggregated map of

  $$\mathcal{M}^{(S'')}_{\mathcal{K}} : F_{S''} \longrightarrow \mathcal{N},$$
  $$f \longrightarrow \mathcal{K}(T^{\rightarrow}_{V}(S'') \mod f).$$

- number of unique elements in S.

5.2 The Chain Test

The Chain Test takes two positive integers n and p as input, n the number of iterations from the PRNG in question, p the partition size, $n \geqslant p$. The partitioning is uniform, with the exception of the last part having more elements for remainder from division. It is, in many ways, an extension of the classical standard deviation measure, measuring four attributes, each at dual values.

The variance and mean of every i'th part $P^{(i)}_S$ are $V[P^{(i)}_S]$ and $A[P^{(i)}_S]$,respectively. The **accumulated modular characteristic** is an injective mapping from the factors of $\mathcal{M}^{(S)}_{\mathcal{K}}$.

$$C_{\texttt{amod}}(P^{(i)}_S) = f \longrightarrow f \times \mathcal{M}^{P^{(i)}_S}_{\mathcal{K}}[f]; f \in \mathcal{M}^{P^{(i)}_S}_{\mathcal{K}}.$$

Summation of output values from $C_{\texttt{amod}}(P^{(i)}_S)$ is

$$\mathbb{W}(C_{\texttt{amod}}(P^{(i)}_S)) = \sum_{f \in C_{amod}(P^{(i)}_S)} \Big(C_{\texttt{amod}}(P^{(i)}_S)[f]\Big).$$

Given any two parts, $P^{(i)}_S$ and $P^{(j)}_S$, the difference

$$\mathbb{W}(C_{\texttt{amod}}(P^{(i)}_S)) - \mathbb{W}(C_{\texttt{amod}}(P^{(j)}_S))$$

is a map with key space exactly that of $C_{amod}(P_S^{(i)})$ (non-commutative). In cases where

$$\Big(f \in \mathbb{W}(C_{\texttt{amod}}(P_S^{(i)}))\Big) \& \Big(f \notin \mathbb{W}(C_{\texttt{amod}}(P_S^{(j)}))\Big),$$

the value $C_{\texttt{amod}}(P_S^{(j)})[f]$ is 0.

The eight output values are listed below.

- $$\min(\bigcup_{i \in [0, \lfloor n/p \rfloor - 1]} V[P_S^{(i)}]),$$
  $$\max(\bigcup_{i \in [0, \lfloor n/p \rfloor - 1]} V[P_S^{(i)}]).$$
- $$\min(\bigcup_{i \in [0, \lfloor n/p \rfloor - 1]} A[P_S^{(i)}]),$$
  $$\max(\bigcup_{i \in [0, \lfloor n/p \rfloor - 1]} A[P_S^{(i)}]).$$
- $$\texttt{var}\Big(\bigcup_{i \in [0, \lfloor n/p \rfloor - 1]} \big(\mathbb{W}(C_{\texttt{amod}}(P_S^{(i)})) - \mathbb{W}(C_{\texttt{amod}}(P_S^{(i+1)}))\big)\Big),$$
  $$\texttt{var}\Big(\bigcup_{i \in [0, \lfloor n/p \rfloor - 1]} \big(\mathbb{W}(C_{\texttt{amod}}(P_S^{(i+1)})) - \mathbb{W}(C_{\texttt{amod}}(P_S^{(i)}))\big)\Big).$$
- $$\texttt{mean}\Big(\bigcup_{i \in [0, \lfloor n/p \rfloor - 1]} \big(\mathbb{W}(C_{\texttt{amod}}(P_S^{(i)})) - \mathbb{W}(C_{\texttt{amod}}(P_S^{(i+1)}))\big)\Big),$$
  $$\texttt{mean}\Big(\bigcup_{i \in [0, \lfloor n/p \rfloor - 1]} \big(\mathbb{W}(C_{\texttt{amod}}(P_S^{(i+1)})) - \mathbb{W}(C_{\texttt{amod}}(P_S^{(i)}))\big)\Big).$$

5.3 The PRNG I/O Test

This test provides comparative operations that are extensions of basic pairwise PRNG score differentials for the Qual and Chain Tests. At present, there are seven comparators available for this test. Each comparator observes a subset of the Qual and Chain Test output variables. The comparators abide by hardcoded conditional statements, such that for any observed variable from the Qual and Chain Tests, greater values correspond to greater "randomness". This basis is not without flaw, since it ignores relativity from the differences between PRNG output spans. In cases of a PRNG having a relatively large output span, but with more predictable output values than another PRNG, the PRNG may be categorized as more "random" than the other PRNG, by the conditionals provided in the *Seqbuild* system. The algorithm uses a standard ternary output

function for ascribing comparative randomness: 1 for greater than, 0 for equal, and -1 for less than.

The test observes n input PRNGs, $n \geqslant 1$, each used as a parameter generator for one output PRNG. A copy of the output PRNG used for each of these n parameter generators, so that the measurements are on the same output value indices. There are four categories of comparison:

- pairwise comparison between input PRNGs,
- comparison between input PRNG and its corresponding output PRNG,
- pairwise comparison between output PRNGs,
- pairwise comparison between two (input,output) PRNG differences.

Information on PRNG I/O Test Comparators	
Comparator Number	Variables Observed
#1	All measures from both Qual Test and Chain Test.
#2	All measures from Chain Test.
#3	1st order $\mathcal{K}$-complexity, 2nd order $\mathcal{K}$-complexity, *mcs*-complexity, and all measures from Chain Test.
#4	1st order $\mathcal{K}$-complexity, 2nd order $\mathcal{K}$-complexity, *mcs*-complexity, number of unique elements, and all measures from Chain Test.
#5	All measures except for modular characteristic from the Qual Test and accumulated modular characteristics (variance and mean) from the Chain Test.
#6	All measures in Qual Test, except for modular characteristic and number of unique elements.
#7	All measures from Qual Test.

Table 2: The variables that each available comparator of PRNG I/O observes.

Let comparison operator function be C_{rand}. For two PRNGs, G_1 and G_2, and a comparator H that observes variable set Y_{H},

$$C_{rand}(G_1, G_2, \mathrm{H}) = \begin{cases} 1 \texttt{ if } (\sum_{y \in Y_\mathrm{E}}[\mathrm{H}(G_1 > G_2 \texttt{ on } y)]) > \frac{|Y_\mathrm{H}|}{2}, \\ 0 \texttt{ if } (\sum_{y \in Y_\mathrm{E}}[\mathrm{H}(G_1 > G_2 \texttt{ on } y)]) \equiv \frac{|Y_\mathrm{H}|}{2}, \\ -1 \texttt{ otherwise.} \end{cases}$$

For a PRNG I/O Test that uses a set of comparators $\{H^{(i_1)}, \ldots, H^{(i_k)}\}$, $k \leqslant 7$, for any one of the four categories of comparison, the test simply uses the same ratio function as above, on the comparators' resultant ternary vector to output a single ternary.

5.3.1 Variants on the PRNG I/O Test

The randomness comparators of the PRNG I/O Test are not confined in application solely to measuring the effects of parameter generators on their host generators. The category of comparison between input PRNGs could actually be applied to any sequence of PRNGs, without considering that PRNG sequence's comparative differences as parameter generator to an arbitrary host. There are some test results in this paper that strictly focus on comparisons between generators of a sequence, without reference to any sole host generator.

5.4 The Cycle Test

The Cycle Test is essentially a Markov Model, with some additional frequency-based measures. It requires three parameters to run: positive integers n (length of prefix), m (length of suffix), and d (number of generator output values). The test conducts calculations on n-gram prefixes from a map, $\mathbb{M}$:

$$\mathbb{M} : \mathcal{R}^n \to \mathcal{N}_{\geqslant 0} \to \mathcal{R}^m,$$

$$\texttt{n-gram} \to \texttt{ start index} \to \texttt{ m-gram.}$$

The m-gram is the disjoint subsequence immediately after the n-gram.

The usage of *Seqbuild*'s *mcs*-complexity measure differs from that of Qual Test. In the Cycle Test, it is an average of four variants, set by different modes for the operating function of *mcs*.

Given a prefix $s_n^{(i)}$, there are two sequences that *mcs* is applied on. Let $T_{s_n}^{(i)}$ be the ordered sequence of indices for $\mathbb{M}[s_n^{(i)}]$.

- The concatenation of ordered suffixes into a sequence,

$$\overrightarrow{C}_{s_n^{(i)}} = \Big\Vert_{t \in T_{s_n}^{(i)}} \mathbb{M}[s_n^{(i)}][t].$$

- The concatenation of ordered and labelled suffixes into a sequence. Let there be a labelling function $\mathcal{L}$ and have it map each element in the sorted set of suffixes $\overrightarrow{\mathcal{S}}_{\texttt{suffix}}$ to an integer, starting from 0,

$$\mathcal{L} : s_j \longrightarrow \texttt{index}(\overrightarrow{\mathcal{S}}_{\texttt{suffix}}, s_j); s_j \in \overrightarrow{\mathcal{S}}_{\texttt{suffix}}.$$

$$\overrightarrow{C}^{(\mathcal{L})}_{s_n^{(i)}} = \Big\Vert_{t \in T^{(i)}_{s_n}} \mathcal{L}(\mathbb{M}[s_n^{(i)}][t]).$$

The first two measures from Cycle Test are the *mcs* function, already defined, applied on $\overrightarrow{C}_{s_n^{(i)}}$ and $\overrightarrow{C}^{(\mathcal{L})}_{s_n^{(i)}}$. The next three measures are the following.

- Normalized variance of suffix frequency, $\texttt{var}_{\texttt{norm}}$. Let there be a map f,

$$f : \mathcal{R}^m \to \mathcal{N}_{\geqslant 0},$$

$$\texttt{suffix} \ \rightarrow \ \texttt{frequency of suffix}.$$

  The minumum,maximum, and mean frequencies are $\min(f_{out})$,$\max(f_{out})$, and $\texttt{mean}(f_{out})$,respectively. Then prefix $s_n^{(i)}$ yields

$$\texttt{var}_{\texttt{norm}}(s_n^{(i)}) = \sum_{t \in T^{(i)}_{s_n}} \frac{\big|\mathbb{M}[s_n^{(i)}][t]\big| - \texttt{mean}(f_{out})}{\max(f_{out}) - \min(f_{out})}.$$

- Set size for difference vector of prefix indices of occurrence, $\triangledown_{index}$,

$$\triangledown_{index}(s_n{}^{(i)}) = \Big|\big(T^{(i)}_{s_n}[k+1] - T^{(i)}_{s_n}[k]\big) \forall k \in [0, \big|T^{(i)}_{s_n}\big| - 2]\Big|.$$

- Mean cyclical kernel difference, $\mathcal{D}_{\texttt{cycle}}$, for $\overrightarrow{C}^{(\mathcal{L})}_{s_n^{(i)}}$ and $\overrightarrow{C}_{s_n^{(i)}}$. Iterate through the subsequences $\overrightarrow{C}_{\texttt{sub}}$ of $\overrightarrow{C}^{(?)}_{s_n^{(i)}}$ that are in the top $1/q$ most frequent. Each subsequence c_i,$c_i \in \overrightarrow{C}_{\texttt{sub}}$, is to serve as a cyclical kernel, $\mathbb{B}^{(c_i)}_{\texttt{cycle}}$.

$$R_{\texttt{cycle}} := \Big\Vert_{i=1}^{\lceil \overrightarrow{C}^{(?)}_{s_n^{(i)}} \rceil / |c_i|} \big(c_i\big),$$

$$R_{\texttt{cycle}} := R_{\texttt{cycle}}[: \big|\overrightarrow{C}^{(?)}_{s_n^{(i)}}\big|],$$

$$j := \min(T_{s_n^{(i)}}),$$

$$L_{\texttt{cycle}} := \Big\Vert_{i=1}^{\lceil j \rceil / |c_i|} (c_i),$$

$$\mathbb{B}^{(0)}_{\texttt{cycle}} := L_{\texttt{cycle}} \oplus R_{\texttt{cycle}},$$

$$\mathbb{B}^{(c_i)}_{\texttt{cycle}} := \mathbb{B}^{(0)}_{\texttt{cycle}}[\big|\mathbb{B}^{(0)}_{\texttt{cycle}}\big| - \big|\overrightarrow{C}^{(?)}_{s_n^{(i)}}\big| :].$$

  Then the cyclical kernel difference is the element size of difference between $\mathbb{B}^{(c_i)}_{\texttt{cycle}}$ and $\overrightarrow{C}^{(?)}_{s_n^{(i)}}$,

$$\mathcal{D}^{(0)}_{\texttt{cycle}}(\overrightarrow{C}^{(?)}_{s_n^{(i)}}, c_i) = \sum_{j=0}^{|\mathbb{B}^{(c_i)}_{\texttt{cycle}}|-1} \big(\mathbb{1}[\mathbb{B}^{(c_i)}_{\texttt{cycle}}[j] \equiv \overrightarrow{C}^{(?)}_{s_n^{(i)}}[j]]\big),$$

$$\mathcal{D}_{\texttt{cycle}}(\overrightarrow{C}^{(?)}_{s_n^{(i)}}) = \sum_{c_i \in \overrightarrow{C}_{\texttt{sub}}} \frac{\mathcal{D}^{(0)}_{\texttt{cycle}}(\overrightarrow{C}^{(?)}_{s_n^{(i)}}, c_i))}{\left|\overrightarrow{C}_{\texttt{sub}}\right|}.$$

Lemma 5.1. *The Cycle Test can detect if the PRNG, in question, is a linear congruential generator if a real number r, $r \in \mathcal{R}$, has frequency at least two in the n output values of observation.*

Note the word "can" in Lemma 5.1. A number sequence with two contiguous real numbers $(r0, r1)$ appearing more than once is not necessarily from a linear congruential generator. But an algorithm can use that existence to start up a log to record the frequencies of other pairs from the sequence, and then confirm or reject that hypothesis. Summarizations of Cycle Test scores consist of means of the aforementioned measures over all prefixes.

5.5 A Characteristic Trait of Pseudo-Random Number Generators

Measure of the *periodicity* of a PRNG was not included in these four randomness tests. Periodicity, with respect to these kinds of generators, is not the same as in elementary cases, such as travelling around a circle or a one-dimensional loop lacking states beneath what can be observed.

L'Ecuyer relies on automata theory to define PRNG periodicity as a trait distinct from generator output values.

Definition 5.1 (Random Number Generator (L'Ecuyer Definition))**.** [LEc10, p. 2] A random number generator is a structure $(S, \sqcap, f, U, g)$ such that

- S is a finite set of states (the state space),
- $\sqcap$ is the probability distribution on S of selecting the initial state (seed),
- $f : S \longrightarrow S$ is the transition function,
- U is the output space,
- $g : S \longrightarrow S$ is the output function.

Definition 5.2 (Pseudo-Random Number Generator Period (L'Ecuyer Definition))**.** [LEc10, p. 2] Let there be a pseudo-random number generator of the form $(S, \sqcap, f, U, g)$. Since S is finite, the implication is there must exist two integers $l, j \geqslant 0$ such that for any state s_i, $i \geqslant l$,

$$s_i = s_{i+j}.$$

The smallest integer for j is the generator's period. Additionally, since functions f and g are deterministic, then for the output u_i, $u_i = u_{i+j}$.

Definition 7.3 holds especially true for pure pseudo-random number generators. But there is an issue with the notion that an arbitrary state s_i an automaton transitions to will consistently output u_i. That is, there is correlation but not causation of an output from s_i. Deterministic algorithms make Definition 7.3 convincing enough. But in practical and knowledgeable uses of PRNGs, however, determining the period of a PRNG cannot be accomplished as easily as simply observing repeated output values. The elementary linear congruential generators comprise one PRNG type with the state and the output sets sharing a clear bijective relation.

There are some cases of *Seqbuild* PRNGs and general PRNG use that make for good illustration. The **idforest** primitive structure was described in *Some Primitive Structures in Seqbuild*. To summarize, **idforest** is comprised of some k mutable trees, single-rooted directed simple graphs such that every child-set of a parent node can be greater than size two. These trees comprise the forest from where output values are calculated, and the forest is mutable. A faulty approach to predicting output values from an **idforest** is to consider the k trees as the singular set of states. When those trees are modified, the linear combination or polynomial expression functions associated with their nodes undergo change. Specifically, if a node n is supposed to be a state or part of a state, and n_t (at timestamp t) differs from the node n_{t+j} (at timestamp $(t + j)$), then node n does not remain an identity to itself and thus cannot be considered a consistent part of the algorithm. In the primary form of **rch**, the functions associated with the nodes in the unidirectional path do not remain constant. They can change from linear expression to polynomial and vice-versa, and the weights are non-static. In **ssino**, the nodes associated with modular decomposed representative and operation triangle functions are intermittently updated by the algorithm through its parameter generator.

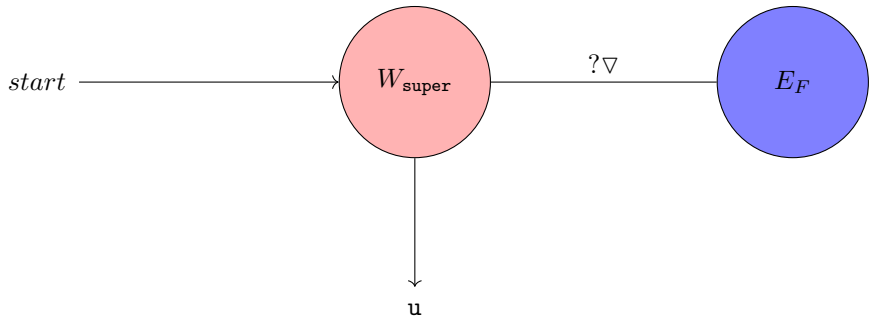


Figure 4: Pseudo-random number generator G, shown as the purple and red nodes, is prompted to output one value. The superset of states $W_{\texttt{super}}$, responsible for conducting the calculations, refers to the exemplification E_F of G's primary form. The values of E_F constitute a subset of values for the output calculation. Changes to E_F, entire or partial replacement of values, by parameter generator P may also occur. Real number u is outputted afterwards.

6 Weak Cases of Phenotypic Security

Before moving on to demonstrating the four described randomness tests on selected PRNGs, there are some fundamental characteristics of PRNG output that allow the adversarial party a significantly greater degree of ease in determining pertinent PRNG parameters.

Characteristics Positively Correlated withGreater PRNG Vulnerability	
Characteristic	Examples
Known pattern	•Pattern in modulated sequence. Example: $\overrightarrow{S} \mod 3 \rightarrow \langle 0, 2, 5 \rangle^*$. •One or more generators in G, comprised of a tree of generators $\langle g_1, g_2, \ldots, g_k \rangle$ outputting values from them in cyclical iteration. Example: Sub-generators g_1 and g_3 are known. For any index i from G's output, if $i \mod (k+1) \in \{0, 2\}$, then corresponding real number s_i is known.
Out of range of uniform distribution for computational indistinguishability	•Example: Cryptographic program maps letter to integer. Map is public knowledge. PRNG outputs values only in $[0., 1.]$.
Permutation of known values	Example: PRNG G only outputs values from relatively small set of real numbers, X. Then for any output index range $[i, i+j]$ of G, probable search space has been reduced to permutation space over X, instead of over continuous distribution $\mathcal{D}$, such that $\lvert\mathcal{D}\rvert > c * \lvert X\rvert$; `c a large value in` $\mathcal{R}_{>0}$.

Table 3: Some general characteristics of outputs from a pseudo-random number generator G that make G more predictable and thus, less suitable for cryptographic operations.

The *Seqbuild* pseudo-random number generating system, with a relatively diverse assortment of PRNG types, still does not depart from mathematical principles on number spaces, principally of homomorphisms.

7 Demonstrations with Randomness Tests

In this section, results are organized into tables from the four randomness tests described in Section 4. The focus on generator attributes ranges from the Class 0 generators, LCG variants and Modular Decomposed Representative, a form that uses multiples, additives, and modulii in the same formula as LCGs, to statistical synopses over distributions of (parameter,host) generators. The results are accomodated with interpretation of the subject generators' output sequences in the context of patterns. Mathematical details of PRNG primary forms and objectives are also provided. There is an important principle at the end of this section. A mathematical proof is attempted to explain the validity of the principle, on the matter of computation complexity and PRNG randomness. *Some Primitive Structures in Seqbuild* provided brief descriptions of the three main theories on randomness. The emphasis in this paper focuses more on the empirical findings from randomness testing and on the practical problems that concern PRNGs. As such, the third theory on randomness, where phenomena to an observer is categorically random or not, only because of observer's perspective that includes the degrees of their background knowledge on the phenomena and their real-time observation of it.

7.1 Measuring Randomness on Classical Linear Congruential Generators

Seqbuild Randomness Scores on Select LCGs		
Parameters	Qual	Chain
$3, 2, 0, 224$	•$\mathbb{C} = 0.33992, \mathbb{U}_{pd} = 0.26303, \mathbb{E} = 0.32159, k_1 = 163, k_2 = 163, mcs = 0.02$. •Average of 1st and 2nd order $k_{mod}^{(i)} : 4 \rightarrow 2.5, 8 \rightarrow 3, 16 \rightarrow 3.5, 32 \rightarrow 3.5, 192 \rightarrow 164$. •Unique element size: 8.	$\min_{\texttt{mean}} = 92.28$,$\max_{\texttt{mean}} = 192$, $\min_{\texttt{var}} = 332.1216$,$\max_{\texttt{var}} = 0$, $\texttt{var}(C_{\texttt{amod}}, i - (i + 1)) = 547.16667$,$\texttt{var}(C_{\texttt{amod}}, (i + 1) - i) = 547.16667$, $\texttt{mean}(C_{\texttt{amod}}, i - (i + 1)) = 100585.06$,$\texttt{mean}(C_{\texttt{amod}}, (i + 1) - i) = 100585.06$.

$3, 2, 5, 224$	$\bullet\mathbb{C} = 0.50707, \mathbb{U}_{pd} = 0.4, \mathbb{E} = 0.44267, k_1 = 165, k_2 = 165, mcs = 0.008$. •Average of 1st and 2nd order $k_{mod}^{(i)}$: $123 \rightarrow 165, 59 \rightarrow 166, 9 \rightarrow 166, 41 \rightarrow 166, 27 \rightarrow 166.5$. •Unique element size: 5.	$\min_{\texttt{mean}} = 26.04$,$\max_{\texttt{mean}} = 123$, $\min_{\texttt{var}} = 22.1184$,$\max_{\texttt{var}} = 0$, $\texttt{var}(C_{\texttt{amod}}, i - (i+1)) = 179.5$,$\texttt{var}(C_{\texttt{amod}}, (i+1) - i) = 179.04167$, $\texttt{mean}(C_{\texttt{amod}}, i - (i+1)) = 4085.08333$,$\texttt{mean}(C_{\texttt{amod}}, (i+1) - i) = 3940.9566$.
$23, 6, 0, 1300$	$\bullet\mathbb{C} = 0.86104, \mathbb{U}_{pd} = 0.34737, \mathbb{E} = 0.29522, k_1 = 141, k_2 = 173, mcs = 0.412$. •Average of 1st and 2nd order $k_{mod}^{(i)}$: $4 \rightarrow 2.5, 167 \rightarrow 140.5, 168 \rightarrow 144.5, 86 \rightarrow 144.5, 33 \rightarrow 148.5$. •Unique element size: 62.	$\min_{\texttt{mean}} = 95$,$\max_{\texttt{mean}} = 1204.8$, $\min_{\texttt{var}} = 12856$,$\max_{\texttt{var}} = 3253.76$, $\texttt{var}(C_{\texttt{amod}}, i - (i+1)) = 13327.75$,$\texttt{var}(C_{\texttt{amod}}, (i+1) - i) = 13343.9167$, $\texttt{mean}(C_{\texttt{amod}}, i - (i+1)) = 6454782.27$,$\texttt{mean}(C_{\texttt{amod}}, (i+1) - i) = 6488176.33$.
$23, 6, -43, 1300$	$\bullet\mathbb{C} = 0.77392, \mathbb{U}_{pd} = 0.33043, \mathbb{E} = 0.28909, k_1 = 156, k_2 = 172, mcs = 0.704$. •Average of 1st and 2nd order $k_{mod}^{(i)}$: $13 \rightarrow 124.5, 61 \rightarrow 160, 875 \rightarrow 163, 1271 \rightarrow 163, 871 \rightarrow 163$. •Unique element size: 250.	$\min_{\texttt{mean}} = 140.44$,$\max_{\texttt{mean}} = 1187.64$, $\min_{\texttt{var}} = 11080.8064$,$\max_{\texttt{var}} = 13749.3504$, $\texttt{var}(C_{\texttt{amod}}, i - (i+1)) = 7901.79167$,$\texttt{var}(C_{\texttt{amod}}, (i+1) - i) = 8025.95833$, $\texttt{mean}(C_{\texttt{amod}}, i - (i+1)) = 3875846.58$,$\texttt{mean}(C_{\texttt{amod}}, (i+1) - i) = 4110566.21$

Table 4: Display of Qual and Chain Test results for four linear congruential generators. For every LCG, each test observes the first 250 output values from it. The Qual Test considers n-grams of length 8, and the chain length l (partition size) is set to 10 for the Chain Test.

7.2 Basic Form-Fitters and Randomness

The *Seqbuild* PRNGs **mdrgen**,**optri**, and **qval** base their number generations on primary forms that do not depend on as much memory, relative to their instantiating parameters, and calculate their output values on less computational depth than the generators of Class 2. Each of the seeds of these generators center around a numerical sequence $\overrightarrow{S}$, the reference.

7.2.1 Description of Mathematical Structure

The **mdrgen** generator depends on a search algorithm to estimate (multiple,additive) pairs and their corresponding modulii. This search algorithm does not use linear systems of equations. Instead, it stores (multiple,additive) candidates for every pair of contiguous elements in the reference sequence, in effect, acting as a data mining algorithm as it looks for most frequent (multiple,additive) candidates. One deficit of the implementation of Modular Decomposed Representative (version one) is that it is unidirectional. For any contiguous pair of elements $s_i, s_{i+1} \in \overrightarrow{S}$, if $s_{i+1} \leqslant s_i$, the algorithm will assume a modulo operation occurred for a sign change from s_i. The orientation in *Seqbuild*'s implementation expects the actual (multiple,additive) pair, (m, a), to result in

$$T(s_i * m + a, s_i) \equiv T(s_i, s_{i-1}).$$

A ternary vector $T_{\overrightarrow{V}}(\overrightarrow{S})$ that has a high rate of change, indicated by $\mathcal{K}(T_{\overrightarrow{V}}(\overrightarrow{S}))$, requires a significantly greater number of modulii in the Modular Decomposed Representative's solution to effectively replicate $\overrightarrow{S}$. Specifically, the data structure of the solution consists of two sequences, $P_{(m,a)}$ and $P_{\texttt{mod}}$ (the sequence of modulii), the second of length shorter by one to the first. Each element of $P_{(m,a)}$ is of the form,

1. (start index,end index) $I_{\texttt{sub}}$ of $\overrightarrow{S}$,
2. Most frequent (multiple,additive) pairs for every sub-range,

$$[[(m_0, a_0), [i_{00}, i_{01}]], ..., [(m_j, a_j), [i_{j0}, i_{j1}]],$$

m_k are multiples,a_k are additives, and $[i_{p0}, i_{p1}]$ are index sub-ranges of $I_{\texttt{sub}}$.

Every i'th modulo in $P_{\texttt{mod}}$ connects the last element, the j'th index of $\overrightarrow{S}$, of the i'th sub-range,

$$j := P_{(m,a)}[i][-1][0][-1],$$

to $\overrightarrow{S}[j+1]$, such that for the last (multiple,additive) pair in $P(m,a)[i]$,

$$(m_l^{(i)}, a_l^{(i)}) := P_{(m,a)}[i][-1][1][-1][0],$$

then

$$(m_l^{(i)} * \overrightarrow{S}[j] + a_l^{(i)}) \mod P_{\texttt{mod}}[i] = \overrightarrow{S}[j+1].$$

The data mining to find $P_{(m,a)}$ and $P_{\texttt{mod}}$ is computationally expensive, as mentioned in the beginning of this paper, as well as in *Some Primitive Structures in Seqbuild*. The algorithm only applies to integer sequences. By default, the algorithm searches through all possible integer multiples in range

$$[1, f_q] \oplus ([-f_q, f_q] - \{0\}),$$

depending on if "exclude negative multiples" is set to True. The lone algorithm sets f_q to a default of

$$f_q := \max_{i=0}^{|\overrightarrow{S}|-2} \Big(\frac{\overrightarrow{S}[i+1]}{\overrightarrow{S}[i]} \Big).$$

This default proves to be computationally expensive when there are large differences between contiguous sequence elements. The *Seqbuild* system sets f_q to a default of 50 or so, when Modular Decomposed Representative is used for more structurally complex data structures, such as the primary forms of **mdrgen** and **ssino** (Search Space Iterating Network Operator).

The Modular Decomposed Representative outputs a sequence $\overrightarrow{S}_2$, of equal length to its reference $\overrightarrow{S}_2$, when it is given one input r, $r \in \mathcal{R}$. Let the function $\mathbb{Y}$ retrieve the (multiple,additive) pair, (m_i, a_i), from $P_{(m,a)}$ for the i'th index of $\overrightarrow{S}$. The function $\mathbb{Y}_2$ maps i to its j'th subrange of $P_{(m,a)}$. And the function $Q_{\texttt{last}}$ determines if that index i is the last one in its sub-range of $P_{(m,a)}$. The sequence $\overrightarrow{S}_2$ is then

$$\overrightarrow{S}_2[i] = \begin{cases} r \ \texttt{if} \ \ i \equiv 0, \\ \overrightarrow{S}_2[i-1] * \mathbb{Y}(i)[0] + \mathbb{Y}(i)[1] \ \texttt{if} \ \ Q_{\texttt{last}}(i) \equiv 0, \\ \Big(\overrightarrow{S}_2[i-1] * \mathbb{Y}(i)[0] + \mathbb{Y}(i)[1] \Big) \mod P_{\texttt{mod}}[\mathbb{Y}_2(i)] \ \texttt{otherwise}. \end{cases}$$

The seed for the **optri** generator includes three critical variables: a forward pairwise function, a backward pairwise function, a reference sequence $\overrightarrow{S}$, and a starting integer seed i. $\overrightarrow{S}$ is mapped to an upper-right triangular matrix M with the forward pairwise operator. It is this matrix M (mutable) that the generator draws values from whenever its cache is empty, via one or two schemes: $45-90$ split along the triangle's diagonal and rows, and flip-derivation (group operation on the matrix). *Some Primitive Structures on Seqbuild* provides more details on this generator.

The **qval** generator is instantiated with three important variables, besides from the reference integer sequence $\overrightarrow{S}$. Those variables are PRNGs G_i (index selector), G_l (length selector), and G_r (ranged modulus). $\overrightarrow{S}$ stays constant once a **qval** is instantiated. The generator G_i outputs two-dimensional index vectors, used to select any two elements in $\overrightarrow{S}$,

$$G_i() \to \big[0, \big|\overrightarrow{S}\big| - 1\big]^2.$$

Generator G_r outputs values in $\mathcal{R}^2$. The **qval** has a queue q that it draws values from. The queue is a stringized integer that concatenates stringized decimals d

to its right end, $d \in [0., 1.]$ and

$$G_i() \to (i_0, i_1),$$
$$r = \overrightarrow{S}[i_0]/\overrightarrow{S}[i_1],$$
$$d = r - \lfloor r \rfloor.$$

At any timestamp when **qval** is to output a real number, it first outputs a positive integer n_l from G_l. It obtains the stringized integer $x = q[: n_l]$, then updates $q := q[n_l :]$. If $|x| < n_l$, **qval** enters a loop, appends additional integers to q from integer pair quotients of $\overrightarrow{S}$, as mathematically defined. After exiting the loop, append $x_2 := q[: n_l - |x|]$ to x so that $|x| = n_l$, and update q by removing prefix x_2 from it. Convert x to an integer x', and the real number output x'' is

$$x'' = (x' \mod (r_1 - r_0)) + r_0,$$
$$G_r() \to [r_0, r_1].$$

The design of **qval** is related to that of **fit22**,**lps**, and **pid**. All of these generators are subclasses of one that requires generators (G_i, G_l, G_r) and an output queue q that is an extendible stringized integer. Generator **qval** is the fastest out of these four, since its primary computation is simple division. The primary computation of the other three involve finding fits between two-dimensional points.

7.2.2 Qual and Chain Tests on Mdrgen

Tests Results of Varying $\{\overrightarrow{S}_{\texttt{ref}}\}$ and G_0 on **mdrgen**			
#	$\{\overrightarrow{S}_{\texttt{ref}}\}, G_0$	Qual Test	Chain Test
1	$\overrightarrow{S}_{\texttt{ref}} = \left\langle 1, 1 \right\rangle, G_0 = cycle(\left\langle 1, 2, 3 \right\rangle)$	$\bullet\mathbb{C} = 0.5, \mathbb{U}_{pd} = 0.487, \mathbb{E} = 0.33276, k_1 = 249, k_2 = 166, mcs = 0.02$. •Average of 1st and 2nd order $k^{(i)}_{mod} : 4 \to 116.5, 7 \to 155.5, 8 \to 165, 3 \to 173, 14 \to 173.5$ •Unique element size: 3.	$\min_{\texttt{mean}} = 1$,$\max_{\texttt{mean}} = 3$, $\min_{\texttt{var}} = 0$,$\max_{\texttt{var}} = 0$, $\texttt{var}(C_{\texttt{amod}}, i - (i + 1)) = 8$,$\texttt{var}(C_{\texttt{amod}}, (i+1)-i) = 8$, $\texttt{mean}(C_{\texttt{amod}}, i - (i + 1)) = 2.\overline{66}$,$\texttt{mean}(C_{\texttt{amod}}, (i + 1) - i) = 2.\overline{66}$

2	$\overrightarrow{S}_{\texttt{ref}} = \left\langle 1,1 \right\rangle, G_0 = \texttt{lcg}(2,3,0,50)$	$\bullet\mathbb{C} = 0.58472, \mathbb{U}_{pd} = 0.37897, \mathbb{E} = 0.21314, k_1 = 249, k_2 = 178, mcs = 0.308$. •Average of 1st and 2nd order $k^{(i)}_{mod} : 4 \rightarrow 116.5, 7 \rightarrow 155.5, 8 \rightarrow 165, 3 \rightarrow 173, 14 \rightarrow 173.5$ •Unique element size: 12.	$\min_{\texttt{mean}} = 1.92$,$\max_{\texttt{mean}} = 32.96$, $\min_{\texttt{var}} = 1.5136$,$\max_{\texttt{var}} = 33.3184$, $\texttt{var}(C_{\texttt{amod}}, i-(i+1)) = 207.\overline{33}$,$\texttt{var}(C_{\texttt{amod}}, (i+1)-i) = 206.8\overline{33}$, $\texttt{mean}(C_{\texttt{amod}}, i-(i+1)) = 11115.\overline{8}$,$\texttt{mean}(C_{\texttt{amod}}, (i+1)-i) = 8183.97\overline{2}$
3	$\overrightarrow{S}_{\texttt{ref}} = \left\langle 1,1 \right\rangle, G_0 = \texttt{lcg}(2+\frac{1}{3}, 3-\frac{1}{9}, 1+\frac{2}{11}, 50)$	$\bullet\mathbb{C} = 0.57455, \mathbb{U}_{pd} = 0.31768, \mathbb{E} = 0.17226, k_1 = 249, k_2 = 172, mcs = 0.684$. •Average of 1st and 2nd order $k^{(i)}_{mod} : 3 \rightarrow 165, 4 \rightarrow 178, 7 \rightarrow 183, 6 \rightarrow 188, 8 \rightarrow 190.5$ •Unique element size: 47.	$\min_{\texttt{mean}} = 3.48$,$\max_{\texttt{mean}} = 39.6$, $\min_{\texttt{var}} = 16.4096$,$\max_{\texttt{var}} = 48.08$, $\texttt{var}(C_{\texttt{amod}}, i-(i+1)) = 277.25$,$\texttt{var}(C_{\texttt{amod}}, (i+1)-i) = 285$, $\texttt{mean}(C_{\texttt{amod}}, i-(i+1)) = 1.14823$,$\texttt{mean}(C_{\texttt{amod}}, (i+1)-i) = 12823.\overline{6}$
4	$\overrightarrow{S}_{\texttt{ref}} = \left\langle 1,1 \right\rangle, G_0 = (\texttt{lcg}(45,12,74,200.5) + \texttt{lcg}(23.5,-78+\frac{2}{3},66,223)) \bmod 50$.	$\bullet\mathbb{C} = 0.57455, \mathbb{U}_{pd} = 0.31768, \mathbb{E} = 0.17226, k_1 = 249, k_2 = 172, mcs = 0.684$. •Average of 1st and 2nd order $k^{(i)}_{mod} : 3 \rightarrow 156, 4 \rightarrow 170.5, 6 \rightarrow 180.5, 5 \rightarrow 184.5, 8 \rightarrow 191$ •Unique element size: 49.	$\min_{\texttt{mean}} = 2.64$,$\max_{\texttt{mean}} = 39.68$, $\min_{\texttt{var}} = 10.3904$,$\max_{\texttt{var}} = 61.6576$, $\texttt{var}(C_{\texttt{amod}}, i-(i+1)) = 290$,$\texttt{var}(C_{\texttt{amod}}, (i+1)-i) = 286.58\overline{3}$, $\texttt{mean}(C_{\texttt{amod}}, i-(i+1)) = 12687.\overline{33}$,$\texttt{mean}(C_{\texttt{amod}}, (i+1)-i) = 14421.8264$

5	$\overrightarrow{S}_{\texttt{ref}} = \left\langle 343, 99 \right\rangle, G_0 = cycle(\left\langle 1, 2, 3 \right\rangle)$	$\bullet \mathbb{C} = 0.19085, \mathbb{U}_{pd} = 0.10306, \mathbb{E} = 0.19768, k_1 = 223, k_2 = 235, mcs = 0.104.$ $\bullet$Average of 1st and 2nd order $k_{mod}^{(i)}$: $4 \rightarrow 189.5, 121 \rightarrow 199.5, 10 \rightarrow 202, 242 \rightarrow 203, 41 \rightarrow 205$ $\bullet$Unique element size: 30.	$\min_{\texttt{mean}} = -153.68$,$\max_{\texttt{mean}} = 24239.72$, $\min_{\texttt{var}} = 22740.2176$,$\max_{\texttt{var}} = 908502837.$, $\texttt{var}(C_{\texttt{amod}}, i - (i+1)) = 59261.75$,$\texttt{var}(C_{\texttt{amod}}, (i+1) - i) = 65666.7083$, $\texttt{mean}(C_{\texttt{amod}}, i - (i+1)) = 7109123800.$,$\texttt{mean}(C_{\texttt{amod}}, (i+1) - i) = 7345066100.$
6	$\overrightarrow{S}_{\texttt{ref}} = \left\langle 343, 99 \right\rangle, G_0 = \texttt{lcg}(2, 3, 0, 50)$	$\bullet \mathbb{C} = 0.18778, \mathbb{U}_{pd} = 0.1308, \mathbb{E} = 0.28979, k_1 = 217, k_2 = 219, mcs = 0.328.$ $\bullet$Average of 1st and 2nd order $k_{mod}^{(i)}$: $24 \rightarrow 180.5, 48 \rightarrow 185.0, 36 \rightarrow 196.5, 16 \rightarrow 197, 9 \rightarrow 198$ $\bullet$Unique element size: 52.	$\min_{\texttt{mean}} = -101.76$,$\max_{\texttt{mean}} = 1711.16$, $\min_{\texttt{var}} = 19365.7824$,$\max_{\texttt{var}} = 3400619.89$, $\texttt{var}(C_{\texttt{amod}}, i - (i+1)) = 6786.29167$,$\texttt{var}(C_{\texttt{amod}}, (i+1) - i) = 6902.45833$, $\texttt{mean}(C_{\texttt{amod}}, i - (i+1)) = 54308966.\overline{6}$,$\texttt{mean}(C_{\texttt{amod}}, (i+1) - i) = 58283502.7$
7	$\overrightarrow{S}_{\texttt{ref}} = \left\langle 343, 99 \right\rangle, G_0 = \texttt{lcg}(2+\frac{1}{3}, 3-\frac{1}{9}, 1+\frac{2}{11}, 50)$	$\bullet \mathbb{C} = 0.10686, \mathbb{U}_{pd} = 0.05742, \mathbb{E} = 0.25822, k_1 = 224, k_2 = 225, mcs = 0.696.$ $\bullet$Average of 1st and 2nd order $k_{mod}^{(i)}$: $4 \rightarrow 179.5, 25 \rightarrow 186.5, 51 \rightarrow 192.5, 73 \rightarrow 193, 60 \rightarrow 193$ $\bullet$Unique element size: 133.	$\min_{\texttt{mean}} = -156.4$,$\max_{\texttt{mean}} = 1180.48$, $\min_{\texttt{var}} = 42120.24$,$\max_{\texttt{var}} = 3304078.01$, $\texttt{var}(C_{\texttt{amod}}, i - (i+1)) = 5791.41\overline{6}$,$\texttt{var}(C_{\texttt{amod}}, (i+1) - i) = 4830.\overline{6}$, $\texttt{mean}(C_{\texttt{amod}}, i - (i+1)) = 80897984.9$,$\texttt{mean}(C_{\texttt{amod}}, (i+1) - i) = 66371457.1$

8	$\overrightarrow{S}_{\texttt{ref}} = \left\langle 343, 99 \right\rangle, G_0 = (\texttt{lcg}(45, 12, 74, 200.5) + \texttt{lcg}(23.5, -78 + \frac{2}{3}, 66, 223)) \mod 50$	$\bullet\mathbb{C} = 0.11946, \mathbb{U}_{pd} = 0.07151, \mathbb{E} = 0.28795, k_1 = 279, k_2 = 228, mcs = 0.7.$ •Average of 1st and 2nd order $k_{mod}^{(i)} : 60 \rightarrow 186, 61 \rightarrow 187.5, 66 \rightarrow 192, 59 \rightarrow 192, 48 \rightarrow 192.5$ •Unique element size: 140.	$\min_{\texttt{mean}} = -199.04, \max_{\texttt{mean}} = 2481.8,$ $\min_{\texttt{var}} = 36710.0384, \max_{\texttt{var}} = 14427782.8,$ $\texttt{var}(C_{\texttt{amod}}, i - (i+1)) = 8023.458\overline{3}, \texttt{var}(C_{\texttt{amod}}, (i+1) - i) = 7967.458\overline{3},$ $\texttt{mean}(C_{\texttt{amod}}, i - (i+1)) = 184184785., \texttt{mean}(C_{\texttt{amod}}, (i+1) - i) = 182944638.$
9	$\overrightarrow{S}_{\texttt{ref}} = \left\langle 2, 4, 8, 16, 32, 25, 25, 65, 65 \right\rangle, G_0 = cycle(\left\langle 1, 2, 3 \right\rangle)$	$\bullet\mathbb{C} = 0.32287, \mathbb{U}_{pd} = 0.20277, \mathbb{E} = 0.19290, k_1 = 148, k_2 = 110, mcs = 0.476.$ •Average of 1st and 2nd order $k_{mod}^{(i)} : 182 \rightarrow 121, 278 \rightarrow 126.5, 139 \rightarrow 126.5, 91 \rightarrow 127, 86 \rightarrow 130$ •Unique element size: 20.	$\min_{\texttt{mean}} = 2.24, \max_{\texttt{mean}} = 141.56,$ $\min_{\texttt{var}} = 0.5824, \max_{\texttt{var}} = 6735.9264,$ $\texttt{var}(C_{\texttt{amod}}, i - (i+1)) = 707.25, \texttt{var}(C_{\texttt{amod}}, (i+1) - i) = 766.625,$ $\texttt{mean}(C_{\texttt{amod}}, i - (i+1)) = 128779.521, \texttt{mean}(C_{\texttt{amod}}, (i+1) - i) = 128364.068$
10	$\overrightarrow{S}_{\texttt{ref}} = \left\langle 2, 4, 8, 16, 32, 25, 25, 65, 65 \right\rangle, G_0 = \texttt{lcg}(2, 3, 0, 50)$	$\bullet\mathbb{C} = 0.32287, \mathbb{U}_{pd} = 0.07736, \mathbb{E} = 0.08243, k_1 = 139, k_2 = 110, mcs = 0.68.$ •Average of 1st and 2nd order $k_{mod}^{(i)} : 4 \rightarrow 102.5, 1192 \rightarrow 125.5, 1730 \rightarrow 125.5, 2012 \rightarrow 125.5, 1429 \rightarrow 125.5$ •Unique element size: 49.	$\min_{\texttt{mean}} = -52.4, \max_{\texttt{mean}} = 634.92,$ $\min_{\texttt{var}} = 27112.64, \max_{\texttt{var}} = 464964.314,$ $\texttt{var}(C_{\texttt{amod}}, i - (i+1)) = 35775.8\overline{3}, \texttt{var}(C_{\texttt{amod}}, (i+1) - i) = 3541.5,$ $\texttt{mean}(C_{\texttt{amod}}, i - (i+1)) = 16568928., \texttt{mean}(C_{\texttt{amod}}, (i+1) - i) = 9853552.58$

11	$\overrightarrow{S}_{\texttt{ref}} = \Big\langle 2,4,8,16,32, 25,25,65,65 \Big\rangle, G_0 = \texttt{lcg}(2+\frac{1}{3}, 3-\frac{1}{9}, 1+\frac{2}{11}, 50)$	•$\mathbb{C} = 0.27015, \mathbb{U}_{pd} = 0.15189, \mathbb{E} = 0.14443, k_1 = 134, k_2 = 116, mcs = 0.692$. •Average of 1st and 2nd order $k_{mod}^{(i)}: 4 \rightarrow 94, 1248 \rightarrow 121.5, 1246 \rightarrow 123, 1586 \rightarrow 124, 3172 \rightarrow 124$ •Unique element size: 66.	$\min_{\texttt{mean}} = -61.32$,$\max_{\texttt{mean}} = 1526.52$, $\min_{\texttt{var}} = 18934.7776$,$\max_{\texttt{var}} = 1702488.\overline{8}$, $\texttt{var}(C_{\texttt{amod}}, i-(i+1)) = 7557.58\overline{3}$,$\texttt{var}(C_{\texttt{amod}}, (i+1)-i) = 8029.54167$, $\texttt{mean}(C_{\texttt{amod}}, i-(i+1)) = 63060615.1$,$\texttt{mean}(C_{\texttt{amod}}, (i+1)-i) = 55616722.6$
12	$\overrightarrow{S}_{\texttt{ref}} = \Big\langle 2,4,8,16,32, 25,25,65,65 \Big\rangle, G_0 = (\texttt{lcg}(45,12,74,200.5) + \texttt{lcg}(23.5,-78+\frac{2}{3},66,223)) \bmod 50.$	•$\mathbb{C} = 0.16412, \mathbb{U}_{pd} = 0.08818, \mathbb{E} = 0.12172, k_1 = 137, k_2 = 112, mcs = 0.688$. •Average of 1st and 2nd order $k_{mod}^{(i)}: 2615 \rightarrow 124.5, 2832 \rightarrow 124.5, 4 \rightarrow 126, 2874 \rightarrow 126, 2796 \rightarrow 126$ •Unique element size: 69.	$\min_{\texttt{mean}} = -76.48$,$\max_{\texttt{mean}} = 1220.4$, $\min_{\texttt{var}} = 29309.1296$,$\max_{\texttt{var}} = 2479723.76$, $\texttt{var}(C_{\texttt{amod}}, i-(i+1)) = 7638.8\overline{3}$,$\texttt{var}(C_{\texttt{amod}}, (i+1)-i) = 7986.1\overline{6}$, $\texttt{mean}(C_{\texttt{amod}}, i-(i+1)) = 80944445.3$,$\texttt{mean}(C_{\texttt{amod}}, (i+1)-i) = 73228158.6$
13	$\overrightarrow{S}_{\texttt{ref}} = \Big\langle 14,1400,15,-43, -410,32,108, 90,934 \Big\rangle, G_0 = cycle(\Big\langle 1,2,3 \Big\rangle)$	•$\mathbb{C} = 0.02733, \mathbb{U}_{pd} = 0.016544, \mathbb{E} = 0.08097, k_1 = 145, k_2 = 185, mcs = 0.7$. •Average of 1st and 2nd order $k_{mod}^{(i)}: 333333 \rightarrow 138.5, 111111 \rightarrow 138.5, 499993 \rightarrow 139, 499999 \rightarrow 139, 3003 \rightarrow 139$ •Unique element size: 194.	$\min_{\texttt{mean}} = -2052.76$,$\max_{\texttt{mean}} = 311280.72$, $\min_{\texttt{var}} = 14789551.2$,$\max_{\texttt{var}} = 1.06249699 * 10^{12}$, $\texttt{var}(C_{\texttt{amod}}, i-(i+1)) = 1252536.92$,$\texttt{var}(C_{\texttt{amod}}, (i+1)-i) = 1253404.21$, $\texttt{mean}(C_{\texttt{amod}}, i-(i+1)) = 1.53004166 * 10^{13}$,$\texttt{mean}(C_{\texttt{amod}}, (i+1)-i) = 1.52988602 * 10^{13}$

14	$\overrightarrow{S}_{\texttt{ref}} = \Big\langle 14, 1400, 15, -43, -410, 32, 108, 90, 934 \Big\rangle, G_0 = \texttt{lcg}(2, 3, 0, 50)$	$\bullet \mathbb{C} = 0.0227, \mathbb{U}_{pd} = 0.01579, \mathbb{E} = 0.02349, k_1 = 141, k_2 = 180, mcs = 0.7.$ •Average of 1st and 2nd order $k_{mod}^{(i)}$: $41653 \rightarrow 154,999982 \rightarrow 155,999986 \rightarrow 155,999932 \rightarrow 155,999942 \rightarrow 155$ •Unique element size: 194.	$\min_{\texttt{mean}} = -70127.44, \max_{\texttt{mean}} = 11346653.8,$ $\min_{\texttt{var}} = 8.83823070 * 10^{10}, \max_{\texttt{var}} = 1.48456857 * 10^{15},$ $\texttt{var}(C_{\texttt{amod}}, i - (i + 1)) = 2520874.17 * 10^6, \texttt{var}(C_{\texttt{amod}}, (i+1)-i) = 2520804.88 * 10^6,$ $\texttt{mean}(C_{\texttt{amod}}, i - (i + 1)) = 4.30269005 * 10^{13}, \texttt{mean}(C_{\texttt{amod}}, (i + 1) - i) = 4.30303325 * 10^{13}$
15	$\overrightarrow{S}_{\texttt{ref}} = \Big\langle 14, 1400, 15, -43, -410, 32, 108, 90, 934 \Big\rangle, G_0 = \texttt{lcg}(2+\frac{1}{3}, 3-\frac{1}{9}, 1+\frac{2}{11}, 50)$	$\bullet \mathbb{C} = 0.01616, \mathbb{U}_{pd} = 0.01174, \mathbb{E} = 0.00805, k_1 = 127, k_2 = 177, mcs = 0.704.$ •Average of 1st and 2nd order $k_{mod}^{(i)}$: $333294 \rightarrow 142,166647 \rightarrow 142.5,499941 \rightarrow 142.5,499977 \rightarrow 142.5,499951 \rightarrow 142.5$ •Unique element size: 202.	$\min_{\texttt{mean}} = -60046315.2, \max_{\texttt{mean}} = 3711.04,$ $\min_{\texttt{var}} = 8.51813639 * 10^{16}, \max_{\texttt{var}} = 59154910.5,$ $\texttt{var}(C_{\texttt{amod}}, i - (i + 1)) = 1509340.75, \texttt{var}(C_{\texttt{amod}}, (i + 1) - i) = 1509647.42,$ $\texttt{mean}(C_{\texttt{amod}}, i - (i + 1)) = 2.31419204 * 10^{13}, \texttt{mean}(C_{\texttt{amod}}, (i + 1) - i) = 2.31414486 * 10^{13}$

16	$\overrightarrow{S}_{\texttt{ref}} = \Big\langle 14, 1400, 15, -43, -410, 32, 108, 90, 934 \Big\rangle, G_0 = (\texttt{lcg}(45, 12, 74, 200.5) + \texttt{lcg}(23.5, -78 + \frac{2}{3}, 66, 223)) \bmod 50$	•$\mathbb{C} = 0.01065, \mathbb{U}_{pd} = 0.00855, \mathbb{E} = 0.0213, k_1 = 130, k_2 = 174, mcs = 0.7$. •Average of 1st and 2nd order $k_{mod}^{(i)}$: $249973 \rightarrow 156,999892 \rightarrow 156,499946 \rightarrow 156,999978 \rightarrow 156,499989 \rightarrow 156$ •Unique element size: 202.	$\min_{\texttt{mean}} = -16855.84$,$\max_{\texttt{mean}} = 18266.08$, $\min_{\texttt{var}} = 3.83328148 * 10^9$,$\max_{\texttt{var}} = 5.64630411 * 10^9$, $\texttt{var}(C_{\texttt{amod}}, i - (i+1)) = 40991.0417$,$\texttt{var}(C_{\texttt{amod}}, (i+1) - i) = 59399.4167$, $\texttt{mean}(C_{\texttt{amod}}, i - (i+1)) = 1.11949013 * 10^{10}$,$\texttt{mean}(C_{\texttt{amod}}, (i+1) - i) = 1.77191484 * 10^{10}$

Table 5: Display of Qual and Chain Test results for four unique reference sequences of $\overrightarrow{S}_{\texttt{ref}}$ and four unique parameter generators assigned to variable G_0. Qual Test was run with 250 iterations and n-gram of $n = 8$. Chain Test was run with 250 iterations and a chain length of 10. Variables $\overrightarrow{S}_{\texttt{ref}}$ and G_0 are the main variables comprising the seed. There is a general positive correlation between the randomness of G_0 and the output quality of **mdrgen**. Reference sequences that contain elements "irregular" in relation to each other also generally produce more randomness in **mdrgen**'s output quality. The term "irregular", in the context of the Modular Decomposed Representative's fitting algorithm, is a label for real numbers that share a relatively few number of factors with each other. The **mdrgen** generator type is set to #1, the more basic type.

The **mdrgen** was designed in attempt of expanding the output space of $G_0 \circ \overrightarrow{S}_{\texttt{ref}}$. Table 5 results indicate positive correlation between **mdrgen** and its parameter generator's output size, $|G_0|$. There is also a positive correlation between **mdrgen** and the set cardinality of contiguous pairwise difference vector for $\overrightarrow{S}_{\texttt{ref}}$,

$$\Delta(\overrightarrow{S}_{\texttt{ref}}) = \langle \overrightarrow{S}_{\texttt{ref}}[i+1] - \overrightarrow{S}_{\texttt{ref}}[i] \forall i \in [0, |\overrightarrow{S}_{\texttt{ref}}| - 2] \rangle.$$

The coverage measure, $\mathbb{C}$, can be somewhat misleading because there is not a positive correlation between the measure's value and the predictability of the generator output. Generators 15 and 16 each produced 202 unique elements, with coverage in in the range of $[0.01, 0.02]$, while generators 1 and 2 produced three and 12 unique elements, respectively, yet their coverage is in the range of $[0.5, 0.6]$. Similarly, the unidirectional weighted point distance measure, $\mathbb{U}$, fails to act as an easy distinguisher between more and less predictable generators. Since these two measures are normalized to the range $[0., 1.]$, they do

not give the exact scale of space (difference) between values of a PRNG's number sequence. The Kolmogorov complexity measures k_1 and k_2, as defined in a previous section of this paper, is also not a simple measure of how "random" a PRNG is. When these measures of Qual Test are taken together, however, they make for a signature on the output qualities of the generator.

Definition 7.1 (pseudo-random number generator signature)**.** A *signature* Q for a pseudo-random number generator G is a map M comprised of values $\overrightarrow{V}$, every i'th value a variable output for a metrological category c_i, $c_i \in \mathcal{C}$. The map M is a signature for G if there exists a distinguisher D for G in a set of comparable generators W_G. That is, for any generator $G_2 \in W_G \backslash \{G\}$, its map M_2 on category set $\mathcal{C}$ has difference with M of G satisfying

$$Pr[D(M - M_2)] > 1/|W_G|.$$

The PRNG I/O Test mitigates the normalization problem of $\mathbb{C}$, $\mathbb{U}$, and the Kolmogorov complexity measures by allowing for a ranged modulus $r_{\texttt{mod}}$ to be applied to both input and output PRNGs, measuring their randomness in the range of $r_{\texttt{mod}}$.

7.2.3 Cycle Test on Optri

Varying $\{\overrightarrow{S}_{\texttt{ref}}\}$, $n_{\texttt{iter}}$, and $d_{\texttt{mod}}$ on **optri**			
#	Parameters	$\overrightarrow{S}_1$	$\overrightarrow{S}_2$
1	$n_{\texttt{iter}} = 250$. $d_{\texttt{mod}} = \varnothing$.	seq: mcs $\rightarrow$ 0.01367, $\mathcal{D}_{\texttt{cycle}} \rightarrow 4.27856$. set: mcs $\rightarrow$ 0.01339, $\mathcal{D}_{\texttt{cycle}} \rightarrow 4.24945$. valfreq: mcs $\rightarrow$ 0.00352, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.65196$. l-seq: mcs $\rightarrow$ 0.01339, $\mathcal{D}_{\texttt{cycle}} \rightarrow 4.24945$. $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow$ 0.00107.	seq: mcs $\rightarrow$ 0.005804, $\mathcal{D}_{\texttt{cycle}} \rightarrow 1.730970$. set: mcs $\rightarrow$ 0.005813, $\mathcal{D}_{\texttt{cycle}} \rightarrow 1.730970$. valfreq: mcs $\rightarrow$ 0.003072, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.527542$. l-seq: mcs $\rightarrow$ 0.00581, $\mathcal{D}_{\texttt{cycle}} \rightarrow 1.730970$. $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow$ 0.

2	$n_{\texttt{iter}} = 500.$ $d_{\texttt{mod}} = \varnothing$	seq: $\texttt{mcs} \rightarrow 0.00622$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 3.93052.$ set: $\texttt{mcs} \rightarrow 0.00615$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 3.91583.$ valfreq: $\texttt{mcs} \rightarrow 0.00164$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.57778.$ l-seq: $\texttt{mcs} \rightarrow 0.00615$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 3.91583.$ $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.00044.$	seq: $\texttt{mcs} \rightarrow 0.00314$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 1.89564.$ set: $\texttt{mcs} \rightarrow 0.00315$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 1.89564.$ valfreq: $\texttt{mcs} \rightarrow 0.00161$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.57303.$ l-seq: $\texttt{mcs} \rightarrow 0.00315$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 1.89564.$ $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.00493.$
3	$n_{\texttt{iter}} = 800.$ $d_{\texttt{mod}} = \varnothing$	seq: $\texttt{mcs} \rightarrow 0.00372$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 3.78688.$ set: $\texttt{mcs} \rightarrow 0.0037$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 3.77807.$ valfreq: $\texttt{mcs} \rightarrow 0.00099$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.54667.$ l-seq: $\texttt{mcs} \rightarrow 0.0037$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 3.77807.$ $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.00026.$	seq: $\texttt{mcs} \rightarrow 0.00192$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 1.84638.$ set: $\texttt{mcs} \rightarrow 0.00193$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 1.84638.$ valfreq: $\texttt{mcs} \rightarrow 0.0001$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.5625.$ l-seq: $\texttt{mcs} \rightarrow 0.00193$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 1.84638.$ $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.00294.$
4	$n_{\texttt{iter}} = 250.$ $d_{\texttt{mod}} = 32$	seq: $\texttt{mcs} \rightarrow 0.01543$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 4.53094.$ set: $\texttt{mcs} \rightarrow 0.01203$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 3.79626].$ valfreq: $\texttt{mcs} \rightarrow 0.00419$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.84132.$ l-seq: $\texttt{mcs} \rightarrow 0.01203$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 3.79626].$ $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.00418.$	seq: $\texttt{mcs} \rightarrow 0.06975$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 20.02670.$ set: $\texttt{mcs} \rightarrow 0.06691$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 19.47759.$ valfreq: $\texttt{mcs} \rightarrow 0.02538$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 6.71875.$ l-seq: $\texttt{mcs} \rightarrow 0.06691$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 19.4776.$ $\nabla_{index} \rightarrow 6.21875$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.$

5	$n_{\texttt{iter}} = 500.$ $d_{\texttt{mod}} = 32$	seq: $\texttt{mcs} \rightarrow 0.00931$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 5.43584.$ set: $\texttt{mcs} \rightarrow 0.00709$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 4.43211.$ valfreq: $\texttt{mcs} \rightarrow 0.00225$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.98814.$ l-seq: $\texttt{mcs} \rightarrow 0.00709$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 4.43211.$ $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.00231.$	seq: $\texttt{mcs} \rightarrow 0.07211$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 41.8125.$ set: $\texttt{mcs} \rightarrow 0.07032$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 40.8125.$ valfreq: $\texttt{mcs} \rightarrow 0.02781$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 14.46875.$ l-seq: $\texttt{mcs} \rightarrow 0.07032$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 40.8125.$ $\nabla_{index} \rightarrow 12.375$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.03423.$
6	$n_{\texttt{iter}} = 800.$ $d_{\texttt{mod}} = 32$	seq: $\texttt{mcs} \rightarrow 0.00723$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 6.76605.$ set: $\texttt{mcs} \rightarrow 0.00562$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 5.58694.$ valfreq: $\texttt{mcs} \rightarrow 0.0016$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 1.19652.$ l-seq: $\texttt{mcs} \rightarrow 0.00562$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 5.58694.$ $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.00162.$	seq: $\texttt{mcs} \rightarrow 0.07354$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 68.34375.$ set: $\texttt{mcs} \rightarrow 0.07171$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 66.625.$ valfreq: $\texttt{mcs} \rightarrow 0.03064$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 23.8125.$ l-seq: $\texttt{mcs} \rightarrow 0.07171$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 66.625.$ $\nabla_{index} \rightarrow 19.4375$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.02894.$
7	$n_{\texttt{iter}} = 250.$ $d_{\texttt{mod}} = 16$	seq: $\texttt{mcs} \rightarrow 0.038411$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 11.078125.$ set: $\texttt{mcs} \rightarrow 0.0226045$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 7.493576.$ valfreq: $\texttt{mcs} \rightarrow 0.011234$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 2.867187.$ l-seq: $\texttt{mcs} \rightarrow 0.022604$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 7.493576.$ $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.02869472.$	seq: $\texttt{mcs} \rightarrow 0.136625$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 39.625.$ set: $\texttt{mcs} \rightarrow 0.12825$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 37.125.$ valfreq: $\texttt{mcs} \rightarrow 0.056125$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 14.4375.$ l-seq: $\texttt{mcs} \rightarrow 0.12825$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 37.125.$ $\nabla_{index} \rightarrow 10.9375$, $\texttt{var}_{\texttt{norm}} \rightarrow 0..$

8	$n_{\texttt{iter}} = 500.$ $d_{\texttt{mod}} = 16$	seq: $\texttt{mcs} \rightarrow 0.04256$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 24.71875.$ set: $\texttt{mcs} \rightarrow 0.02463$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 15.94271.$ valfreq: $\texttt{mcs} \rightarrow 0.01316$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 6.70313.$ l-seq: $\texttt{mcs} \rightarrow 0.02463$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 15.94271.$ $\nabla_{index} \rightarrow 6.125$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.02642.$	seq: $\texttt{mcs} \rightarrow 0.142093$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 82.3125.$ set: $\texttt{mcs} \rightarrow 0.13322$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 77.5625.$ valfreq: $\texttt{mcs} \rightarrow 0.05788$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 29.9375.$ l-seq: $\texttt{mcs} \rightarrow 0.13322$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 77.5625.$ $\nabla_{index} \rightarrow 19.1875$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.0525.$
9	$n_{\texttt{iter}} = 800.$ $d_{\texttt{mod}} = 16$	seq: $\texttt{mcs} \rightarrow 0.04442$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 41.21875.$ set: $\texttt{mcs} \rightarrow 0.02707$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 26.38021.$ valfreq: $\texttt{mcs} \rightarrow 0.01369$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 11.39063.$ l-seq: $\texttt{mcs} \rightarrow 0.02707$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 26.38021.$ $\nabla_{index} \rightarrow 10.26563$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.02976.$	seq: $\texttt{mcs} \rightarrow 0.1435$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 133.1875.$ set: $\texttt{mcs} \rightarrow 0.13629$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 126.375.$ valfreq: $\texttt{mcs} \rightarrow 0.06117$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 48.625.$ l-seq: $\texttt{mcs} \rightarrow 0.13629$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 126.375.$ $\nabla_{index} \rightarrow 26.0$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.04094.$
10	$n_{\texttt{iter}} = 250.$ $d_{\texttt{mod}} = 8$	seq: $\texttt{mcs} \rightarrow 0.24025$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 68.5.$ set: $\texttt{mcs} \rightarrow 0.08263$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 28.375.$ valfreq: $\texttt{mcs} \rightarrow 0.1055$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 27.5.$ l-seq: $\texttt{mcs} \rightarrow 0.08263$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 28.375.$ $\nabla_{index} \rightarrow 11.5$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.30389.$	seq: $\texttt{mcs} \rightarrow 0.26113$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 74.375.$ set: $\texttt{mcs} \rightarrow 0.22175$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 64.25.$ valfreq: $\texttt{mcs} \rightarrow 0.1145$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 29.875.$ l-seq: $\texttt{mcs} \rightarrow 0.22175$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 64.25.$ $\nabla_{index} \rightarrow 14.875$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.06351.$

11	$n_{\texttt{iter}} = 500.$ $d_{\texttt{mod}} = 8$	seq: mcs → 0.24944, $\mathcal{D}_{\texttt{cycle}} \rightarrow 146.5.$ set: mcs → 0.09, $\mathcal{D}_{\texttt{cycle}} \rightarrow 58.125.$ valfreq: mcs → 0.07888, $\mathcal{D}_{\texttt{cycle}} \rightarrow 44.125.$ l-seq: mcs → 0.09, $\mathcal{D}_{\texttt{cycle}} \rightarrow 58.125.$ $\nabla_{index} \rightarrow 17.5$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.24855.$	seq: mcs → 0.26881, $\mathcal{D}_{\texttt{cycle}} \rightarrow 154.125.$ set: mcs → 0.23319, $\mathcal{D}_{\texttt{cycle}} \rightarrow 135.25.$ valfreq: mcs → 0.11138, $\mathcal{D}_{\texttt{cycle}} \rightarrow 60.625.$ l-seq: mcs → 0.23319, $\mathcal{D}_{\texttt{cycle}} \rightarrow 135.25.$ $\nabla_{index} \rightarrow 20.125$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.13287.$
12	$n_{\texttt{iter}} = 800.$ $d_{\texttt{mod}} = 8$	seq: mcs → 0.25187, $\mathcal{D}_{\texttt{cycle}} \rightarrow 239.625.$ set: mcs → 0.08734, $\mathcal{D}_{\texttt{cycle}} \rightarrow 95.25.$ valfreq: mcs → 0.057070, $\mathcal{D}_{\texttt{cycle}} \rightarrow 51.5.$ l-seq: mcs → 0.08734, $\mathcal{D}_{\texttt{cycle}} \rightarrow 95.25.$ $\nabla_{index} \rightarrow 19.75$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.23153.$	seq: mcs → 0.26883, $\mathcal{D}_{\texttt{cycle}} \rightarrow 249.875.$ set: mcs → 0.23941, $\mathcal{D}_{\texttt{cycle}} \rightarrow 223..$ valfreq: mcs → 0.11531, $\mathcal{D}_{\texttt{cycle}} \rightarrow 97.125.$ l-seq: mcs → 0.23941, $\mathcal{D}_{\texttt{cycle}} \rightarrow 223..$ $\nabla_{index} \rightarrow 23.75$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.12055.$

Table 6: Display of Cycle Test results for two unique reference sequences of $\overrightarrow{S}_{\texttt{ref}}$, $\overrightarrow{S}_1 = \left\langle 343, 99 \right\rangle$ and $\overrightarrow{S}_2 = \left\langle 2, 4, 8, 16, 32, 25, 25, 65, 65 \right\rangle$. Testing of $\overrightarrow{S}_1$ is run with (pre-n= 3,post-n= 5) and $\overrightarrow{S}_2$ with (pre-n= 1, post-n= 3). Cycle Test indicates stronger scores for existence of cycles for greater iterations and when generator output is modulated by $d_{\texttt{mod}}$. The constant instantiation parameters for the **optri** generator are "gentype"= 1, addition for"forward function", subtraction for "backward function", and false for "add noise". These parameters were chosen to demonstrate **optri** performance on familiar functions (addition and subtraction), with no additional noise from the **optri** parameter generator.

7.2.4 Qual Test on Qval

The **qval** generator requires two two-vector generators, G_i (index) and G_r (ranged modulii). *Seqbuild* can cast any PRNG that outputs single real numbers into one outputting other formats. Denote $\mathcal{U}_j^d$, d the length of the reference

sequence, as the casting map for any PRNG G that outputs single numbers,

$$\mathcal{U}_j^d(G) = \big\langle \lfloor G() \mod j \rfloor \forall i \in [0, d-1] \big\rangle.$$

Denote $\mathcal{U}_{\texttt{mrange}}$ as the casting map for G to output ranged modulii,

$$\mathcal{U}_{\texttt{mrange}}(G) = \texttt{sorted}\big\langle G(), G() \big\rangle.$$

Varying G_i and G_l on **qval**			
Parameters		$\overrightarrow{S}_1$	$\overrightarrow{S}_2$
G_i $\mathcal{U}^2_{\overrightarrow{\lvert S_? \rvert}}(\texttt{constant}(5))$, G_r $\mathcal{U}_{\texttt{mrange}}(\texttt{constant}(5))$	$:=$ $:=$	$\bullet\mathbb{C} = 0, \mathbb{U}_{pd} = 0, \mathbb{E} = 0, k_1 = 1, k_2 = 1, mcs = 0.$ $\bullet$Average of 1st and 2nd order $k^{(i)}_{mod}$: $5 \rightarrow 1$ $\bullet$Unique element size: 1.	$\bullet\mathbb{C} = 0, \mathbb{U}_{pd} = 0, \mathbb{E} = 0, k_1 = 1, k_2 = 1, mcs = 0.$ $\bullet$Average of 1st and 2nd order $k^{(i)}_{mod}$: $5 \rightarrow 1$ $\bullet$Unique element size: 1.
G_i $\mathcal{U}^2_{\overrightarrow{\lvert S_? \rvert}}(\texttt{constant}(5))$, G_r $\mathcal{U}_{\texttt{mrange}}(\texttt{lcg}(79, -117, 322, 917))$	$:=$ $:=$	$\bullet\mathbb{C} = 0.78740, \mathbb{U}_{pd} = 0.32934, \mathbb{E} = 0.30718, k_1 = 165, k_2 = 180, mcs = 0.652.$ $\bullet$Average of 1st and 2nd order $k^{(i)}_{mod}$: $319 \rightarrow 161, 118 \rightarrow 161.5, 205 \rightarrow 162, 48 \rightarrow 162.5, 149 \rightarrow 162.5$ $\bullet$Unique element size: 195.	$\bullet\mathbb{C} = 0.78740, \mathbb{U}_{pd} = 0.32934, \mathbb{E} = 0.30718, k_1 = 165, k_2 = 180, mcs = 0.652.$ $\bullet$Average of 1st and 2nd order $k^{(i)}_{mod}$: $319 \rightarrow 161, 118 \rightarrow 161.5, 205 \rightarrow 162, 48 \rightarrow 162.5, 149 \rightarrow 162.5$ $\bullet$Unique element size: 195.
G_i $\mathcal{U}^2_{\overrightarrow{\lvert S_? \rvert}}(\texttt{lcg}(79, -117, 322, 917))$, G_r $\mathcal{U}_{\texttt{mrange}}(\texttt{constant}(5))$	$:=$ $:=$	$\bullet\mathbb{C} = 0, \mathbb{U}_{pd} = 0, \mathbb{E} = 0, k_1 = 1, k_2 = 1, mcs = 0.$ $\bullet$Average of 1st and 2nd order $k^{(i)}_{mod}$: $5 \rightarrow 1$ $\bullet$Unique element size: 1.	$\bullet\mathbb{C} = 0, \mathbb{U}_{pd} = 0, \mathbb{E} = 0, k_1 = 1, k_2 = 1, mcs = 0.$ $\bullet$Average of 1st and 2nd order $k^{(i)}_{mod}$: $5 \rightarrow 1$ $\bullet$Unique element size: 1.
G_i $\mathcal{U}^2_{\overrightarrow{\lvert S_? \rvert}}(\texttt{lcg}(79, -117, 322, 917))$, G_r $\mathcal{U}_{\texttt{mrange}}(\texttt{lcg}(79, -117, 322, 917))$	$:=$ $:=$	$\bullet\mathbb{C} = 0.79376, \mathbb{U}_{pd} = 0.3415, \mathbb{E} = 0.30488, k_1 = 164, k_2 = 185, mcs = 0.652.$ $\bullet$Average of 1st and 2nd order $k^{(i)}_{mod}$: $118 \rightarrow 157.5, 136 \rightarrow 160, 48 \rightarrow 160.5, 205 \rightarrow 160.5, 198 \rightarrow 161.5$ $\bullet$Unique element size: 195.	$\bullet\mathbb{C} = 0.61966, \mathbb{U}_{pd} = 0.2277, \mathbb{E} = 0.28948, k_1 = 160, k_2 = 181, mcs = 0.702.$ $\bullet$Average of 1st and 2nd order $k^{(i)}_{mod}$: $390 \rightarrow 160.5, 195 \rightarrow 162.5, 1216 \rightarrow 162.5, 1170 \rightarrow 164.5, 1217 \rightarrow 164.5$ $\bullet$Unique element size: 223.

Table 7: Display of Qual Test results for two unique reference sequences, $\overrightarrow{S}_1 = \langle 5, 5 \rangle$ and $\overrightarrow{S}_2 = \langle 2, 4, 8, 16, 32, 25, 25, 65, 65 \rangle$. Since the only quotient is 1 in $\overrightarrow{S}_1$, the *qval* on this sequence completely relies on G_r to produce randomness in its output. There are also multiple 0 unit interval quotient values in $\overrightarrow{S}_2$, but the sequence is not entirely of $\varnothing$. Predictable to the design of **qval**, it produces the most unique values when applied on the hardcore predicates $\overrightarrow{S}_2$ and $\mathcal{U}^2_{\overrightarrow{|S_?|}}(\texttt{lcg}(79, -117, 322, 917))$.

7.3 Qual and Cycle Tests on Auto-Correcting Generators

7.3.1 Description of Mathematical Structure

There was some writing in *Some Primitive Structures in Seqbuild* dedicated to the concepts of "self-modifying" and "self-reflecting" programs. Any PRNG that modifies its second-order seed is self-modifying. The two primitive structures, **gg** (gauge-guided) and **shadow**, also perform self-reflecting calculations that shift their pre-image real numbers into output. Generator **gg** observes the metrics of coverage, $\mathcal{C}$, and unidirectional weighted point distance $\mathbb{U}_{pd}$, on its pre-image values. If those pre-image values fall into the same pattern, not yet quantitatively defined, then **gg** deters away from that pattern by adding noise to those pre-image values before outputting them. Generator **shadow** relies on one of operation triangle, factor vector, ternary vector, and modular decomposed representative as the reference data structure to deter away from. The operation triangle and modular decomposed representative data structures have already been elaborated on in the description of the generators **optri** and **mdrgen**, respectively, that use them as their primary forms.

The **gg** generator observes strictly $\mathcal{C}$ or $\mathbb{U}_{pd}$. Its instantiation parameter set is the following.

- Parameter generator, P_0.
- Base vector length range, l_r.
- Density measure (size of records on metric), δ_r.
- Super-range with respect to generator image (output), $r_{\texttt{super}}$.
- N-gram length used for calculating metric values, i_n.
- Target measure m_t, $\mathcal{C}$ (coverage) $\oplus \mathbb{U}_{pd}$ (unidirectional weighted point distance).
- Allow sub-range drift, $r_{\texttt{drift}}$ (boolean).

When its output queue is null, **gg** loads up a new base vector,

$$\overrightarrow{V}_{\texttt{base}} := \langle r_{\texttt{super}}[0] + (P_0() \mod (r_{\texttt{super}}[1] - r_{\texttt{super}}[0]))\forall j \in [0, l]. \rangle,$$

$$l := l_r[0] + (\lfloor P_0() \rfloor \mod (l_r[1] - l_r[0])).$$

Generator then calculates a new category size $s_{\texttt{cat}}$ (the default category size range is $c_{gg} = [3, 11]$),

$$s_{\texttt{cat}} := c_{gg}[0] + (\lfloor P_0() \rfloor \mod (c_{gg}[1] - c_{gg}[0])).$$

This category size is a uniform partitioning of the unit interval, and the labeling function L is

$$L : [0., 1.] \longrightarrow [1, s_{\texttt{cat}}],$$
$$L(d) = \lceil d / s_{\texttt{cat}} \rceil.$$

Generator adjusts $\overrightarrow{V}_{\texttt{base}}$ such that $L(m_t(\overrightarrow{V}_{\texttt{base}}))$ results in a maximum value according to a measure with **gg**'s category vector $\overrightarrow{V}_{\texttt{cat}}$, of length at most δ_r. This measure with $\overrightarrow{V}_{\texttt{cat}}$ is called index-based weighted point distance $\delta_{\texttt{index}}$, and it assigns higher scores for labels that differ more from the more recent elements of $\overrightarrow{V}_{\texttt{cat}}$:

$$\arg \max_{\nabla \in \texttt{SPACE}} \delta_{\texttt{index}}(\overrightarrow{V}_{\texttt{cat}}, L(m_t(\nabla + \overrightarrow{V}_{\texttt{base}}))),$$

$$\delta_{\texttt{index}}(\overrightarrow{V}_{\texttt{cat}}, l_{\overrightarrow{V}_{\texttt{base}}}) = \sum_{i \in [0, |\overrightarrow{V}_{\texttt{cat}}| - 1]} \left| l_{\overrightarrow{V}_{\texttt{base}}} - \overrightarrow{V}_{\texttt{cat}} \right| (i + 1).$$

Adjusted vector $\nabla + \overrightarrow{V}_{\texttt{base}}$ is then appended to the output queue.

The above formula does imply a brute-force search over `SPACE`, the set of candidates for the delta vector to $\overrightarrow{V}_{\texttt{base}}$. This set of candidates is calculated by P_0 through iterations of permutative addition.

The **shadow** generator was given its name because it "shadows" over a csv (comma-separated values) file of numbers. This file can be of any length, so long as the server device can process it. Its main instantiation set of parameters consists of the following.

- Parameter generator, P_0.
- File path to comma-separate vector, $f_{\texttt{path}}$.
- Fitting structure F, one of operation triangle ("optri"), factor vector ("fvec"), ternary vector ("tvec"), and modular decomposed representative ("mdr").

Generator **shadow** cyclically iterates through file $f_{\texttt{path}}$ for vectors of contiguous real numbers. Similar to **gg**, **shadow** loads up another base vector, $\overrightarrow{V}_{\texttt{base}}$, as a pre-image to add to its output queue. Generator also keeps a cache $C_{\texttt{vec}}$ of previous vectors containing the values placed into the output queue. P_0 calculates a length l with the default **shadow** vector length range,

$$l := c_{\texttt{shadow}}[0] + \left(P_0() \mod (c_{\texttt{shadow}}[1] - c_{\texttt{shadow}}[0])\right).$$

Then **shadow** retrieves the next vector, $\overrightarrow{V}$, from $f_{\texttt{path}}$:

$$\overrightarrow{V}_{\texttt{base}} := \texttt{cycle}(f_{\texttt{path}}, i, l);$$

```
i the current index in the csv file.
```

A set of k vectors $C'_{\texttt{vec}} \subseteq C_{\texttt{vec}}$ is added to $\overrightarrow{V}_{\texttt{base}}$:

$$\overrightarrow{V}_{\texttt{base}} := \overrightarrow{V}_{\texttt{base}} + \sum_{v \in C'_{\texttt{vec}}} (v).$$

This addition operation constitutes the "first shadow" phase of the generator's modification scheme. The second phase involves distorting this shadow $\overrightarrow{V}_{\texttt{base}}$ of the original vector from $f_{\texttt{path}}$. Distortion follows by F, the fitting structure.

- "optri": Algorithm chooses two pairwise functions, $f_{\rightarrow}$ (forward function) and $f_{\leftarrow}$ (backward function). On $\overrightarrow{V}_{\texttt{base}}$, it produces one upper-right triangle with $f_{\rightarrow}$ and another M_2 with $f_{\leftarrow}$. It flips M_2 on the 0-axis and adds it to M,

  $$M := M + \texttt{flip}(M_2, 0),$$

  to produce a matrix without any default zeros. Algorithm flattens M into a sequence $\overrightarrow{S}$ and uses P_0 to permute it,

  $$\overrightarrow{S} := \texttt{permute}(\overrightarrow{S}, P_0).$$

  Sequence $\overrightarrow{S}$ is appended to the output queue.

- "mdr": Algorithm finds a modular decomposed representative, in the form of $P_{(m,a)}$ and $P_{\texttt{mod}}$, for $\overrightarrow{V}_{\texttt{base}}$. These structures have been described in Section 7.2.1. Depending on if P_0 outputs a decimal greater than 0.5 in ranged modulus of the unit interval, algorithm proceeds to modify the modular decomposed representative by shifting the indices of $P_{(m,a)}$ by a length s, such that the i'th index of $P_{(m,a)}$ is now at the $((i+s) \mod |P_{(m,a)}|)$'th index, or by adding real numbers from P_0 to those of $P_{(m,a)}$ and then applying a ranged modulus to it. The modular decomposed representative reconstructs a sequence $\overrightarrow{S}$ using the different $P_{(m,a)}$, and adds $\overrightarrow{S}$ to the output queue.

- "tvec": Algorithm calculates the ternary vector for $\overrightarrow{V}_{\texttt{base}}$,

  $$\overrightarrow{V}_t := T(\overrightarrow{V}_{\texttt{base}}).$$

  Algorithm adds noise to $\overrightarrow{V}_t$ to produce another ternary vector of maximum difference to $\overrightarrow{V}_t$:

  $$\overrightarrow{V}_t := \overrightarrow{V}_t + \arg\max_{\nabla \in \texttt{SPACE}} \sum (|\overrightarrow{V}_t - (\overrightarrow{V}_t + \nabla)|).$$

  Finally, algorithm adds outputs from P_0 onto $\overrightarrow{V}_{\texttt{base}}$ such that

  $$T(F(P_0, \overrightarrow{V}_{\texttt{base}})) \equiv \overrightarrow{V}_t.$$

  This revised $\overrightarrow{V}_t$ is added to the output queue.

- "fvec": For every i'th element of $\overrightarrow{V}_{\texttt{base}}$, algorithm replaces the value at that index by first calculating all factors of $\overrightarrow{V}_{\texttt{base}}[i]$, W. For every element of W, P_0 adds its output values to it, and W' is produced. The new element is

$$\overrightarrow{V}_{\texttt{base}}[i] := \prod_{w \in W} w.$$

  Note that modulo operations are used to prevent the exploding value problem from occurring. The revised $\overrightarrow{V}_{\texttt{base}}$ is appended to the output queue.

Despite the attempts by **shadow** and **gg** to break away from their number-generating patterns, there is no guarantee in their programming to actually do so. This is due, in predominant part, to their parameter generators, P_0. The auto-correcting algorithm of **gg**, for example, performs a brute-force search over a space of candidate derivatives, `SPACE`. This space of candidates is generated by P_0. If P_0 is not able to generate ternary vectors or other vector types, depending on the auto-correcting function, of a uniform distribution, then `SPACE` is fairly predictable. This predictability can be expressed by the general form of a ring, with set R:

$$R_{\texttt{gg}} := \{\nabla_{m_t}(\overrightarrow{V}^{(t)}_{\texttt{base}}); \nabla_{m_t} \in \texttt{SPACE}(P_0)\},$$

$$R_{\texttt{shadow}} := \{\nabla_F(\overrightarrow{V}^{(t)}_{\texttt{base}} + (\overrightarrow{V}^{(t-i_1)}_{\texttt{base}} + \cdots + \overrightarrow{V}^{(t-i_k)}_{\texttt{base}})); \nabla_F \in \texttt{SPACE}(P_0)\}.$$

`SPACE` denotes the parameter generator's output values in arbitrary usage.

Smaller rings imply less effective qualitative difference from these auto-correction processes. In the broad sense, these generators' self-recognition of their output qualities does not necessarily guarantee that they can produce the wanted maximal change from their pre-image values. Their parameter generators may inadvertently fall into a pattern of a relatively small, and thus predictable, ring-like set. The "fvec" mode of **shadow** generator provides another specific set of cases where the pattern (factor set) is known to the algorithm, yet the ∇ applied to that factor set produces a cumulative product, the revised number, that shares factors with the original number.

7.3.2 Cycle Test on Gg Generator of Selected LCGs

Cycle Test of P_0 on **gg**			
#	Parameters	$m_t = \mathbb{C}$	$m_t = \mathbb{U}_{pd}$

1	$P_0 = \texttt{lcg}(87, -31, 993, 5000), \texttt{pre-}n = 1, \texttt{post-}n = 4$	seq: $\texttt{mcs} \to 0.00243$, $\mathcal{D}_{\texttt{cycle}} \to 2.44444$. set: $\texttt{mcs} \to 0.00243$, $\mathcal{D}_{\texttt{cycle}} \to 2.44444$. valfreq: $\texttt{mcs} \to 0.00094$, $\mathcal{D}_{\texttt{cycle}} \to 0.5$. l-seq: $\texttt{mcs} \to 0.00243$, $\mathcal{D}_{\texttt{cycle}} \to 2.4444$. $\nabla_{index} \to \infty$, $\texttt{var}_{\texttt{norm}} \to 0.0$.	seq: $\texttt{mcs} \to 0.00254$, $\mathcal{D}_{\texttt{cycle}} \to 2.55574$. set: $\texttt{mcs} \to 0.00254$, $\mathcal{D}_{\texttt{cycle}} \to 2.5566$. valfreq: $\texttt{mcs} \to 0.00095$, $\mathcal{D}_{\texttt{cycle}} \to 0.51552$. l-seq: $\texttt{mcs} \to 0.00254$, $\mathcal{D}_{\texttt{cycle}} \to 2.5566$. $\nabla_{index} \to \infty$, $\texttt{var}_{\texttt{norm}} \to 0.0$.
2	$P_0 = \texttt{lcg}(87, -31, 993, 5000), \texttt{pre-}n = 1, \texttt{post-}n = 9$	seq: $\texttt{mcs} \to 0.00705$, $\mathcal{D}_{\texttt{cycle}} \to 7.45455$. set: $\texttt{mcs} \to 0.00705$, $\mathcal{D}_{\texttt{cycle}} \to 7.45455$. valfreq: $\texttt{mcs} \to 0.00094$, $\mathcal{D}_{\texttt{cycle}} \to 0.5$. l-seq: $\texttt{mcs} \to 0.00705$, $\mathcal{D}_{\texttt{cycle}} \to 7.45455$. $\nabla_{index} \to \infty$, $\texttt{var}_{\texttt{norm}} \to 0.0$.	seq: $\texttt{mcs} \to 0.00731$, $\mathcal{D}_{\texttt{cycle}} \to 7.71676$. set: $\texttt{mcs} \to 0.00732$, $\mathcal{D}_{\texttt{cycle}} \to 7.71596$. valfreq: $\texttt{mcs} \to 0.00095$, $\mathcal{D}_{\texttt{cycle}} \to 0.51563$. l-seq: $\texttt{mcs} \to 0.00732$, $\mathcal{D}_{\texttt{cycle}} \to 7.71596$. $\nabla_{index} \to \infty$, $\texttt{var}_{\texttt{norm}} \to 0.0$.
3	$P_1 = \texttt{lcg}(18, -131, 93, 5000), \texttt{pre-}n = 1, \texttt{post-}n = 4$	seq: $\texttt{mcs} \to 0.00243$, $\mathcal{D}_{\texttt{cycle}} \to 2.44444$. set: $\texttt{mcs} \to 0.00243$, $\mathcal{D}_{\texttt{cycle}} \to 2.44444$. valfreq: $\texttt{mcs} \to 0.00094$, $\mathcal{D}_{\texttt{cycle}} \to 0.5$. l-seq: $\texttt{mcs} \to 0.00243$, $\mathcal{D}_{\texttt{cycle}} \to 2.44444$. $\nabla_{index} \to \infty$, $\texttt{var}_{\texttt{norm}} \to 0.0$.	seq: $\texttt{mcs} \to 0.00261$, $\mathcal{D}_{\texttt{cycle}} \to 2.63383$. set: $\texttt{mcs} \to 0.00261$, $\mathcal{D}_{\texttt{cycle}} \to 2.63471$. valfreq: $\texttt{mcs} \to 0.00095$, $\mathcal{D}_{\texttt{cycle}} \to 0.52437$. l-seq: $\texttt{mcs} \to 0.00261$, $\mathcal{D}_{\texttt{cycle}} \to 2.63471$. $\nabla_{index} \to \infty$, $\texttt{var}_{\texttt{norm}} \to 0.0$.

4	$P_1 = \texttt{lcg}(18, -131, 93, 5000), \texttt{pre-}n = 1, \texttt{post-}n = 9$	seq: $\texttt{mcs} \rightarrow 0.00705$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 7.45455$. set: $\texttt{mcs} \rightarrow 0.00705$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 7.45455$. valfreq: $\texttt{mcs} \rightarrow 0.00094$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.5$. l-seq: $\texttt{mcs} \rightarrow 0.007$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 7.45455$. $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.0$.	seq: $\texttt{mcs} \rightarrow 0.00745$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 7.88737$. set: $\texttt{mcs} \rightarrow 0.00745$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 7.88737$. valfreq: $\texttt{mcs} \rightarrow 0.00095$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.52454$. l-seq: $\texttt{mcs} \rightarrow 0.00745$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 7.88737$. $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.0$.
5	$P_3 = \texttt{lcg}(87, -31, 993, 5000) - \texttt{lcg}(18, -131, 93, 5000), \texttt{pre-}n = 1, \texttt{post-}n = 4$	seq: $\texttt{mcs} \rightarrow 0.00243$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 2.44444$. set: $\texttt{mcs} \rightarrow 0.00243$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 2.44444$. valfreq: $\texttt{mcs} \rightarrow 0.00094$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.5$. l-seq: $\texttt{mcs} \rightarrow 0.00243$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 2.44444$. $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.0$.	seq: $\texttt{mcs} \rightarrow 0.00337$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 3.53382$. set: $\texttt{mcs} \rightarrow 0.00337$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 3.53162$. valfreq: $\texttt{mcs} \rightarrow 0.00106$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.64439$. l-seq: $\texttt{mcs} \rightarrow 0.00337$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 3.53162$. $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.01678$.
6	$P_3 = \texttt{lcg}(87, -31, 993, 5000) - \texttt{lcg}(18, -131, 93, 5000), \texttt{pre-}n = 1, \texttt{post-}n = 9$	seq: $\texttt{mcs} \rightarrow 0.00705$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 7.45455$. set: $\texttt{mcs} \rightarrow 0.00705$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 7.45455$. valfreq: $\texttt{mcs} \rightarrow 0.00094$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.5$. l-seq: $\texttt{mcs} \rightarrow 0.00705$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 7.45455$. $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.0$.	seq: $\texttt{mcs} \rightarrow 0.00932$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 10.03823$. set: $\texttt{mcs} \rightarrow 0.00934$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 10.03603$. valfreq: $\texttt{mcs} \rightarrow 0.00106$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 0.64274$. l-seq: $\texttt{mcs} \rightarrow 0.00934$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 10.03603$. $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.01595$.

7	$P_2 = \texttt{lcg}(3, 5, 1993, 5000), \texttt{pre-}n = 1, \texttt{post-}n = 4$	seq: $\texttt{mcs} \rightarrow 0.07545$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 86.82444$. set: $\texttt{mcs} \rightarrow 0.07765$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 86.58111$. valfreq: $\texttt{mcs} \rightarrow 0.00167$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 1.36222$. l-seq: $\texttt{mcs} \rightarrow 0.07765$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 86.58111$. $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.24$.	seq: $\texttt{mcs} \rightarrow 0.06053$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 66.38889$. set: $\texttt{mcs} \rightarrow 0.06053$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 67.22661$. valfreq: $\texttt{mcs} \rightarrow 0.00499$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 4.63104$. l-seq: $\texttt{mcs} \rightarrow 0.06053$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 67.22661$. $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.31847$.
8	$P_2 = \texttt{lcg}(3, 5, 1993, 5000), \texttt{pre-}n = 1, \texttt{post-}n = 9$	seq: $\texttt{mcs} \rightarrow 0.24886$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 225.88$. set: $\texttt{mcs} \rightarrow 0.21918$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 225.16527$. valfreq: $\texttt{mcs} \rightarrow 0.00257$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 2.38889$. l-seq: $\texttt{mcs} \rightarrow 0.21918$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 225.16527$. $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.47333$.	seq: $\texttt{mcs} \rightarrow 0.15225$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 160.92105$. set: $\texttt{mcs} \rightarrow 0.14128$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 152.22975$. valfreq: $\texttt{mcs} \rightarrow 0.00772$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 6.91667$. l-seq: $\texttt{mcs} \rightarrow 0.14128$, $\mathcal{D}_{\texttt{cycle}} \rightarrow 152.22975$. $\nabla_{index} \rightarrow \infty$, $\texttt{var}_{\texttt{norm}} \rightarrow 0.35205$.

Table 8: Display of Cycle Test results for both metrics $\mathbb{C}$ (coverage) and $\mathbb{U}_{pd}$ (unidirectional weighted point distance). There are four parameter generators observed. The first two P_0 and P_1 are linear congruential generators that each output 1000 unique values. The P_3 generator is $P_0 - P_1$, and outputs 747 unique values. Generator P_2 is a linear congruential generator that outputs a significantly less number of unique values, 11. The Cycle Test is able to detect output patterns from the *gg* that uses P_2 as a parameter generator. The measures $\mathcal{D}_{\texttt{cycle}}$ (mean cyclical kernel difference) and $\texttt{var}_{\texttt{norm}}$ (normalized variance) are markedly higher than in the other cases P_0, P_1, and P_3. However, there is not obvious repetition of subsequences, as demonstrated by the null values for $\nabla_{\texttt{index}}$ (set size for difference vector of prefix indices of occurrence), marked as ∞. Note: The **gg** mode "allow sub-range drift" is set to False.

Results from 8 exclusively focus on **gg** in mode "allow sub-range drift := False". When sub-range drift is enabled, **gg** chooses some k sub-ranges over its super-range $r_{\texttt{super}}$, possibly complicating the results.

Number of unique values from **gg** over 10000 iterations		
P	$m_t = \mathbb{C}$	$m_t = \mathbb{U}_{pd}$
P_0	7277	2983
P_1	6338	3462
P_2	49	44
P_3	6577	2629

Table 9: Display of set cardinality for **gg** over 10000 iterations, using parameter generator P from Table 8.

7.3.3 Comparing Randomness Between Different Shadow Generator Modes

This section uses the generators P_0,P_1,P_2, and P_3 from the previous Section 7.3.2 and these files of number sequences for testing the **shadow** generator.

- $\overrightarrow{S}_0 = \langle 6, 0, 0, 0, 0, 6, 0, 0, 0, 0, 6 \rangle$.
- $\overrightarrow{S}_1 = \langle 873, 49, -1447, 875, 6, -117, 92, 30, 55, 551, 73, -432, -12, 666, 11133 \rangle$.
- $\overrightarrow{S}_2 = \langle 76, -554, 321, 78, -42, 76, -554, 321, 78, -42, 76, -554, 321, 78, -42 \rangle$.

In the following matrices, ten consecutive segments are measured by PRNG I/O Test, without any host (output) generator, for each of the five **shadow** generators. The value v at the (row,column) (i, j)-th index, is the cumulative ternary value from the ten segments. If the j'th generator is measured as more random than the i'th generator, v is positive. If it is equally random, then 0. Otherwise, v is negative. The captions of every matrix show the (score,rank) for each of the generators. The score of the k'th generator, $R^{(k)}_{\texttt{score}}$, is

$$R^{(k)}_{\texttt{score}} = \sum M_s.col(k) - \sum M_s.row(k).$$

$[\overrightarrow{S}_0, P_0]$

	fvec	mdr	mdrv2	optri	tvec
fvec	∅	−2	−3	−10	−7
mdr	∅	∅	2	−10	−7
mdrv2	∅	∅	∅	−10	−10
optri	∅	∅	∅	∅	10
tvec	∅	∅	∅	∅	∅

$[\overrightarrow{S}_0, P_1]$

	fvec	mdr	mdrv2	optri	tvec
fvec	∅	−1	2	4	−7
mdr	∅	∅	5	9	−9
mdrv2	∅	∅	∅	4	−8
optri	∅	∅	∅	∅	−2
tvec	∅	∅	∅	∅	∅

Figure 5: Two batches of pairwise comparisons between five **shadow** generators, each with a unique fitting structure. The (score,rank) for the generators of the left-hand side is `fvec` $\rightarrow (22, 0)$, `mdr` $\rightarrow (13, 2)$, `mdrv2` $\rightarrow (19, 1)$, `optri` $\rightarrow (-40, 4)$, `tvec` $\rightarrow (-14, 3)$, and that for the right-hand side is `fvec` $\rightarrow (2, 2)$, `mdr` $\rightarrow (-6, 3)$, `mdrv2` $\rightarrow (11, 1)$, `optri` $\rightarrow (19, 0)$, `tvec` $\rightarrow (-26, 4)$.

$[\overrightarrow{S}_0, P_2]$

	fvec	mdr	mdrv2	optri	tvec
fvec	∅	0	3	3	−2
mdr	∅	∅	2	4	1
mdrv2	∅	∅	∅	5	−4
optri	∅	∅	∅	∅	10
tvec	∅	∅	∅	∅	∅

$[\overrightarrow{S}_0, P_3]$

	fvec	mdr	mdrv2	optri	tvec
fvec	∅	−4	−5	−4	−4
mdr	∅	∅	3	−4	−3
mdrv2	∅	∅	∅	0	−6
optri	∅	∅	∅	∅	−4
tvec	∅	∅	∅	∅	∅

Figure 6: Two batches of pairwise comparisons between five **shadow** generators, each with a unique fitting structure. The (score,rank) for the generators of the left-hand side is `fvec` $\rightarrow (-4, 3)$, `mdr` $\rightarrow (-5, 4)$, `mdrv2` $\rightarrow (4, 0)$, `optri` $\rightarrow (2, 2)$, `tvec` $\rightarrow (3, 1)$, and that for the right-hand side is `fvec` $\rightarrow (17, 0)$, `mdr` $\rightarrow (0, 2)$, `mdrv2` $\rightarrow (4, 1)$, `optri` $\rightarrow (-4, 3)$, `tvec` $\rightarrow (-17, 4)$.

$[\overrightarrow{S}_1, P_0]$

	fvec	mdr	mdrv2	optri	tvec
fvec	∅	4	5	−10	−4
mdr	∅	∅	4	−10	−9
mdrv2	∅	∅	∅	−10	−9
optri	∅	∅	∅	∅	10
tvec	∅	∅	∅	∅	∅

$[\overrightarrow{S}_1, P_1]$

	fvec	mdr	mdrv2	optri	tvec
fvec	∅	−6	−5	8	−6
mdr	∅	∅	5	6	−4
mdrv2	∅	∅	∅	8	−7
optri	∅	∅	∅	∅	1
tvec	∅	∅	∅	∅	∅

Figure 7: Two batches of pairwise comparisons between five **shadow** generators, each with a unique fitting structure. The (score,rank) for the generators of the left-hand side is `fvec` $\rightarrow (5, 2)$, `mdr` $\rightarrow (19, 1)$, `mdrv2` $\rightarrow (28, 0)$, `optri` $\rightarrow (-40, 4)$, `tvec` $\rightarrow (-12, 3)$, and that for the right-hand side is `fvec` $\rightarrow (9, 1)$, `mdr` $\rightarrow (-13, 3)$, `mdrv2` $\rightarrow (-1, 2)$, `optri` $\rightarrow (21, 0)$, `tvec` $\rightarrow (-16, 4)$.

$[\overrightarrow{S}_1, P_2]$

	fvec	mdr	mdrv2	optri	tvec
fvec	∅	−2	−2	1	−3
mdr	∅	∅	1	2	−4
mdrv2	∅	∅	∅	4	−2
optri	∅	∅	∅	∅	6
tvec	∅	∅	∅	∅	∅

$[\overrightarrow{S}_1, P_3]$

	fvec	mdr	mdrv2	optri	tvec
fvec	∅	−2	−2	−8	−9
mdr	∅	∅	2	−10	−7
mdrv2	∅	∅	∅	−7	−6
optri	∅	∅	∅	∅	−8
tvec	∅	∅	∅	∅	∅

Figure 8: Two batches of pairwise comparisons between five **shadow** generators, each with a unique fitting structure. The (score,rank) for the generators of the left-hand side is `fvec` $\rightarrow (6,0)$, `mdr` $\rightarrow (-1,2)$, `mdrv2` $\rightarrow (-3,3)$, `optri` $\rightarrow (1,1)$, `tvec` $\rightarrow (-3,4)$, and that for the right-hand side is `fvec` $\rightarrow (21,0)$, `mdr` $\rightarrow (13,1)$, `mdrv2` $\rightarrow (13,1)$, `optri` $\rightarrow (-17,2)$, `tvec` $\rightarrow (-30,3)$.

$[\overrightarrow{S}_2, P_0]$

	fvec	mdr	mdrv2	optri	tvec
fvec	∅	−1	−2	−10	−4
mdr	∅	∅	−3	−10	−6
mdrv2	∅	∅	∅	−10	−4
optri	∅	∅	∅	∅	10
tvec	∅	∅	∅	∅	∅

$[\overrightarrow{S}_2, P_1]$

	fvec	mdr	mdrv2	optri	tvec
fvec	∅	−3	0	4	−5
mdr	∅	∅	0	6	−7
mdrv2	∅	∅	∅	8	−6
optri	∅	∅	∅	∅	−5
tvec	∅	∅	∅	∅	∅

Figure 9: Two batches of pairwise comparisons between five **shadow** generators, each with a unique fitting structure. The (score,rank) for the generators of the left-hand side is `fvec` $\rightarrow (17,1)$, `mdr` $\rightarrow (18,0)$, `mdrv2` $\rightarrow (9,2)$, `optri` $\rightarrow (-40,4)$, `tvec` $\rightarrow (-4,3)$, and that for the right-hand side is `fvec` $\rightarrow (4,1)$, `mdr` $\rightarrow (-2,2)$, `mdrv2` $\rightarrow (-2,3)$, `optri` $\rightarrow (23,0)$, `tvec` $\rightarrow (-23,4)$.

$[\overrightarrow{S}_2, P_2]$

	fvec	mdr	mdrv2	optri	tvec
fvec	∅	2	−5	8	−6
mdr	∅	∅	−2	5	−2
mdrv2	∅	∅	∅	8	−6
optri	∅	∅	∅	∅	7
tvec	∅	∅	∅	∅	∅

$[\overrightarrow{S}_2, P_3]$

	fvec	mdr	mdrv2	optri	tvec
fvec	∅	−4	−1	−1	−6
mdr	∅	∅	1	−1	−7
mdrv2	∅	∅	∅	−4	−5
optri	∅	∅	∅	∅	−6
tvec	∅	∅	∅	∅	∅

Figure 10: Two batches of pairwise comparisons between five **shadow** generators, each with a unique fitting structure. The (score,rank) for the generators of the left-hand side is `fvec` $\rightarrow (1,1)$, `mdr` $\rightarrow (1,1)$, `mdrv2` $\rightarrow (-9,3)$, `optri` $\rightarrow (14,0)$, `tvec` $\rightarrow (-7,2)$, and that for the right-hand side is `fvec` $\rightarrow (12,0)$, `mdr` $\rightarrow (3,2)$, `mdrv2` $\rightarrow (9,1)$, `optri` $\rightarrow (0,3)$, `tvec` $\rightarrow (-24,4)$.

Across these cases, results show **shadow** generators of "fvec" mode the highest scoring and "tvec" the lowest scoring.

7.4 Auto-Generation of PRNGs

Seqbuild contains code that allows auto-generation of PRNG instances. The set of instantiation parameters for some of the PRNG primitive structures are relatively large in size. The **idforest** and **ssino** generators require graphs at instantiation, and **rch** requires a mutable unidirectional chain of polynomial or linear expression node functions. Class D generators, **lps** and **pid**, require sequences of two-dimensional points that additionally have to be unique and within ranges not going into the millions. Auto-generation code was implemented to facilitate convenient usage. There are two categories of auto-generation: T-random (time-based) and P-random (PRNG provided by external source). In both categories, algorithms use one PRNG along with default constraints, pre-assigned to every *Seqbuild* primitive structure, to generate the respective parameter values for that structure. The PRNG used to generate that structure would, in applicable cases, also serve as a parameter generator for it.

The T-random generation scheme uses **pdd**, the acronym standing for "PRNG Decimal Delta". T-random generates without any external parameter values given, so the algorithm fetches the Python programming language's time value v. Then **pdd** applies its algorithm to map v into real numbers. The time value from Python is longer than the conventional "hour:minute:second" notation, so the technique proves feasible for generating numbers along a uniform basis and of a "random" appearance, although it is not entirely bug-free due to incidental patterns found in certain Python time values. The P-random generation scheme requires a PRNG, either one explicitly passed by a human user into the generative function, or another computer program (not developed). In the P-random generation scheme, there are two primary patterns by which a sequence of generators can be generated.

Definition 7.2 (Chain-Generated Pseudo-Random Number Generators)**.** A sequence of pseudo-random number generators $\overrightarrow{G}$ is *chain-generated* if for every i'th generator of $\overrightarrow{G}$, $0 < i < |\overrightarrow{G}|$, $\overrightarrow{G}[i-1]$ was used by an arbitrary algorithm A to generate it.

Definition 7.3 (Single-Source Generated Sibling Pseudo-Random Number Generators)**.** In a sequence of pseudo-random number generators $\overrightarrow{G}$, any pair of generators $G_i, G_j \in \overrightarrow{G}$ are siblings if the contiguous outputs from a generator

$G_{\texttt{source}}$, starting at its n'th output index, were used to generate

$$\overrightarrow{G}[0], \overrightarrow{G}[1], \ldots, \overrightarrow{G}[|\overrightarrow{G}| - 1]$$

in consecutive order by index.

7.5 Discrepancies in Comparative Randomness

The area of randomness testing is very closely related to metric geometry, at least in the objective sense of measurability. PRNG periodicity is one important quality that may skew the results of randomness comparison between two or more PRNGs. Linear congruential generators, for example, that exceed a periodicity higher than constraints set by their test/s can score higher than generators of a lesser known convention.

The PRNG I/O Test is used in this section to show how the Qual and Chain Tests are not foolproof in measuring comparative randomness. Per the descriptions of these two tests provided in Section 5, there are indeed performance issues in their calculations. The Qual Test is a highly intensive procedure. On a personal computer, this test shows a degradation in timeliness. At $|\overrightarrow{S}| = 1000$, the Qual Test usually takes more than five minutes to finish. The unidirectional weighted point distance calculation takes $O(n^2)$ time. In order to calculate the modular characteristic of $\overrightarrow{S}$, all factors of elements of $\overrightarrow{S}$, except for 1, have to be calculated. The Chain Test requires a sample size n and a partition size p. So it is constrained to observing a PRNG in the scope of n samples.

Given k PRNGs, each used as a parameter generator for one output PRNG, there are $2k$ PRNGs in consideration. This implies a cost of computation multiples higher than the Qual and Chain Tests that PRNG I/O Test operates. The PRNG I/O Test has default constraints in sample size, so that runtime is reduced to a manageable upper limit for the user. Segment length is 200 for Qual Test and 1000 for Chain Test. The program for this test can run as many segments for Qual and Chain Tests as specified. For every additional segment, program either draws leftover PRNG output values from the corresponding queue, or have the subject PRNG output more values. Consider a case of one of the **gg** generators from Table 8. PRNG I/O Test results showcase the difficulty of condensing mathematical metrics into notions as potentially complicating and relative, considering the observer, into simple answers of "greater than", "less than", and "equal to". While the intensity of calculations is able to capture critical details of PRNG number sequences, Qual and Chain Test require more than what their metrics offer to effectively handle more difficult tasks, such as efficient replication that does not need to store the entire subject number sequence into memory, pattern recognition based on hidden states, and accurate prediction of next output/s.

PRNG I/O Test of P_0 on **gg**			
segment #	gg/mode:=uwpd	`lcg`$(18, -131, 93, 5000)$	scores

0	$\bullet\mathbb{E} = 0.28479, \mathbb{C} = 0.77847, \# = 200, \texttt{mean}(C_{\texttt{amod}} = 55.39698, \Bbbk_f = 279.78, \texttt{var}(C_{\texttt{amod}}) = , 190448.04346\texttt{mcs} = 0.705, \Bbbk_1 = 128, \Bbbk_2 = 151, \mathbb{U}_{pd} = 0.31446, \texttt{mean}_{\texttt{max-min}} = 3266.475, \texttt{var}_{\texttt{(max - min)/2}} = 94933.24344$	$\bullet\mathbb{E} = 0.31125, \mathbb{C} = 0.81835, \# = 200, \texttt{mean}(C_{\texttt{amod}} = 28.57286, \Bbbk_f = 279.925, \texttt{var}(C_{\texttt{amod}}) = 12734.43383, \texttt{mcs} = 0.705, \Bbbk_1 = 138, \Bbbk_2 = 147, \mathbb{U}_{pd} = 0.33747, \texttt{mean}_{\texttt{max-min}} = 3365.795, \texttt{var}_{\texttt{(max - min)/2}} = 24683.66766$	$M_1 = -1, M_2 = 1, M_3 = 1, M_4 = 1, M_5 = -1, M_6 = -1, M_7 = -1, R = -1$
5	$\bullet\mathbb{E} = 0.31514, \mathbb{C} = 0.80905, \# = 199, \texttt{mean}(C_{\texttt{amod}} = 26.1407, \Bbbk_f = 277.89, \texttt{var}(C_{\texttt{amod}}) = 200424.09752, \texttt{mcs} = 0.705, \Bbbk_1 = 134, \Bbbk_2 = 143, \mathbb{U}_{pd} = 0.35567, \texttt{mean}_{\texttt{max-min}} = 3304.365, \texttt{var}_{\texttt{(max - min)/2}} = 82770.58141$	$\bullet\mathbb{E} = 0.31125, \mathbb{C} = 0.81835, \# = 200, \texttt{mean}(C_{\texttt{amod}} = 28.57286, \Bbbk_f = 279.925, \texttt{var}(C_{\texttt{amod}}) = 12734.43383, \texttt{mcs} = 0.705, \Bbbk_1 = 138, \Bbbk_2 = 147, \mathbb{U}_{pd} = 0.33747, \texttt{mean}_{\texttt{max-min}} = 3365.795, \texttt{var}_{\texttt{(max - min)/2}} = 24683.66766$	$M_1 = -1, M_2 = 0, M_3 = -1, M_4 = -1, M_5 = -1, M_6 = -1, M_7 = -1, R = -1$
12	$\bullet\mathbb{E} = 0.30943, \mathbb{C} = 0.82626, \# = 188, \texttt{mean}(C_{\texttt{amod}} = 7.26633, \Bbbk_f = 278.24, \texttt{var}(C_{\texttt{amod}}) = 549338.77304, \texttt{mcs} = 0.705, \Bbbk_1 = 136, \Bbbk_2 = 145, \mathbb{U}_{pd} = 0.34525, \texttt{mean}_{\texttt{max-min}} = 3347.955, \texttt{var}_{\texttt{(max - min)/2}} = 29529.94401$	$\bullet\mathbb{E} = 0.28833, \mathbb{C} = 0.80977, \# = 200, \texttt{mean}(C_{\texttt{amod}} = 28.57286, \Bbbk_f = 280.755, \texttt{var}(C_{\texttt{amod}}) = 12734.43383, \texttt{mcs} = 0.705, \Bbbk_1 = 128, \Bbbk_2 = 137, \mathbb{U}_{pd} = 0.36244, \texttt{mean}_{\texttt{max-min}} = 3365.795, \texttt{var}_{\texttt{(max - min)/2}} = 24683.66766$	$M_1 = 1, M_2 = 0, M_3 = 1, M_4 = 0, M_5 = 1, M_6 = 1, M_7 = 1, R = 1$
19	$\bullet\mathbb{E} = 0.30707, \mathbb{C} = 0.78645, \# = 198, \texttt{mean}(C_{\texttt{amod}} = 15.55779, \Bbbk_f = 279.18, \texttt{var}(C_{\texttt{amod}}) = 95497.17916, \texttt{mcs} = 0.705, \Bbbk_1 = 144, \Bbbk_2 = 150, \mathbb{U}_{pd} = 0.32316, \texttt{mean}_{\texttt{max-min}} = 3427.795, \texttt{var}_{\texttt{(max - min)/2}} = 30634.31604$	$\bullet\mathbb{E} = 0.29394, \mathbb{C} = 0.78277, \# = 200, \texttt{mean}(C_{\texttt{amod}} = 28.57286, \Bbbk_f = 277.805, \texttt{var}(C_{\texttt{amod}}) = 12734.43383, \texttt{mcs} = 0.705, \Bbbk_1 = 133, \Bbbk_2 = 143, \mathbb{U}_{pd} = 0.31936, \texttt{mean}_{\texttt{max-min}} = 3365.795, \texttt{var}_{\texttt{(max - min)/2}} = 24683.66766$	$M_1 = 1, M_2 = 1, M_3 = 1, M_4 = 1, M_5 = 1, M_6 = 1, M_7 = 1, R = 1$

Table 10: Twenty contiguous segments of Qual and Chain Test, under a PRNG I/O Test, for a **gg**. Ranged modulus of [0., 5000.] was applied to the generators before measurement. This test considers all seven metrics; refer to Table 2 for the specific variables those metrics observe. Four of those segments are displayed. The number 1 denotes **gg** being more random, 0 as equal, and −1 as less random. The entire twenty ternary results of randomness are the following: $0:-1, 1:-1, 2:-1, 3:1, 4:1, 5:-1, 6:-1, 7:1, 8:0, 9:-1, 10:-1, 11:-1, 12:1, 13:-1, 14:-1, 15:-1, 16:-1, 17:1, 18:-1, 19:1$.There are 13 segments that show **gg** being less random. By these test results, **gg** is less in randomness than `lcg`(18,-131,93,5000) despite it being a parameter generator to **gg**.

Recall that the unique output (image) size of `lcg`(18,-131,93,5000) is 1000. The image size of a linear congruential generator is its period, as well. Default constraints for PRNG I/O Test were a contributing factor in why this linear congruential generator measured as more random than its host generator, an auto-corrector of its output values. However, when the intra-variance of the two generators' quality scores are observed between the twenty segments, the **gg** generator has greater variance, as shown:

$$\mathbb{E} \to 1, \mathbb{C} \to 1, \# \to 1, \texttt{mean}(C_{\texttt{amod}} \to 1, \Bbbk_f \to -1, \texttt{var}(C_{\texttt{amod}}) \to 1, \texttt{mcs} \to 0,$$

$$\Bbbk_1 \to 1, \Bbbk_2 \to 1, \mathbb{U}_{pd} \to -1, \texttt{mean}_{\texttt{max-min}} \to 1, \texttt{var}_{\texttt{(max - min)/2}} \to 1.$$

In the greater scope of matters concerning the randomness from number generation, the assumption of there being a hierarchy of randomness for embeddings of pseudo-random number generators into one another, so that every subsequent embedding of a generator produces another generator with a greater randomness cannot be satisfied on a consistent basis. There are several key issues with this assumption. One of the principle ones is algorithm implementation. It is quite easy to design a number generator that outputs a recognizable pattern, given any parameter generator that outputs contrary to any obvious pattern, for example. Another is the procedure of measuring the output, a contiguous number sequence from the PRNG. Section 8 (Considering the Cryptanalyst) provides mathematical proof against this assumption. The next matrices in this section attempt to disprove the assumption of there being a hierarchy of randomness in PRNG embeddings. There are three PRNGs used to instantiate these cases of chain-generated PRNGs.

$P_0 = \texttt{lcg}(5462.41, -35.254, 2542.325, 23523.11); \left|P_0\right| = ?, \left|\left\lfloor P_0 \right\rfloor\right| = 23524.$

$P_1 = \texttt{lcg}(-56, 32, 41, -76); |P_1| = 19.$

$P_2 = \texttt{lcg}(1, 2, 3, 50) + \texttt{cycle}(80, 160, 240, 320, 400, 480, 560, 640, 720); |P_2| = 180.$

The output range for the PRNG I/O Test, applied as a ranged modulus, is $[-50000, 50000]$. The n-gram is set to $n = 8$, and the chain length to 5. In the following matrices, the value v at the (row,column) (i, j)-th index, is the cumulative ternary value from five consecutive segments from a PRNG I/O Stats comparison. If the j'th generator is measured as more random than the i'th generator, v is positive. If it is equally random, then 0. Otherwise, v is negative.

$$
\begin{array}{c} \\ P_0 \\ \texttt{lps} \\ \texttt{mdr} \\ \texttt{opt} \\ \texttt{rch} \\ \texttt{gg} \end{array}
\begin{array}{c}
\begin{array}{cccccc} P_0 & \texttt{lps} & \texttt{mdr} & \texttt{opt} & \texttt{rch} & \texttt{gg} \end{array} \\
\begin{pmatrix} \varnothing & -5 & -2 & -3 & -1 & 3 \\ \varnothing & \varnothing & 3 & 5 & 5 & 5 \\ \varnothing & \varnothing & \varnothing & 3 & 1 & 3 \\ \varnothing & \varnothing & \varnothing & \varnothing & 0 & 5 \\ \varnothing & \varnothing & \varnothing & \varnothing & \varnothing & 3 \\ \varnothing & \varnothing & \varnothing & \varnothing & \varnothing & \varnothing \end{pmatrix}
\end{array}
\qquad
\begin{array}{c} \\ P_1 \\ \texttt{lps} \\ \texttt{mdr} \\ \texttt{opt} \\ \texttt{rch} \\ \texttt{gg} \end{array}
\begin{array}{c}
\begin{array}{cccccc} P_1 & \texttt{lps} & \texttt{mdr} & \texttt{opt} & \texttt{rch} & \texttt{gg} \end{array} \\
\begin{pmatrix} \varnothing & 3 & -5 & 4 & -1 & 0 \\ \varnothing & \varnothing & -5 & 5 & -5 & 0 \\ \varnothing & \varnothing & \varnothing & 5 & 5 & 5 \\ \varnothing & \varnothing & \varnothing & \varnothing & -5 & -5 \\ \varnothing & \varnothing & \varnothing & \varnothing & \varnothing & 5 \\ \varnothing & \varnothing & \varnothing & \varnothing & \varnothing & \varnothing \end{pmatrix}
\end{array}
$$

Figure 11: Pairwise comparisons between chain-generated PRNGs by the P-random scheme, starting from parameter generator $P_?$. The primitive structures of the chain are **lps,mdrgen,optri,rch,gg**. Non-null values are summations over five segments of the PRNG I/O Test comparisons (all seven metrics).

$$
\begin{array}{c} \\ P_0 \\ \texttt{csh} \\ \texttt{csh} \\ \texttt{csh} \\ \texttt{csh} \\ \texttt{csh} \end{array}
\begin{array}{c}
\begin{array}{cccccc} P_0 & \texttt{csh} & \texttt{csh} & \texttt{csh} & \texttt{csh} & \texttt{csh} \end{array} \\
\begin{pmatrix} \varnothing & -2 & -1 & 0 & -1 & -5 \\ \varnothing & \varnothing & 3 & -1 & -1 & -2 \\ \varnothing & \varnothing & \varnothing & -3 & -5 & -1 \\ \varnothing & \varnothing & \varnothing & \varnothing & 0 & -1 \\ \varnothing & \varnothing & \varnothing & \varnothing & \varnothing & -1 \\ \varnothing & \varnothing & \varnothing & \varnothing & \varnothing & \varnothing \end{pmatrix}
\end{array}
\qquad
\begin{array}{c} \\ P_1 \\ \texttt{csh} \\ \texttt{csh} \\ \texttt{csh} \\ \texttt{csh} \\ \texttt{csh} \end{array}
\begin{array}{c}
\begin{array}{cccccc} P_1 & \texttt{csh} & \texttt{csh} & \texttt{csh} & \texttt{csh} & \texttt{csh} \end{array} \\
\begin{pmatrix} \varnothing & 3 & 5 & 4 & 4 & 3 \\ \varnothing & \varnothing & 2 & -1 & -2 & -5 \\ \varnothing & \varnothing & \varnothing & -1 & -3 & 0 \\ \varnothing & \varnothing & \varnothing & \varnothing & -2 & -1 \\ \varnothing & \varnothing & \varnothing & \varnothing & \varnothing & -1 \\ \varnothing & \varnothing & \varnothing & \varnothing & \varnothing & \varnothing \end{pmatrix}
\end{array}
$$

Figure 12: Pairwise comparisons between chain-generated PRNGs by the P-random scheme, starting from parameter generator $P_?$. The primitive structures of the chain are **cshift,cshift,cshift,cshift,cshift**. Non-null values are summations over five segments of the PRNG I/O Test comparisons (all seven metrics).

	P_0	afs	qval	afs	qval	afs
P_0	∅	2	−3	−4	−5	−1
afs	∅	∅	−4	−3	1	1
qval	∅	∅	∅	0	−3	3
afs	∅	∅	∅	∅	0	2
qval	∅	∅	∅	∅	∅	1
afs	∅	∅	∅	∅	∅	∅

	P_1	afs	qval	afs	qval	afs
P_1	∅	5	−5	3	5	5
afs	∅	∅	−3	1	0	−1
qval	∅	∅	∅	3	−3	−2
afs	∅	∅	∅	∅	1	−3
qval	∅	∅	∅	∅	∅	−1
afs	∅	∅	∅	∅	∅	∅

Figure 13: Pairwise comparisons between chain-generated PRNGs by the P-random scheme, starting from parameter generator $P_?$. The primitive structures of the chain are **afs,qval,afs,qval,afs**. Non-null values are summations over five segments of the PRNG I/O Test comparisons (all seven metrics).

	P_0	idf	rch	idf	ssi	rch
P_0	∅	3	−5	0	0	−4
idf	∅	∅	−5	−3	−5	−5
rch	∅	∅	∅	5	5	5
idf	∅	∅	∅	∅	1	−3
ssi	∅	∅	∅	∅	∅	0
rch	∅	∅	∅	∅	∅	∅

	P_1	idf	rch	idf	ssi	rch
P_1	∅	3	5	2	5	−5
idf	∅	∅	5	−3	3	−5
rch	∅	∅	∅	−5	−2	−5
idf	∅	∅	∅	∅	5	−5
ssi	∅	∅	∅	∅	∅	−5
rch	∅	∅	∅	∅	∅	∅

Figure 14: Pairwise comparisons between chain-generated PRNGs by the P-random scheme, starting from parameter generator $P_?$. The primitive structures of the chain are **idforest,rch,idforest,ssino,rch**. Non-null values are summations over five segments of the PRNG I/O Test comparisons (all seven metrics).

	P_2	lps	mdr	opt	rch	gg
P_2	∅	5	5	5	5	5
lps	∅	∅	−2	5	3	1
mdr	∅	∅	∅	5	−3	5
opt	∅	∅	∅	∅	−5	−4
rch	∅	∅	∅	∅	∅	−1
gg	∅	∅	∅	∅	∅	∅

	P_2	csh	csh	csh	csh	csh
P_2	∅	5	5	5	3	5
csh	∅	∅	5	2	1	3
csh	∅	∅	∅	−3	−5	0
csh	∅	∅	∅	∅	−1	−1
csh	∅	∅	∅	∅	∅	1
csh	∅	∅	∅	∅	∅	∅

Figure 15: Pairwise comparisons between chain-generated PRNGs by the P-random scheme, starting from parameter generator P_2. The primitive structures of the two chains are listed in the table rows (or columns). Non-null values are summations over five segments of the PRNG I/O Test comparisons (all seven metrics).

$$
\begin{array}{c}
\\
P_2 \\
\texttt{afs} \\
\texttt{qval} \\
\texttt{afs} \\
\texttt{qval} \\
\texttt{afs}
\end{array}
\begin{array}{c}
\begin{array}{cccccc}
P_2 & \texttt{afs} & \texttt{qval} & \texttt{afs} & \texttt{qval} & \texttt{afs}
\end{array} \\
\begin{pmatrix}
\varnothing & 5 & 5 & 5 & -5 & 5 \\
\varnothing & \varnothing & -1 & 0 & -5 & -5 \\
\varnothing & \varnothing & \varnothing & 3 & -5 & -5 \\
\varnothing & \varnothing & \varnothing & \varnothing & -5 & -5 \\
\varnothing & \varnothing & \varnothing & \varnothing & \varnothing & 5 \\
\varnothing & \varnothing & \varnothing & \varnothing & \varnothing & \varnothing
\end{pmatrix}
\end{array}
\qquad
\begin{array}{c}
\\
P_2 \\
\texttt{idf} \\
\texttt{rch} \\
\texttt{idf} \\
\texttt{ssi} \\
\texttt{rch}
\end{array}
\begin{array}{c}
\begin{array}{cccccc}
P_2 & \texttt{idf} & \texttt{rch} & \texttt{idf} & \texttt{ssi} & \texttt{rch}
\end{array} \\
\begin{pmatrix}
\varnothing & 5 & 5 & -5 & 5 & 5 \\
\varnothing & \varnothing & 3 & -5 & 3 & 1 \\
\varnothing & \varnothing & \varnothing & -5 & -5 & -5 \\
\varnothing & \varnothing & \varnothing & \varnothing & 5 & 5 \\
\varnothing & \varnothing & \varnothing & \varnothing & \varnothing & -1 \\
\varnothing & \varnothing & \varnothing & \varnothing & \varnothing & \varnothing
\end{pmatrix}
\end{array}
$$

Figure 16: Pairwise comparisons between chain-generated PRNGs by the P-random scheme, starting from parameter generator P_2. The primitive structures of the chains are **afs,qval,afs,qval,afs** and **idforest,rch,idforest,ssino,rch**, respectively. Non-null values are summations over five segments of the PRNG I/O Test comparisons (all seven metrics).

7.6 Tests on Various Auto-Generated *Seqbuild* PRNGs

These test results are similar to the matrix results in the previous Section 7.5, except the PRNGs in comparison were not chain-generated, but sibling-generated from a source PRNG. The specifics of the algorithm for sibling-generated PRNGs are in the open-source code of *Seqbuild* and its dependency library, *morebs2*. The algorithm for a sibling-generated sequence of LCGs, for example, does not use the direct values for some of the LCG parameters, due to reasons such as ensuring a wide coverage of multiples and additives used and non-zero modulo values.

Information on Source Generators Used in Comparative Randomness		
Source Generator	Sibling-Generated Batch	Batch Identifier
$P_0 = \texttt{lcg}(5462.41, -35.254, 2542.325, 23523.11)$	10 LCGs by *morebs2* algorithm	$\langle G_0 \rangle$
$P_1 = P_0 + \texttt{lcg}(-45.87, 56.66, -7714.1, -10003.11)$	10 LCGs by *morebs2* algorithm	$\langle G_1 \rangle$
$P_2 = P_1/\texttt{lcg}(76, -8992.41, 541.52, 65.0)$	10 LCGs by *morebs2* algorithm	$\langle G_2 \rangle$
$P_3 = \texttt{gen}_p(\texttt{mdrgen}, P_0)$	10 LCGs by *morebs2* algorithm	$\langle G_3 \rangle$
$P_4 = P_2$	10 **optri** generators by *Seqbuild* algorithm	$\langle G_4 \rangle$
$P_5 = P_2$	10 **cshift** generators by *Seqbuild* algorithm	$\langle G_5 \rangle$

Table 11: Details on the source generators used to generate each batch of 10 generators. Every generator in these batches of generators, in turn, is used to generate every primitive structure of interest for comparing randomness.

Map of Symbols for Primitive Structure							
T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
`lcg`	`mdrgen`	`optri`	`qval`	`rch`	`pid`	`ssino`	`idforest`
T_9	T_{10}	T_{11}	T_{12}	T_{13}	T_{14}	T_{15}	T_{16}
`n2m`	`gg`	`afs`	`fit22`	`lps`	`cshift`	`udls`	`pmulti`

Table 12: A total of sixteen primitive structures under consideration.

For these next matrices, each of the ten generators of the pertinent batch is used to generate one generator per primitive structures. The generation of these sixteen generators do not follow the definition of sibling-generated. Instead, the source generator is copied by the algorithm before being used to generate the PRNG. By this approach, the source generator is always at output index 0 every one of the sixteen to generate. Three segments are measured for each generator, and the possible score range of pairwise randomness for any (i, j)'th position is $[-30, 30]$, such that 1 signifies T_j as more random than T_i, -1 as less random, and 0 as equally random, identical to the PRNG I/O matrix scores in the previous section. For these matrices M_s, the rank of any generator T_k is the index of the sorted list $\langle T_{k_1}, \ldots, T_{k_{16}} \rangle$, in descending order by the formula

$$R^{(k)}_{\texttt{score}} = \sum M_s.col(k) - \sum M_s.row(k).$$

Note that the P-random generation scheme for **lcg** generates between two and six linear congruential generators, each the classical version, version two (ranged modulo and cycle detection), and version three (ternary vector fitting). Then the scheme merges this sequence of LCGs, via arbitrarily-selected pairwise operators. Technically, this would not be a linear congruential generator, but an LCG bunch. All seven metrics of PRNG I/O Stats are used to calculate every pairwise comparison of randomness between generators.

	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9	T_{10}	T_{11}	T_{12}	T_{13}	T_{14}	T_{15}	T_{16}
T_1	∅	7	9	−10	−10	−8	5	−4	12	−6	−6	−10	−16	−5	10	−8
T_2	∅	∅	−10	−19	−22	−12	0	−11	0	−17	−24	−16	−17	−10	−4	−13
T_3	∅	∅	∅	−29	−26	−24	−9	−27	5	−15	−26	−28	−28	−12	−2	−23
T_4	∅	∅	∅	∅	−2	5	17	13	27	1	−4	2	−2	1	24	12
T_5	∅	∅	∅	∅	∅	8	17	8	26	5	−3	−1	−1	4	19	7
T_6	∅	∅	∅	∅	∅	∅	18	9	25	−7	−6	7	−3	0	20	6
T_7	∅	∅	∅	∅	∅	∅	∅	−19	11	−18	−22	−22	−22	−17	12	−17
T_8	∅	∅	∅	∅	∅	∅	∅	∅	28	−1	−6	−4	−6	−7	20	1
T_9	∅	∅	∅	∅	∅	∅	∅	∅	∅	−15	−28	−29	−29	−20	−7	−21
T_{10}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−2	5	3	0	18	5
T_{11}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−5	−3	8	24	10
T_{12}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−6	6	23	6
T_{13}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	5	26	6
T_{14}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	14	5
T_{15}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−16
T_{16}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅

Figure 17: Generators of batch $\langle G_0 \rangle$ generated PRNGs of these sixteen types. The scores $R_{\texttt{score}}$ and corresponding ranks for these PRNGs, by index order, are $0 \to (40, 5), 1 \to (182, 3), 2 \to (243, 1), 3 \to (-152, 13), 4 \to (-149, 12), 5 \to (-100, 9), 6 \to (162, 4), 7 \to (-56, 7), 8 \to (283, 0), 9 \to (-102, 10), 10 \to (-161, 14), 11 \to (-130, 11), 12 \to (-167, 15), 13 \to (-66, 8), 14 \to (213, 2), 15 \to (-40, 6)$.

	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9	T_{10}	T_{11}	T_{12}	T_{13}	T_{14}	T_{15}	T_{16}
T_1	∅	11	15	−16	−4	−5	9	−5	20	−1	−12	−10	−8	−15	10	−5
T_2	∅	∅	−4	−13	−6	−12	5	−17	8	−6	−17	−14	−15	−22	−8	−11
T_3	∅	∅	∅	−22	−8	−26	2	−19	6	−11	−20	−24	−19	−18	2	−15
T_4	∅	∅	∅	∅	12	−2	21	5	25	8	6	4	7	4	21	5
T_5	∅	∅	∅	∅	∅	−16	9	−12	15	0	−4	−8	−4	−12	10	−3
T_6	∅	∅	∅	∅	∅	∅	15	18	29	9	9	4	7	−3	25	4
T_7	∅	∅	∅	∅	∅	∅	∅	−20	3	−4	−16	−18	−17	−22	3	−12
T_8	∅	∅	∅	∅	∅	∅	∅	∅	21	5	1	−4	−7	−13	16	−3
T_9	∅	∅	∅	∅	∅	∅	∅	∅	∅	−13	−24	−30	−29	−24	−11	−23
T_{10}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−10	−4	−11	−14	14	−8
T_{11}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−5	−3	−15	18	1
T_{12}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	2	−10	23	8
T_{13}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−12	15	−2
T_{14}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	20	8
T_{15}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−14
T_{16}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅

Figure 18: Generators of batch $\langle G_1 \rangle$ generated PRNGs of these sixteen types. The scores $R_{\texttt{score}}$ and corresponding ranks for these PRNGs, by index order, are $0 \to (16, 7), 1 \to (143, 4), 2 \to (183, 1), 3 \to (-167, 13), 4 \to (19, 6), 5 \to (-178, 14), 6 \to (164, 3), 7 \to (-66, 8), 8 \to (281, 0), 9 \to (20, 5), 10 \to (-83, 10), 11 \to (-132, 12), 12 \to (-98, 11), 13 \to (-204, 15), 14 \to (172, 2), 15 \to (-70, 9)$.

	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9	T_{10}	T_{11}	T_{12}	T_{13}	T_{14}	T_{15}	T_{16}
T_1	∅	7	15	−5	−1	−6	4	4	18	3	−11	−2	1	−4	21	−8
T_2	∅	∅	−2	−8	−15	−12	−4	−10	11	−8	−10	−4	−8	−21	−3	−13
T_3	∅	∅	∅	−22	−19	−26	−14	−12	5	−9	−21	−19	−23	−24	3	−22
T_4	∅	∅	∅	∅	−4	4	2	8	23	−7	−7	6	12	−5	23	−2
T_5	∅	∅	∅	∅	∅	0	11	7	20	1	3	7	−1	−3	12	6
T_6	∅	∅	∅	∅	∅	∅	7	8	30	−6	−3	−3	3	−4	24	0
T_7	∅	∅	∅	∅	∅	∅	∅	−6	22	−7	−14	−2	−8	−24	11	−9
T_8	∅	∅	∅	∅	∅	∅	∅	∅	17	−2	−8	−9	−4	−24	16	−6
T_9	∅	∅	∅	∅	∅	∅	∅	∅	∅	−14	−28	−28	−28	−30	−7	−28
T_{10}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	3	7	6	−10	12	5
T_{11}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	8	2	−13	26	7
T_{12}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	1	−18	24	−7
T_{13}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−15	22	−6
T_{14}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	24	−3
T_{15}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−28
T_{16}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅

Figure 19: Generators of batch $\langle G_2 \rangle$ generated PRNGs of these sixteen types. The scores $R_{\texttt{score}}$ and corresponding ranks for these PRNGs, by index order, are $0 \to (-36, 6), 1 \to (114, 3), 2 \to (216, 2), 3 \to (-88, 10), 4 \to (-102, 12), 5 \to (-96, 11), 6 \to (43, 4), 7 \to (19, 5), 8 \to (309, 0), 9 \to (-72, 9), 10 \to (-126, 14), 11 \to (-39, 7), 12 \to (-48, 8), 13 \to (-216, 15), 14 \to (236, 1), 15 \to (-114, 13)$.

	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9	T_{10}	T_{11}	T_{12}	T_{13}	T_{14}	T_{15}	T_{16}
T_1	∅	−5	−1	−20	−16	−12	−4	−16	7	−10	−16	−22	−24	−20	1	−14
T_2	∅	∅	4	−13	−21	−7	2	−14	13	−8	−6	−15	−9	−20	9	−6
T_3	∅	∅	∅	−21	−23	−24	−4	−18	12	−11	−25	−25	−28	−17	6	−21
T_4	∅	∅	∅	∅	−3	3	8	8	26	2	3	−14	−3	1	29	7
T_5	∅	∅	∅	∅	∅	−2	26	9	26	1	8	−2	−1	−4	18	12
T_6	∅	∅	∅	∅	∅	∅	18	4	28	−2	2	−9	−3	−6	26	12
T_7	∅	∅	∅	∅	∅	∅	∅	−10	11	−12	−5	−18	−20	−24	10	−8
T_8	∅	∅	∅	∅	∅	∅	∅	∅	26	7	1	−10	−8	−17	24	9
T_9	∅	∅	∅	∅	∅	∅	∅	∅	∅	−14	−24	−28	−30	−23	−3	−22
T_{10}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−5	−5	0	−11	16	9
T_{11}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−6	−8	−5	23	9
T_{12}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	9	0	28	13
T_{13}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	4	29	13
T_{14}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	21	14
T_{15}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−22
T_{16}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅

Figure 20: Generators of batch $\langle G_3 \rangle$ generated PRNGs of these sixteen types. The scores $R_{\texttt{score}}$ and corresponding ranks for these PRNGs, by index order, are $0 \to (172, 3), 1 \to (86, 5), 2 \to (202, 2), 3 \to (-121, 11), 4 \to (-154, 12), 5 \to (-112, 10), 6 \to (122, 4), 7 \to (-69, 8), 8 \to (293, 0), 9 \to (-51, 7), 10 \to (-80, 9), 11 \to (-204, 15), 12 \to (-171, 13), 13 \to (-177, 14), 14 \to (259, 1), 15 \to (5, 6)$.

	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9	T_{10}	T_{11}	T_{12}	T_{13}	T_{14}	T_{15}	T_{16}
T_1	∅	17	24	−1	−13	2	16	11	26	2	5	−5	1	−3	25	13
T_2	∅	∅	5	−17	−16	−6	7	2	9	2	−9	−6	−7	−13	13	−11
T_3	∅	∅	∅	−24	−22	−24	2	−8	3	−10	−15	−24	−26	−22	10	−13
T_4	∅	∅	∅	∅	−6	6	27	16	30	2	13	−6	−5	−4	28	11
T_5	∅	∅	∅	∅	∅	10	22	15	28	9	12	5	6	11	28	14
T_6	∅	∅	∅	∅	∅	∅	26	15	28	−1	6	−7	−7	−6	23	9
T_7	∅	∅	∅	∅	∅	∅	∅	−14	3	−20	−14	−26	−26	−27	−4	−13
T_8	∅	∅	∅	∅	∅	∅	∅	∅	16	−6	−10	−19	−16	−23	18	−8
T_9	∅	∅	∅	∅	∅	∅	∅	∅	∅	−22	−23	−28	−29	−24	−3	−24
T_{10}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−6	−7	−1	−11	16	5
T_{11}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−10	−10	−14	22	5
T_{12}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	6	−4	28	14
T_{13}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	4	30	16
T_{14}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	26	10
T_{15}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−21
T_{16}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅

Figure 21: Generators of batch $\langle G_4 \rangle$ generated PRNGs of these sixteen types. The scores $R_{\texttt{score}}$ and corresponding ranks for these PRNGs, by index order, are $0 \to (-120, 10), 1 \to (64, 5), 2 \to (202, 3), 3 \to (-154, 11), 4 \to (-217, 15), 5 \to (-98, 9), 6 \to (241, 2), 7 \to (85, 4), 8 \to (296, 0), 9 \to (-40, 8), 10 \to (-34, 7), 11 \to (-177, 14), 12 \to (-164, 12), 13 \to (-172, 13), 14 \to (281, 1), 15 \to (7, 6)$.

	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9	T_{10}	T_{11}	T_{12}	T_{13}	T_{14}	T_{15}	T_{16}
T_1	∅	20	22	−4	−1	−4	14	−2	20	18	1	−4	−4	4	10	3
T_2	∅	∅	7	−5	−13	−9	4	−16	11	0	−8	−14	−17	−24	4	−2
T_3	∅	∅	∅	−21	−11	−24	−6	−20	4	−18	−20	−26	−24	−20	−7	−19
T_4	∅	∅	∅	∅	4	−7	17	−2	24	2	2	−1	−3	−4	19	2
T_5	∅	∅	∅	∅	∅	−6	4	−6	20	9	−5	−2	−4	−8	8	4
T_6	∅	∅	∅	∅	∅	∅	18	1	27	1	2	2	4	1	14	4
T_7	∅	∅	∅	∅	∅	∅	∅	−18	13	−3	−19	−21	−16	−22	1	−11
T_8	∅	∅	∅	∅	∅	∅	∅	∅	26	15	−3	−5	−8	−4	20	8
T_9	∅	∅	∅	∅	∅	∅	∅	∅	∅	−13	−18	−28	−26	−26	−13	−16
T_{10}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−6	−3	−8	−20	2	−2
T_{11}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	2	−4	−12	16	3
T_{12}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−6	−12	24	10
T_{13}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−8	24	4
T_{14}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	20	11
T_{15}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	−16
T_{16}	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅	∅

Figure 22: Generators of batch $\langle G_5 \rangle$ generated PRNGs of these sixteen types. The scores $R_{\texttt{score}}$ and corresponding ranks for these PRNGs, by index order, are $0 \to (-93, 10), 1 \to (102, 4), 2 \to (241, 1), 3 \to (-83, 9), 4 \to (-35, 7), 5 \to (-124, 13), 6 \to (147, 3), 7 \to (-112, 11), 8 \to (285, 0), 9 \to (48, 5), 10 \to (-79, 8), 11 \to (-116, 12), 12 \to (-136, 14), 13 \to (-186, 15), 14 \to (158, 2), 15 \to (-17, 6)$.

Primitive structures **n2m**,**ssino**, and **optri** consistently rank highest in these trials, outranking generators such as **gg** and **pmulti** despite their design of attempting their algorithmic definitions of uniform distribution. It seems **gg**'s auto-correcting procedures and **pmulti**'s stochastic partitioning of its superrange, so that each of its parameter generators output values in a unique subrange, did not achieve higher comparative randomness. The structural complexity of **idforest** nevertheless places it as an average primitive structure for generator randomness. The computation complexity of **afs** is relatively high due to its intermittent centroid adjustment process, but that complexity translates to an average to below average primitive structure in randomness, according to

the metric output. Primitive structure **rch** shows a high variance in ranking, most likely because of the source generator used to produce generators of that type. That, alone, weakly implies a perceptible homomorphism of **rch** with its source and parameter generators.

8 Considering the Cryptanalyst

Modern-day pattern-matching capabilities of modern electronic systems comprise the bulk of the kernel of automated cryptanalysis. There are some elementary mathematical principles that are appropriate to mention for contexts of secure (appearing random) number generation. Some general scenarios where changing the underlying number generator are also described in this section.

8.1 Potential Entropy of a Pseudo-Random Number Generator

The term "hardcore predicate" is very much applicable to studies on a pseudo-random number generator's potential entropy. Although the field of physics is pre-eminent in the study of entropy, phenomena with a physical basis, the term has also become pertinent to activity from software, such as the class of computer-operated pseudo-random number generators. Recall Definition 3.1 for a form-fitter. The broad categories of attributes for a pseudo-random number generator, a class of finite state automata, are its form F, exemplification of form E_F, the objective function/s O_F of the form, and the algorithm A that utilizes (F, E_F, O_F) to output numbers that are to appear random. Hardcore predicates, in the context of enabling their host generators to achieve highly unpredictable output values, are supposed to maximize entropy (the physics counterpart to PRNG randomness), under the constraints of the rules of the generator's form, by operating with (E_F, O_F). There is usually interactive computation between hardcore predicates $\{q_x\}$ and the generator seed, the values used to instantiate the generator, when they are not one and the same. Suppose there is a metric M on randomness. Metric M could be a singleton or a set of metrics. To maximize randomness of generator G from hardcore predicates, then, could be expressed as such:

$$\arg \max_{\{q_x\} \in \texttt{SPACE}} (\left| M(G_{\{q_x\}}, G_{\{q_y\}}) \equiv 1 \right|); \{q_y\} \in \texttt{SPACE} \backslash \{q_x\}.$$

$$G_x := \texttt{generator G with hardcore predicates x}.$$

Categorical entropy, $\mathbb{E}$, as defined in Qual Test, measures the spatial change of number location across a sequence, by first labelling those numbers according to partitioning of the (minumum,maximum) range. The rationale is that a number sequence, of a super-range, should contain contiguous numbers of great label difference. This measure, similar to unidirectional weighted point distance $\mathbb{U}_{pd}$, has the flaw of the best-scoring solution being a simple alternation of

extremes,

```
min,max,min,max,....
```

Without balancing measures such as coverage, $\mathbb{C}$, any sequence of alternating extremes (binaries) would be measured as the most random under Qual Test.

The test results from Section 7 provide some relevant examples on poor hardcore predicates, if generator output appearance of randomness is the one objective. For example, a **qval** outputs values from quotients on pairs of an immutable integer sequence $\overrightarrow{I}$. The primitive structure does not operate for any objective besides from using its parameter generator to choose two integers $i_0, i_1 \in \overrightarrow{I}$, storing the value v,

$$v := \frac{i_1}{i_0} - \lfloor \frac{i_1}{i_0} \rfloor,$$

into its output cache, a concatenation of stringized integers. The design of **qval** strongly links its output to $\overrightarrow{I}$, in such ways that its parameter generator cannot surpass certain metrics of randomness, since the parameter generator's output values go towards selecting numbers for the quotient operation. The integer sequence for **optri**, similarly, is a causal variable for the quality of entropy its objective function O_F is granted, in the process of number generation. The period/s of a **shadow** generator over a file that contains only one unique value would be detectable by a capable algorithm over a fraction of a quarter-million samples.

8.2 Complicating the Image

If the objective of number generation that is supposed to appear random is simply the set cardinality of the output, there are some rules that can guarantee its success. This objective, before moving on to describing those rules, is not the best gauge of measurement. As an example against value uniqueness serving as an effective sole criterion, consider a uniformly distributed sequence of numbers $\overrightarrow{S}$. This could be the $(r_1 - r_0)$ integers from the range $[r_0, r_1)$, in any order. Then any permutation P of $\overrightarrow{S}$,

$$\overrightarrow{S}' := P(\overrightarrow{S}, |\overrightarrow{S}|),$$

has equal "randomness". In consideration and review of elementary derivative patterns, the "kernel" of $\overrightarrow{S}'$ would be relatively easy to obtain. The same numbers are output from any $\overrightarrow{S}'$, only possibly in a different order.

When set cardinality of generator output is a value used in the manner of differentials, patterns that are less discernible than permutations of the same numbers can arise. Consider the problem of adding k linear congruential generators together for another generator G,

$$G := G^{(1)}_{\texttt{lcg}} + G^{(2)}_{\texttt{lcg}} + \cdots + G^{(k)}_{\texttt{lcg}}.$$

The set cardinality of G's output is not always

$$\prod_{i=1}^{k} \left| G_{\texttt{lcg}}^{(i)} \right|.$$

This product value would be ideal for uniqueness of output values, especially to an observer that has no reference to those output values, whether those output values themselves or partials, such as values from a subset of the k generators. There would be a lack of understanding of the form then, from the observer's standpoint. There are three rules to follow in attempting to finding any k linear congruential generators that would produce an additive composition with an output set cardinality of $\prod_{i=1}^{k} \left| G_{\texttt{lcg}}^{(i)} \right|$.

(1) For any two generators $G_{\texttt{lcg}}^{(i)}, G_{\texttt{lcg}}^{(j)}$, the intersection of their output is null,

$$G_{\texttt{lcg}}^{(i)} \bigcap G_{\texttt{lcg}}^{(j)} = \varnothing.$$

(2) For any two generators $G_{\texttt{lcg}}^{(i)}, G_{\texttt{lcg}}^{(j)}$, their period lengths are co-prime.

(3) For any output value v from G, the cumulative generator from the k linear congruential generators, there is exactly one sequence of numbers $\overrightarrow{S}_v$,

$$\overrightarrow{S}_v = \langle G_{\texttt{lcg}}^{(1)}(), G_{\texttt{lcg}}^{(2)}(), \ldots, G_{\texttt{lcg}}^{(k)} \rangle,$$

that sums to v.

The first rule may not be necessary to accomplish the maximum output set cardinality. However, it is often linked to the third rule, for a non-null intersection of (minumum,maximum) ranges of any two generators increases the probability of duplicate sums.

8.3 When the Complexity Fails

Section 7.5 exemplified, through the metrics of *Seqbuild*'s randomness tests, that the assumption of there being a hierarchy of randomness for pseudo-random number generator generation is one that cannot be universally accomplished. Objective metric functions constitute the first half of the reason for this truth.

Definition 8.1 (Objective Comparator Metric (Mathematical Measure))**.** A function $\mathcal{M}$ is associated with a set of measures $\mathbb{M} = \{m_1, \ldots, m_k\}$,

$$m_i : \{\overrightarrow{V}\} \longrightarrow \mathcal{R}^*,$$

and an immutable set of values P_V for its parameter set P. The composite function $F_{\mathbb{M}}$ of $\mathbb{M}$, such that each m_i measures vector $\overrightarrow{V}$, is

$$F_{\mathbb{M}} : (\mathbb{M}, \{\overrightarrow{V}\}) \longrightarrow \mathcal{R}^*.$$

Then the function $\mathcal{M}$,

$$\mathcal{M} : (F_{\mathbb{M}}, \{\overrightarrow{V}\}, \{\overrightarrow{V}\}) \longrightarrow \{1, 0, -1\},$$

is an *objective comparator metric* if for any two vectors $\overrightarrow{V}_1$ and $\overrightarrow{V}_2$,

$$\mathcal{M}(F_{\mathbb{M}}, \overrightarrow{V}_1, \overrightarrow{V}_2)$$

is a constant value $r \in \{1, 0, -1\}$, meaning it remains the same value throughout temporal use.

The construction provided in Definition 8.1 supposes a transitive relation of the metric's composite function $F_{\mathbb{M}}$ over its measures, although each $m_i \in \mathbb{M}$ does not necessarily have to be transitive, with respect to the comparisons provided by $\mathcal{M}$. Transitivity is not a necessary requirement, only for there to be one or more global best solutions, for the validity of this next lemma.

Lemma 8.1. *Given any objective comparator metric $\mathcal{M}$, there exists a limit to the measurable randomness, based on the output number sequence, of the end pseudo-random number generator of a chain-generated sequence of pseudo-random number generators, starting from an arbitrary source generator G_0.*

Proof. Suppose $\mathcal{M}$, associated with a set of measures

$$\mathbb{M} = \{m_1, \ldots, m_k\},$$

is an objective comparator metric for comparing the randomness of any two pseudo-random number generators G_1 and G_2, via their contiguous output sequences $\overrightarrow{S}_1$ and $\overrightarrow{S}_2$, $|\overrightarrow{S}_1| = |\overrightarrow{S}_2|$ of arbitrary length l. There is a set of global best sequences B_i for every $m_i \in \mathbb{M}$, and the set of global best sequences for $\mathbb{M}$ could be calculated by a function U that considers every B_i,

$$U(\mathbb{M}, \{B_1, B_2, \ldots, B_k\}, l) = B_{\mathbb{M}}.$$

Let G_0 be a source generator for a chain-generated sequence of PRNGs $\overrightarrow{G} = \langle G_1, G_2, \ldots \rangle$. For any generator G_i, $i \geqslant 0$, there is a set $\{V_{G_i}\}$ of l-sequences reachable from a generator generated by G_i. That is, any sequence in this set $\{V_{G_i}\}$ can be generated by at least one host generator G_{i+1} that G_i generated. Denote $U[G_i](\mathbb{M}, \{B_1, B_2, \ldots, B_k\}, l)$ as the set of global best sequences reachable from the set $\{G_1^{(i)}, G_2^{(i)}, \ldots, G_m^{(i)}\}$ of generators possible for G_i to generate. If

$$U(\mathbb{M}, \{B_1, B_2, \ldots, B_k\}, l) \bigcap U[G_0](\mathbb{M}, \{B_1^{(0)}, B_2^{(0)}, \ldots, B_{k_0}^{(0)}\}, l) = \varnothing,$$

then no global best sequence of length l can be output from any generator directly generated by G_0. There are two broad classes of outcomes, regardless, for G_0. Chain-generation of PRNGs, starting from G_0, can yield a generator G_i that outputs

$$\max_{u_i \in U[G_i](\mathbb{M}, \{B_1, B_2, \ldots, B_k\}, l)} (F_{\mathbb{M}}(u_i)) \geqslant$$

$$\max_{u_{i-1}\in U[G_{i-1}](\mathbb{M},\{B_1,B_2,\ldots,B_k\},l)}(F_{\mathbb{M}}(u_{i-1})) \geqslant$$

$$\max_{u_{i-2}\in U[G_{i-2}](\mathbb{M},\{B_1,B_2,\ldots,B_k\},l)}(F_{\mathbb{M}}(u_{i-2})) \geqslant \ldots$$

$$\max_{u_0\in U[G_0](\mathbb{M},\{B_1,B_2,\ldots,B_k\},l)}(F_{\mathbb{M}}(u_0)).$$

In such cases of increasing randomness by $\mathcal{M}$, there must be a G_i reached by the algorithm for

$$U[G_i](\mathbb{M},\{B_1^{(i)},B_2^{(i)},\ldots,B_{k_i}^{(i)}\},l)\bigcap B_{\mathbb{M}} \neq \varnothing.$$

Any additional generator G_{i+1}, from G_i, would only risk output of a lesser randomness by metric $\mathcal{M}$.

In cases where no generator from a chain-generated sequence starting with G_0 can generate any l-sequences from $B_{\mathbb{M}}$, there is still one or more relatively best sequence reachable, at any end generator G_i during the PRNG chain-generation from G_0. Either these best sequences by G_i are reached, or another set of best sequences are attempted by the next G_{i+1}. However, the best sequence reachable at any index during PRNG chain-generation can be reached. But by metric $\mathcal{M}$, these reachable best sequences at any subsequent index i can only improve to having non-null intersection with $B_{\mathbb{M}}$. □

This proof provides practical principle for placing limits on increasing pseudo-random number generator complexity, via chain-generation. The computation complexity increases with every subsequent embedded generator, while measurable randomness under any metric $\mathcal{M}$ has upper limits such that, under certain constraints caused by the specific source PRNG used in chain generation, local upper limits far below the initial potential can become imposed.

In contradicting situations of the objective metric being influenced by the relative values of their comparisons, the metric would output inconsistent results, results that selectively reference prior sequences of comparison. These comparisons, over extensive iterations of comparisons, essentially consist of looparound rankings, and are more immediately useful for simulating pathological moreso than high-performing automata.

The general flaw with objective comparator metrics is that they base their comparisons on qualities not entirely drawn from relation between any two. There is at least one global best, so that if there are agents that know the global best and/or the means to achieve that global best, then that would be it. In context of pseudo-random number generation, that would certainly imply some number sequences, of any arbitrary length, are absolutely most random. Then those number sequences, after having been known or the means to generate them known by a significant number of entities, would not actually be random.

Suppose that an entity designed a number generator system that is supposed to be random. This system is effectively a simple hash table that outputs number seqences, categorized as "random", of arbitrary length when prompted. The computation complexity is $O(1)$, and the memory complexity grows as

exponentially large as the hash table database does. In the context of perfect and complete information, this would be poor system design. A third-party, after achieving access to the system hosting the hash table, sees these number sequences for what they are, in identity. The perceived randomness would then be no more. This situation, although it should be more rare than not, does lead to questions on preferential balance of computation complexity in tandem with "natural randomness".

When "randomness" is primarily defined as cause of change, then a basic formula for measuring randomness is differentials to PRNGs $\{G_{\texttt{NOT}}\}$ labelled "not random". The best PRNG, given this context, is

$$\arg \max_{G \in \texttt{SPACE}} (\sum_{G' in \{G_{\texttt{NOT}}\}} \left| G - G' \right|).$$

The risks posed by a metric known for calculating most random number sequences, out of a candidate set, are of a similar basis to that of a known hash table of number sequences, labelled "random". When the metric is known, a sufficiently capable adversarial party would conduct a brute-force search over multi-dimensional number spaces to obtain these "most random" number sequences.

8.4 PRNG Phenotypic Objective

A pseudo-random number generator, in contexts where secrecy is prioritized against any observer not of the PRNG serving party, should be unpredictable. The concept of uniform distribution is perhaps the most important in proper practice of principle for unpredictability.

Definition 8.2 (Uniform Distribution)**.** [Dub] A probability distribution P has the property of uniform distribution if its probability density function is

$$Pr(x) \begin{cases} \frac{1}{b-a} \texttt{ if } a \leqslant x \leqslant b, \\ 0 \texttt{ otherwise,} \end{cases}$$

in continuous data of range $[a, b]$. In discrete data of label set L,

$$L = \{l_1, l_2, \ldots, l_k\},$$

the probability density function is

$$Pr(x) \begin{cases} \frac{1}{|L|} \texttt{ if } x \in L, \\ 0 \texttt{ otherwise,} \end{cases}$$

Suppose an adversarial party were to obtain n contiguous numbers from a PRNG G. They are able to use those n to construct an emulator PRNG G_2 that outputs exactly that sequence of n numbers, up to index n. If G satisfies the property of uniform distribution for its output values, G_2 should not be able be output the next k numbers of G with a passing level of accuracy. This condition is more formally described in the next definition.

Definition 8.3 (Anti-Emulating Uniform Distribution (Pseudo-Random Number Generation))**.** An adversarial party P constructs an emulator G_2 to emulate a pseudo-random number generator G over n of its contiguous outputs, using an arbitrary map $M_{\mathcal{C}}$ of category-to-functions,

$$M_{\mathcal{C}} : C \longrightarrow \mathcal{F},$$

$$c \longrightarrow f,$$

$$c \in \mathcal{C} \text{ and } f \in \mathcal{F},$$

`c a category, f a function with image in the relevant numerical space.`

For the next k contiguous outputs $\overrightarrow{S}$ from G, the probability that G_2 is able to output $\overrightarrow{S}$ is at most

$$\frac{1}{|M_{\mathcal{C}}|} * \frac{1}{p(n)},$$

`p a polynomial that outputs strictly positive integers.`

If map $M_{\mathcal{C}}$ contains a function that can be used by G_2 to entirely output $\overrightarrow{S}$, then $p(n) = 1$. Suppose instead that P believes $M_{\mathcal{C}}$ contains a composition function $f_{\texttt{comp}}$ such that

$$f_{\texttt{comp}} := M_{\mathcal{C}}[c_{i_1}] \circ \cdots \circ M_{\mathcal{C}}[c_{i_j}],$$

$$2 \leqslant j \leqslant |M_{\mathcal{C}}|.$$

And $f_{\texttt{comp}}$ would produce those k values. Then $p(n)$ is approximately

$$2^{|M_{\mathcal{C}}|}.$$

Otherwise, $p(n)$ would be inclined towards an exceedingly large positive integer.

Additionally, Party P may not be constrained to using any map of category-to-functions. In those cases, the conventional definition of uniform distribution is applied to produce the probability at most

$$\Big(\frac{1}{\max(G) - \min(G)}\Big)^k.$$

A cryptographically secure PRNG would demand the emulating party to frequently revise the hardcore predicates of their emulator PRNG. These revisions would also have to be challenged by uniform distribution.

Definition 8.4 (Emulator Hypothesis Revision Challenge (Uniform Distribution))**.** An adversarial party P that attempts to emulate a cryptographically secure PRNG G will have to revise their hypothesis H for the hardcore predicates $\{\mathcal{C}\}$ of their emulator G_2. Party P chooses to revise H to subset $C' \subseteq \mathcal{C}_{\texttt{super}}$, $\mathcal{C}_{\texttt{super}}$ a mutable superset that originally contained $\{\mathcal{C}\}$ when it was still acted for a valid hypothesis, by P's belief. Then H has a probability of containing

the hardcore predicates able to emulate the last n outputs, before the point of revision, and the next k outputs with a probability at most

$$\frac{1}{2^{|\mathcal{C}_{\text{super}}|} * p(n+k)},$$

`p a polynomial that outputs strictly positive integers.`

It is doubtful that any primitive structure of *Seqbuild* can independently achieve anti-emulating uniform distribution. This is besides the fact that the primitive structure is most likely deterministic, therefore able to output exactly one possible l-sequence at any output index. From the standpoint of the observer unknowing of the actual PRNG behind the output, the probable PRNG output is, nevertheless, not uniform when the generator's algorithm constrains its probable output to specific numerical subspaces, based on its hardcore predicates. This ideal of uniform distribution for probable PRNG output has greater likelihood of being achieved when PRNG compositions are used to produce the central PRNG. Given an arbitrary number sequence $\overrightarrow{S}$ and the set of all generators $\{G\}$ that can generate $\overrightarrow{S}$, the subset $\{G'\}$, $\{G'\} \subset \{G\}$, would be able to produce the set of l-sequences $\{\overrightarrow{S}_l\}$ for their next independent outputs, and

$$\{\overrightarrow{S}_l\} \equiv \mathcal{R}^l$$

cannot be guaranteed without comprehensive pre-computation.

8.5 One Approach to PRNG Emulation

There are an uncountable number of methodologies to emulate an arbitrary PRNG G. Linear congruential generators (LCGs), for example, can be used to roughly produce any output sequence $\overrightarrow{S}$ from G.

Definition 8.5 (Linear Combination of Linear Congruential Generators)**.** A function f is a linear combination of linear congruential generators $G^{(1)}_{\texttt{lcg}}, \ldots, G^{(k)}_{\texttt{lcg}}$ if it is of the form,

$$f := c_k G^{(k)}_{\texttt{lcg}} + c_{k-1} G^{(k-1)}_{\texttt{lcg}} + \cdots + G^{(1)}_{\texttt{lcg}},$$

with every output equal to

$$c_k G^{(k)}_{\texttt{lcg}}() + c_{k-1} G^{(k-1)}_{\texttt{lcg}}() + \cdots + G^{(1)}_{\texttt{lcg}}().$$

Lemma 8.2. *Any contiguous number sequence $\overrightarrow{S}$ from a pseudo-random number generator can be replicated by a linear combination of modulated floored floating point linear congruential generators.*

Proof. Search algorithm A starts by selecting the best floating point linear congruential generator $G_{\texttt{best}}$,

$$G_{\texttt{best}} := \arg \min_{G \in \texttt{SPACE}} \Big(\sum (\Big| G(|\overrightarrow{S}|) - \overrightarrow{S} \Big|) \Big).$$

Then for the remaining error ϵ,

$$\epsilon := \overrightarrow{S} - G_{\texttt{best}}(|\overrightarrow{S}|),$$

A reduces ϵ by its combinative strategy of targetting absolute mean error value, across the indices of ϵ and elimination of erroneous indices of ϵ. Any additional linear congruential generator that A adds to its running solution undergoes a modulo operation by the ranged modulus $[-1, 2]$ in the case of targetting absolute mean error and by 2 for erroneous index elimination. These additional linear congruential generators, when targetting absolute mean error, act as ternary deltas in reducing ϵ. For a running error of absolute mean m for $\texttt{abs}(\overrightarrow{S})$ and non-zero index set I, A adds an element to its linear combination, the running solution, that is

$$m\big(-1 + (\lfloor G^{(j)}_{\texttt{lcg}} \rfloor \mod 3)\big),$$

$$G^{(j)}_{\texttt{lcg}} := \arg\min_{G \in \texttt{SPACE}} \Big(\sum(\Big| -1 + \lfloor (G(|\overrightarrow{S}|)\rfloor) \mod 3 - T_0(\overrightarrow{S})\Big|)\Big).$$

The vector $T_0(\overrightarrow{S})$ is defined as

$$T_0(\overrightarrow{S})[i] = \begin{cases} 1 \texttt{ if } \overrightarrow{S}[i] > 0, \\ \texttt{-1 if } \overrightarrow{S}[i] < 0, \\ 0 \texttt{ otherwise.} \end{cases}$$

When a significant majority of the erroneous indices belonging to the original ϵ have been reduced, A shifts mostly to elimination of erroneous indices. For any ordered index set $I' \in I$, I the indices of the running error, A chooses another $G^{(j+k)}_{\texttt{lcg}}$ to add to its running solution, such that the first non-zero index from $(\lfloor G^{(j+k)}_{\texttt{lcg}} \rfloor \mod 2)$ is at $I'[0]$, and the periodicity p of $(\lfloor G^{(j+k)}_{\texttt{lcg}} \rfloor \mod 2)$, with respect to outputting 1, is approximately $I'[1] - I'[0]$, or the p that would most affect change to the indices of I'. The coefficient for $(G^{(j+k)}_{\texttt{lcg}} \mod 2)$ is $\epsilon[I'[0]]$.

A eventually finds a linear combination of modulated floored floating point LCGs that produces $\overrightarrow{S}$,

$$G := m_0\big(-1 + (\lfloor G^{(0)}_{\texttt{lcg}} \rfloor \mod 3)\big) + \cdots +$$

$$m_j(\lfloor G^{(j)}_{\texttt{lcg}} \rfloor \mod 2) + \cdots + m_{(j+k)}(\lfloor G^{(j+k)}_{\texttt{lcg}} \rfloor \mod 2) + G_{\texttt{best}}.$$

□

8.6 Signature Patterns in PRNGs

Generator signatures of *Seqbuild*'s PRNGs are not thoroughly recorded in this paper. Attributes of output sequences such as ternary vectors, unidirectional weighted point distance, $\mathcal{K}$-complexity and coverage, as defined for the Qual Test, seem to have much overlap between the different primitive structures.

That is, scores such as from the $\mathcal{K}$-complexity measure for a PRNG cannot be attributed to its primitive structure, for its primitive structure is not restricted to outputting number sequences with scores for these measures that fall in some narrow range. Instead, Qual Test and Chain Test measures of a PRNG can identify that specific PRNG, not the primitive structure of it. PRNG compositions further reinforce naive observer belief of uniform distribution on the probable PRNG of observed output sequence.

Definition 8.6 (Categorical Inseparability (Machine-Learning Recognition Challenge))**.** Let Q be the problem of interest that concerns object $\mathcal{Y}$ and $M_{\texttt{c2c}}$ be a component-to-category map for $\mathcal{Y}$,

$$M_{\texttt{c2c}} : \{Y_c\} \longrightarrow C,$$

$$\prod\{y_c\} = \mathcal{Y}; \{y_c\} \in \{Y_c\}.$$

Additionally, problem Q is associated with set of matters $\{U_Q\}$, such that further comprehension C of Q positively corresponds with more resolution R on $\{U_Q\}$,

$$\Uparrow C(Q, M_{\texttt{c2c}}, \{y_c\}) \implies \Uparrow R(\{U_Q\}).$$

Suppose entity W is to fixate on $\mathcal{Y}$, in order to solve Q by resolving $\{U_Q\}$. W cannot apply $M_{\texttt{c2c}}$ on any componentization $\{y_c\}$ of $\mathcal{Y}$ for passing resolution on Q,

$$\nexists\{y_c\} \in \{Y_c\} \texttt{ such that } C(Q, M_{\texttt{c2c}}, \{y_c\}) \implies [R(\{U_Q\}) \geqslant t];$$

$$\texttt{t the minumum threshold for passing.}$$

Then the object $\mathcal{Y}$ is *categorically inseparable* by entity W under $M_{\texttt{c2c}}$. If W cannot find any component-to-category map to satisfy

$$[R(\{U_Q\}) \geqslant t],$$

$\mathcal{Y}$ is categorically inseparable by W.

PRNGs such as **idforest** and **rch**, without output constraints from ranged modulii, are prone to the exploding value problem, especially due to deltas of their polynomial functions that use cache numbers of monotonically increasing absolute value. Others, such as **optri**, exhibit exploding values when their primary forms likewise reform with monotonically increasing absolute values. Exploding values may seem random to an observer, but their occurrence points to a combination of probable functional forms, polynomials one of the more common ones, and algorithm procedures that do not place limits on function parameters. Yet, exploding values may exhibit predictable patterns from weak homomorphisms and modulo value preservation.

Differentials of reference sequences also act as signatures for PRNGs, especially ones that predominantly rely on them for calculating output numbers. For example, the **qval** generator is relatively predictable, if one were to know

its reference sequence, a critical component of its instantiation seed. A known differential of the reference sequence narrows down the probable search space to a mere fraction of what it was. A known differential sequence for the **optri** generator, especially when its forward and backward functions are subtraction and addition.

8.7 Knowing When to Change

This paper elaborated on the *Seqbuild* system's randomness measures, provided an overview of its primitive PRNG structures, and stated definitions for principles on how number sequences can appear random to an observer. From the cryptanalytic perspective, a more technical inquiry into vulnerabilities of electronic systems would involve review of hardware specifications and the vulnerabilities of their design. PRNGs, from the outside, are still vulnerable to cryptanalysis, especially when the brute-force search process for the underlying PRNGs is constrained to a relatively small fraction over the uniform distribution. Patterns from weak homomorphisms can be detected by adequately programmed and trained computing systems. Large hash tables can be constructed by mass computing networks, so that if a number sequence does not map to the most probable subset of PRNGs producing that number sequence, then the table entries could still serve as dynamic lattice points for a basis to find the suspect PRNGs.

When an adversarial party does not have to rely on brute-forcing, especially from the outside of the PRNG system, said system could, in practice, be much more known through exploitation of reliable vulnerabilities, namely memory attacks.

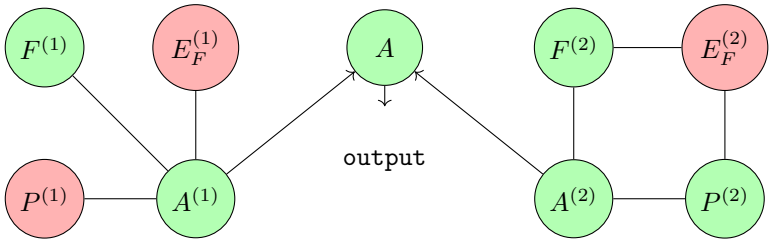


Figure 23: A nodal diagram of a PRNG that is a composition of two PRNGs, denoted with superscripts 1 and 2. Red nodes indicate third-party knows their values, through a memory attack. For third-party to predict the output of this generator with greater accuracy, $P^{(2)}$ would have to be known. Additionally, if the forms $F^{(?)}$ of the PRNGs are known, third-party would be able to know what and when $P^{(?)}$ changes its second-order seed to.

This paper steers away from describing the technical pursuit of executing memory attacks. There are some pertinent principles on these attacks, of interest in abstraction. Figure 23 illustrates a PRNG that is partially known by a third-party, through a memory attack. The complex entropy of the PRNG

G, a composition of two PRNGs, allows the PRNG to still output values not entirely able to be predicted by the third-party with complete accuracy. However, the third-party's knowledge of important bodies of variables such as one of the parameter generators and the component generators' exemplification of form provides a basis for the third-party to improve their hypothesis.

If the design of generator G still allows it to exhibit an entropy uncapturable by the third-party and execution of G on computer hardware does not cede any more memory over to the third-party, then principles of categorical inseparability (Definition 8.6) allow for G to remain unpredictable. This forecast of G's performance, given its compromised state, is given further support if G still satisfies anti-emulating uniformity over a significantly large distribution of candidates. Denote G_0 as the partial knowledge that the third-party knows of G. For any map f_m to calculate G from G_0,

$$f_m(G_0) = \{G_{\texttt{super}}\};$$

$$\left|\{G_{\texttt{super}}\}\right| > t,$$

$$\texttt{t a large positive number}.$$

The odds of acquiring the actual generator G out of the large candidate set $\{G_{\texttt{super}}\}$ is as slim as t makes the odds difficult. Memory attacks are available moreso for the technical difficulty of executing them. On the side of mathematics, the intellect most proficient for carrying out gainful memory attacks is that of the logic of systematic dissection and replacement, such as solving for algebraic equations.

Autonomous pseudo-random number generators would depend on P-random and T-random schemes of generating generators. If they are run on hardware of a security-focused design, memory attacks from the third-party would alert the autonomous system. The autonomous system A would know the status that it has been compromised and ideally, the qualities in memory of what the third-party intrusion obtained. A could resort to applying changes to its present PRNG, via P-random or T-random schemes, adding complexity to it using memory not tapped by the third-party. A could also free up the memory of its present generator, and start fresh. The freeing up of its memory, depending on how its architecture works for the task, could possibly boot out the third-party.

Seqbuild randomness measures of Qual, Chain, and Cycle Tests provide any PRNG information on qualities of its output. A system that is unaware of third-party intrusion could still intermittently change its central PRNG, based on periodicities concerning these randomness measures. Feedback mechanisms, between the device and its external surroundings, are a technical difficulty to implement. These mechanisms would constitute the hardware side of security. The great variance that engineering feats can build into product, from the same principle, is why attempting to describe the best practices require the most factual technical details.

Sources

[Sip06] Michael Sipser. *Introduction to the Theory of Computation Second Edition*. Thomson Course Technology, 2006.

[Gol10] Oded Goldreich. *Primer on Pseudorandom Generators*. Department of Computer Science and Applied Mathematics,Weizmann Institute of Science, Rehovot, Israel., 2010.

[LEc10] Pierre L'Ecuyer. "Random Number Generation". In: *University of Montreal* (2010).

[Sch11] Friedrich Martin Schneider. "Weak Homomorphisms Between Functorial Algebras". In: *Demonstratio Mathematica Vol. XLIV* (2011).

[Sha17] Shahriar Shahriari. *Algebra in Action*. American Mathematical Society, 2017.

[Cyp] John Cyphert. *Intro to Category Theory: Functors*. URL: `https://pages.cs.wisc.edu/~jcyphert/categoryTheoryNotes/basics/2_Functors.pdf`. (accessed: 9.01.2026).

[Dub] University College of Dublin. *Statistics: Uniform Distribution*. URL: `https://www.ucd.ie/msc/t4media/Uniform%20Distribution.pdf`. (accessed: 8.30.2026).

[JM] Daniel Jurafsky and James H. Martin. *N-gram Language Models*. URL: `https://web.stanford.edu/~jurafsky/slp3/3.pdf`. (accessed 9.01.2026).

[He25] Songhua He. "A note on a hierarchy theorem for promise-BPTIME". In: *Electronic Colloquium on Computational Complexity* (December 2025).

[I23] Punyasiri D. L. S. I. "Signature and Behavior Based Malware Detection". PhD thesis. Sri Lanka Institute of Information Technology, September 2023.

[Pha25] Richard Pham. "Some Primitive Structures in Seqbuild". In: *Github* (November 2025).

[Sch23] Julian Schennach. "Linear Congruential Generators". In: *Simon Rubinstein-Salzedo* (December 2023).

[Sul23] Michael Sullivan. *Lecture Notes for Math 620: Groups, Rings, and Fields*. San Diego State University, January 2023.